Constraint-driven API design

API design: from plausible REST to explicit contracts

Two systems, 24 steps. Each step adds one measured constraint that a plausible-looking API does not survive, and ends with the smallest reversible change.

Two systems, one decision loop

Part	ShopEasy (steps 01 to 06)	BankX (steps 07 to 24)
System	E-commerce platform, Java modular monolith with PostgreSQL	Digital banking platform, Java modular monolith with PostgreSQL
API at the start	Nouns, plural collections, nested paths and standard methods; passed code review	Plain resource API for customers, accounts, balances and payment instructions
What grows	One hidden assumption fails per step	Approvals, transfers, clearing, cancellations, exports, reconciliation
Hard rule	No payment or cancellation without customer intent	Accepted is never shown as settled; no duplicate payments
Contract rule	Existing clients keep working while a change rolls out	Existing clients keep working while a change rolls out

Four things an API change touches

Dimension	What moves	Typical safety
Contract	URLs, methods, media types, status codes, headers	Additive change, content negotiation, contract tests
Behavior	Server rules and state transitions	Atomic checks, server-owned state, feature flags
Clients	What existing clients must change or can ignore	Warn mode, old paths kept, deprecation signals
Data	Stored state, retention, keys	Retention rules, expiry jobs, rebuildable projections

A flag can switch a route back. Data written since the change cannot be switched back.

Four phases, 24 steps

Phase	Steps	Focus
1 Resources and HTTP semantics	01–06	Safe methods, conditional writes, server-owned totals and status, resource identity, paging
2 Workflow resources and contracts	07–11	Approval as a resource, advertised actions, error semantics, caching, contract evolution
3 Synchronous and asynchronous interaction	12–19	Retries, accepted work, results, notification, lifecycles, cancellation, webhooks, backpressure
4 Boundaries and distributed consistency	20–24	Resources versus modules, fan-out, interface style, reconciliation, knowing when to stop

Where each timing question is answered

Question about time	Step
The response is lost and the client retries	API12
The work is slower than the request may last	API13
The result must be found later and then expire	API14
The client must learn that the work finished	API15
The outcome comes days after the HTTP exchange	API16
A cancel races with execution	API17
A partner needs a callback	API18
Arrivals exceed what can be processed	API19
A page waits on five services at once	API21

Phase 1

Resources and HTTP semantics

Steps in this phase

Step	Forcing constraint
API01 Keep GET free of business effects	Link scanners and prefetch open cancel and capture links, so orders are cancelled and payments captured without a customer request
API02 Prevent lost updates	Two clients edit the same cart and the last write wins
API03 Take the order total from the server	Promotions, tax recalculation and price changes make a client-supplied total untrustworthy
API04 Replace status edits with server-owned transitions	Fraud review, partial capture and cancellation while pending make a client-set status wrong
API05 Give resources identity independent of their parent	Anonymous carts, shared carts, cart reassignment and several partial refunds break URLs that encode one parent
API06 Page large collections without skipping or repeating rows	The order list passes 2,000,000 rows while orders keep arriving, so deep pages slow down and rows are skipped or repeated

API01 · Resources and HTTP semantics

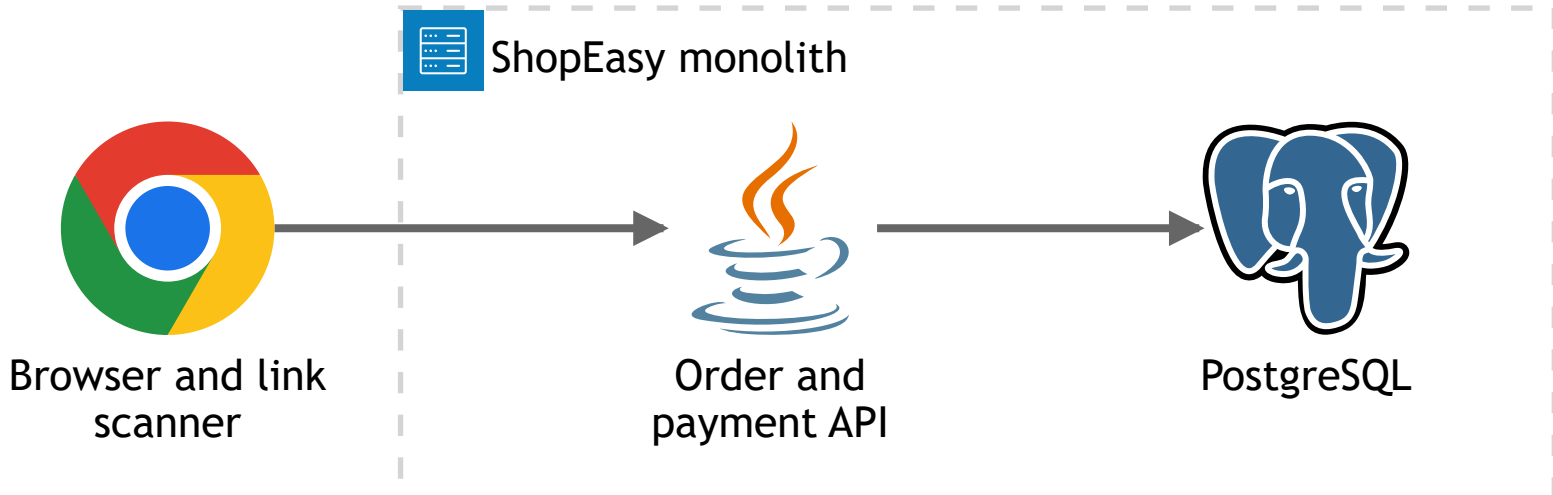
Keep GET free of business effects

Link scanners and prefetch open ShopEasy’s cancel and capture links, so orders are cancelled without a customer request: 37 orders within one hour of a campaign e-mail against a target of 0.

Cancel and capture are links that a GET follows

Component	What it is	What it does
Browser and link scanner	The customer’s browser, mail-security scanners, prefetch and proxies	Send a GET for every link in the order confirmation e-mail
Order confirmation e-mail	Message sent to the customer after checkout	Carries the cancel link GET /orders/O123/cancel
Order and payment API	Routes in the Java modular monolith behind a clean-looking OpenAPI document	Cancels the order or captures the payment when a GET arrives
PostgreSQL	Database that holds orders and payments	Stores the changed order and payment state

Current architecture



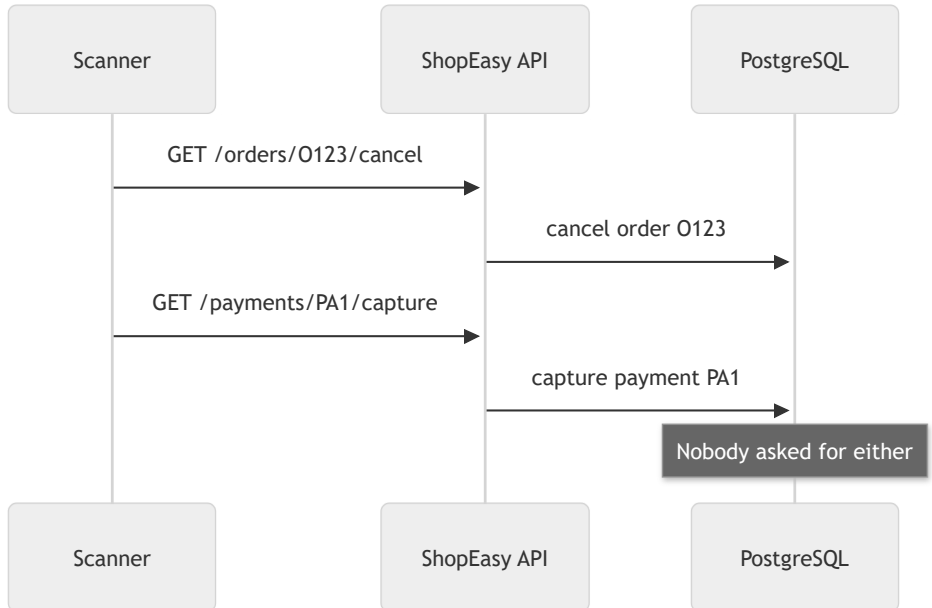
A link scanner cancels the order

A mail scanner and a browser prefetch open every link in the confirmation e-mail. GET is safe by definition, yet ShopEasy cancels the order each time.

37 orders
cancelled within one hour of a campaign e-mail

1 GET
per link, from tools as well as customers

0
state changes by GET in a log replay



Ways to stop an opened link from changing state

Solution	What it is	What it does
POST action endpoint	The cancel and capture actions move to POST requests on the same resources	Scanners and prefetch send only GET, so they never reach the action
Signed one-time GET link	The action stays on GET and the URL carries a signed, expiring token	Only a holder of the token can trigger the action, once
Confirmation page + POST	A GET shows the order page and its confirm button sends the POST	Customer intent is an explicit click, and the GET only reads

Each option pays in clicks, pages, tokens or client changes

Solution	Gains	Costs
POST action endpoint	Safe methods stay safe Scanners cannot trigger it Standard client behavior	E-mail links need a landing page Every caller changes its call POST needs its own retry safety
Signed one-time GET link	Works straight from an e-mail No client change Token expires	Scanner opens it first and uses it up GET still changes state Token leaks into logs and referrers
Confirmation page + POST	Explicit customer intent E-mail links keep working GET only reads	One more click A page and a form to build API clients get no page

Deciding signal: Whether anything other than the customer can issue the request (scanners, prefetch, proxies, retries), measured as GET requests that changed state in a replayed access log. If the action must start from an e-mail link, a confirmation page is needed in addition to the POST; a signed link is not enough while scanners exist

Smallest safe move

Actions move to POST and GET only reads

- **Scope:** Contract and behavior: the cancel and capture routes only; no schema change
- **Change:** Add POST /orders/O123/cancel and POST /payments/PA1/capture; old GET paths return the state with no side effect; the e-mail link points at the order page
- **Safety net:** Old GET paths keep answering read-only; the signed one-time link is not used, scanners consume it; action endpoints stay authenticated and owner-checked
- **Escape route:** A flag restores the old GET behavior for one release only
- **Deferred:** Confirmation page + POST when customers must cancel from an e-mail without signing in; cancellation as a resource when its outcome must be tracked

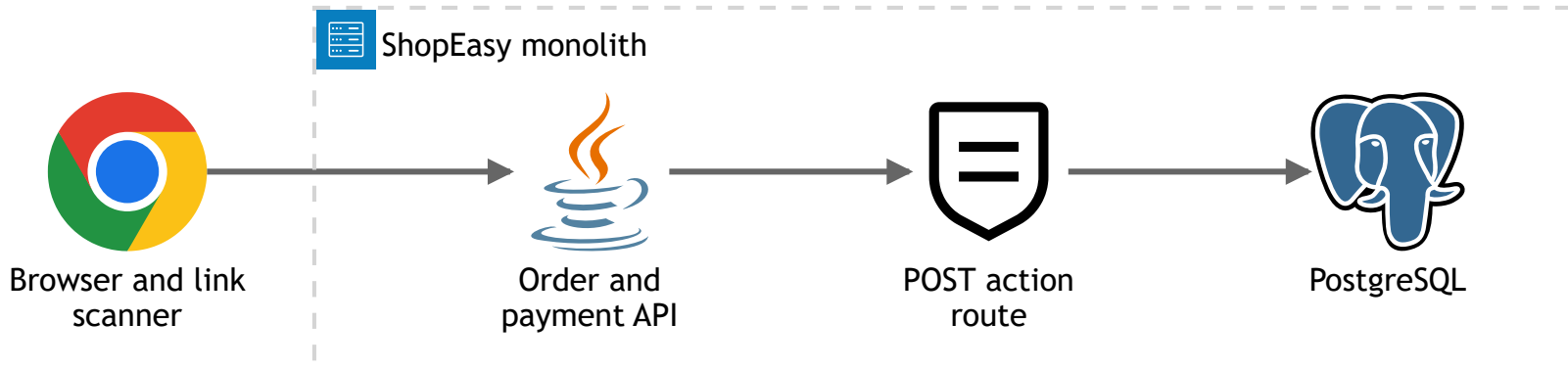
Verification: replayed GET traffic changes nothing

Check	Pass condition
Access log replayed as GET only	0 orders cancelled and 0 payments captured
Crawler follows every link on order pages and e-mails	0 rows changed in orders and payments
Owner sends POST cancel once	Order cancelled once; a second POST changes nothing more
Another customer sends POST cancel	403 and no change
Old GET path requested	200 with the order state and no side effect
Fitness function	A CI test crawls every GET route in the OpenAPI document with a read-only database role plus a row-version snapshot, and fails when any GET changes a row

Rollback: Routing rollback: the flag brings back the old GET behavior at once, and the exposure to scanners returns. State rollback: orders cancelled or payments captured by GET stay changed; list them from the audit log by request method GET, reinstate the orders, void or refund the captures and tell the customers.

Stop: When replayed GET traffic changes no state, no signed-link scheme, confirmation workflow or gateway rule is added

Solution architecture



Same client, API and database; cancel and capture now run only in a POST action route, so a scanner’s GET can read an order but never change it.

Safe and idempotent methods are promises that tools rely on

A link in an e-mail is opened with GET, so the method's promise of no state change is all a scanner has to go on.

Method	Safe	Idempotent	What tools may do
GET	yes	yes	Prefetch, crawl, cache and retry it without asking
HEAD	yes	yes	Probe a link like GET, without a body
PUT	no	yes	Repeat it automatically after a failure
DELETE	no	yes	Repeat it automatically after a failure
POST	no	no	Nothing assumed; each repeat may add an effect
PATCH	no	no	Nothing assumed; the patch format decides

The scanner’s GET reads, the customer’s POST acts

```
GET /orders/O123/cancel HTTP/1.1

HTTP/1.1 200 OK
{ "state": "PLACED" }

POST /orders/O123/cancel HTTP/1.1

HTTP/1.1 200 OK
{ "state": "CANCELLED" }
```

- **GET:** a scanner, prefetch or proxy reads the order state and changes nothing.
- **POST:** only the customer’s confirmed click, owner-checked, cancels the order once.
- **Another customer’s POST:** gets 403 and no change.

Glossary

Used in this step

Term	Definition
Safe method	HTTP method defined to request no state change, such as GET and HEAD
Idempotent method	Method whose repeated request has the same effect as one request
Link scanner	Mail or security tool that opens the links in a message automatically
Prefetch	Browser or proxy loading a linked page before the user asks for it
POST action endpoint	Route that performs one business action when it receives a POST
Signed one-time GET link	URL with a signed, expiring token that triggers an action on GET
Confirmation page + POST	Read-only page whose confirm button sends the state-changing POST
Access log replay	Sending recorded requests to a test system to compare behavior
Owner check	Server rule that only the resource's owner may request the action
Routing rollback	Switching routes or flags back to the previous behavior
State rollback	Restoring data to its earlier content, which switching routes cannot do

Related concepts

Term	Definition
REST	Resource-oriented interface style using uniform HTTP semantics
Uniform interface	Standard method meanings that generic clients and intermediaries understand
HTTP caching	Responses define cacheability, freshness and validators
OpenAPI document	Machine-readable description of an HTTP API's routes and messages
CSRF	Attack that makes a signed-in browser send an unwanted request

API02 · Resources and HTTP semantics

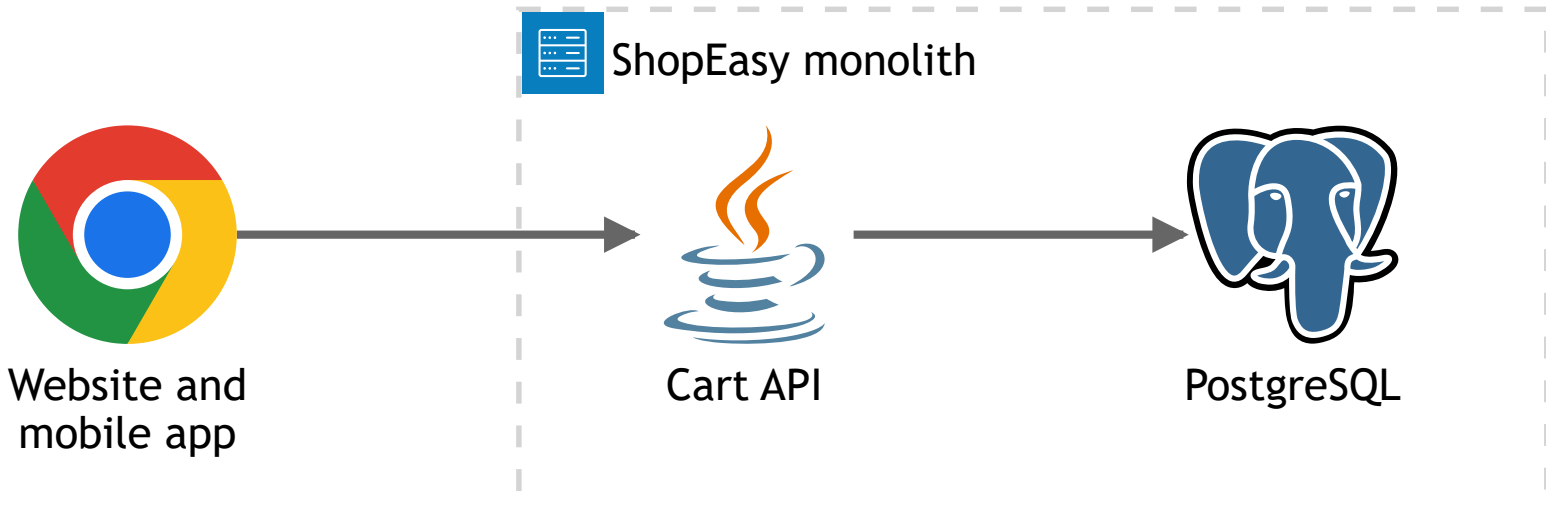
Prevent lost updates

Two clients edit the same cart and the last write wins, so 1.8% of cart saves overwrite a change the client never saw, against a target of 0 silent overwrites.

A cart write is a PUT and the last request wins

Component	What it is	What it does
Website and mobile app	The two clients that edit a shared cart	Send PUT /customers/C7/carts/C9/items/P4 with the new quantity
Cart API	The cart routes in the Java modular monolith	Applies each PUT as received, so the last request wins
PostgreSQL	Database that holds carts and cart items	Stores the quantity of whichever write arrives last
Shared cart	One cart that several people edit from different clients	Changes twice within seconds, and one change disappears

Current architecture



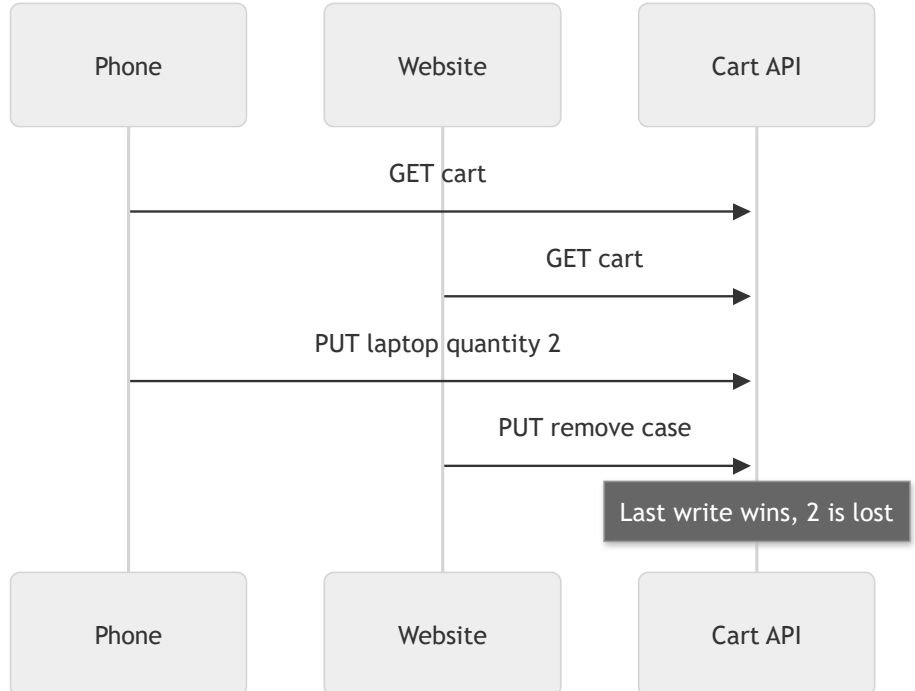
The second save drops the first change

A household shares one cart. The phone sets the laptop quantity to 2 while the website removes the case; the second save drops the first change.

1.8%
of cart saves overwrite a change the client never saw

2 clients
write one cart within seconds

0
silent overwrites in a concurrent-edit test



Ways to refuse a write built from an old cart

Solution	What it is	What it does
ETag + If-Match	The server returns a version tag with each cart and the client sends it back as a precondition on writes	A write on a stale tag is refused with 412, and the client re-reads and reapplies
Version number in the body	The cart JSON carries a version field that the client sends back	The handler compares versions and refuses a stale write
Pessimistic lock	A client takes a lock on the cart before editing and releases it afterwards	Only one client at a time can edit the cart

Detecting a stale write costs a tag, a field or a lock

Solution	Gains	Costs
ETag + If-Match	Standard HTTP headers Generic tools understand it No lock is held	Clients must store and resend the tag A conflict needs a user decision Missing header must be rejected
Version number in the body	Easy to read and log No header handling Works behind simple proxies	Private convention no tool knows Every endpoint repeats the check Easy to forget on a new route
Pessimistic lock	No conflicting writes at all Simple to reason about	Abandoned locks block the cart Needs expiry, renewal and release calls Stateful flow over stateless HTTP

Deciding signal: How often two writers collide and what a collision should cost the user: measure concurrent writes per cart and the share of conflicts that merge cleanly. Optimistic checking wins while collisions are rare; above a few percent of saves (for example 5 percent) merge rules or a different model are needed

Smallest safe move

ETag on cart reads, If-Match on cart writes

- **Scope:** Contract, behavior, clients: cart item writes only
- **Change:** The cart GET returns an ETag; PUT and PATCH on cart items need If-Match: 412 for a stale tag, 428 when missing; warn mode first, then enforce
- **Safety net:** The tag is a version, not a timestamp; check and write share one transaction; no pessimistic lock, abandoned locks block carts
- **Escape route:** A flag returns to warn mode: missing If-Match accepted and logged; ETag stays
- **Deferred:** Version number in the body only for a client without headers; pessimistic lock not planned; merge rules when conflicts exceed the threshold

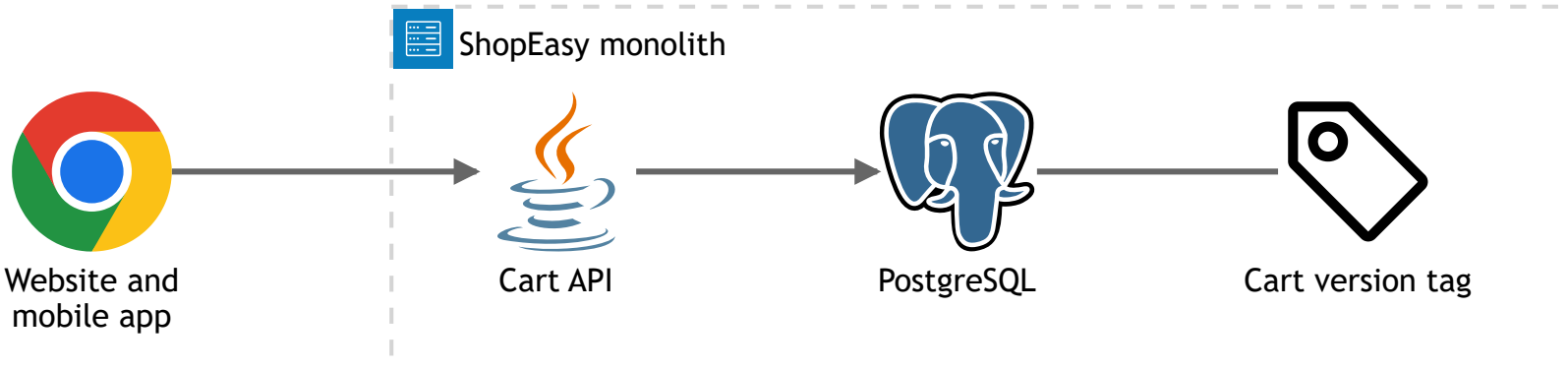
Verification: two writers, one cart, no silent overwrite

Check	Pass condition
Two writers use the same tag	First gets 200; second gets 412 and the cart keeps the first change
Second writer re-reads and retries with the new tag	200 and both changes are present
Write without If-Match after the warn period	428 and no change
100 concurrent edits against one cart	No accepted update is lost; the final cart equals the accepted edits
Fitness function	A concurrency test in CI runs parallel writers against one cart and fails on any lost update; a metric counts 412 and 428 by client version

Rollback: Routing rollback: the flag returns to warn mode, where a missing If-Match is accepted, and lost updates become possible again. State rollback: overwrites that already happened stay lost; recover from the cart change history if one exists, otherwise ask affected customers to re-check their carts.

Stop: When the concurrent-writer test shows no lost update and the conflict rate stays low, no lock, merge engine or event-sourced cart is added

Solution architecture



Same clients, API and database; each cart now carries a version that the API returns as an ETag and checks against If-Match, so a write built from an old cart is refused with 412.

A stale tag is refused and a missing tag is asked for

```
GET /customers/C7/carts/C9 HTTP/1.1
HTTP/1.1 200 OK
ETag: "c9-v3"

PUT /customers/C7/carts/C9/items/P4 HTTP/1.1
If-Match: "c9-v3"
{ "quantity": 2 }
```

Both clients read the cart at the tag `c9-v3`; the second writer still sends that tag after the cart has moved on.

The write carries	The server answers	What the client does
The current tag	`200` and the new ETag	Keeps the new tag for the next write
A stale tag	`412 Precondition Failed`	Re-reads the cart and reapplies its change
No If-Match header	`428 Precondition Required`	Adds the tag and sends again

Conditional headers and statuses answer different questions

Header or status	What it says	What the client does
`ETag`	Opaque version tag of a representation	Stores it with the representation
`If-Match`	Makes a write conditional on the current tag	Sends the stored tag on PUT and PATCH
`If-None-Match`	Asks for an answer only if the tag differs	Sends the stored tag on a repeated GET
`304 Not Modified`	The tag still matches, so no body is sent	Keeps using its stored copy
`412 Precondition Failed`	The condition in the request did not hold	Re-reads, reapplies, tries again
`428 Precondition Required`	The server asks for a conditional request	Adds If-Match and sends again

Glossary

Used in this step

Term	Definition
ETag + If-Match	Version tag in responses, sent back as a precondition on writes
Version number in the body	Version field in the JSON that the client sends back with a write
Pessimistic lock	Exclusive claim on a resource while one client edits it
ETag	Opaque version tag of a representation, returned in a response header
If-Match	Request header that makes a write conditional on the current tag
412 Precondition Failed	Status code that says the condition in the request did not hold
428 Precondition Required	Status code that asks the client to send a conditional request
Lost update	A write silently overwritten by a writer that did not see it
Optimistic concurrency	Detect conflicts at write time instead of locking in advance
Conditional request	Request the server performs only if a stated condition on the resource holds

Related concepts

Term	Definition
Last write wins	Conflict policy where the latest request replaces earlier changes
Idempotent method	Method whose repeated request has the same effect as one request
HTTP caching	Responses define cacheability, freshness and validators
Row version	Counter column that changes on every update and can back the tag

API03 · Resources and HTTP semantics

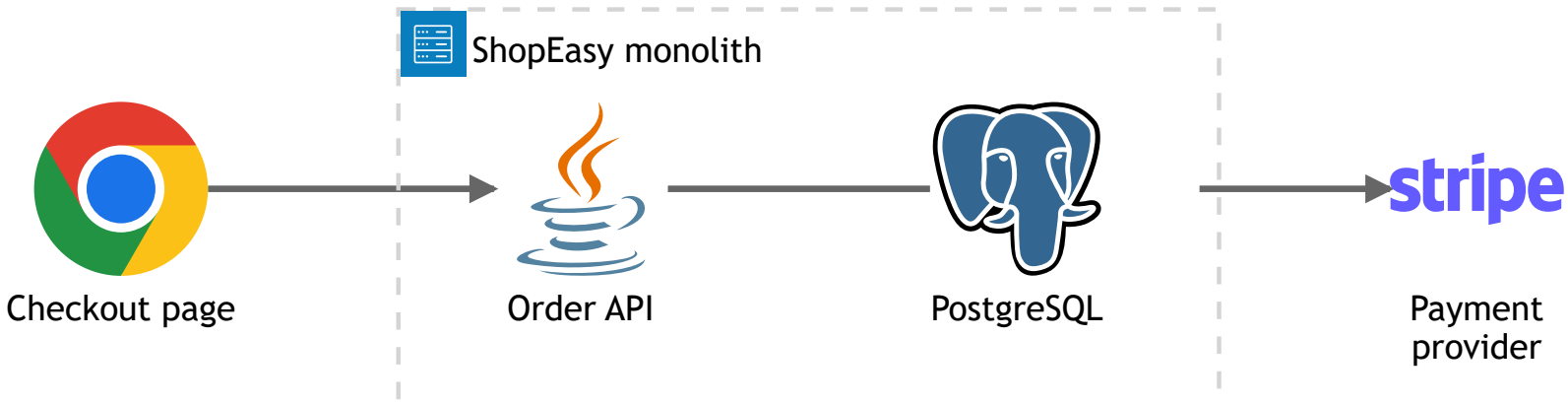
Take the order total from the server

Promotions, tax recalculation and price changes make the client-supplied total untrustworthy: 2.3% of orders store a total that differs from the recomputed one, against a target of 0 orders charged another amount.

The order total is whatever the client sends

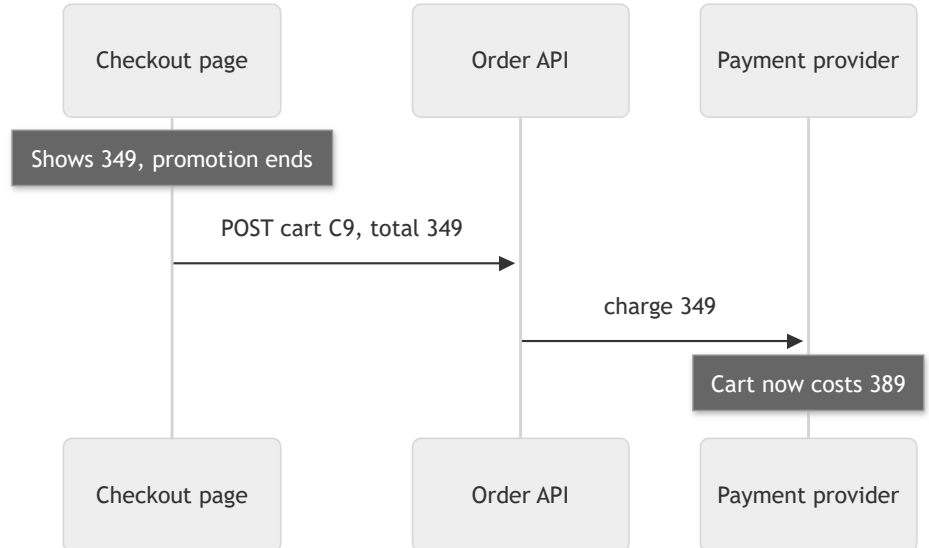
Component	What it is	What it does
Checkout page	The ShopEasy order form shown in the customer’s browser	Sends POST /customers/C7/orders with cartId C9 and the displayed total 349
Order API	The order route in the Java modular monolith	Stores the total sent by the client and asks the payment provider to charge it
PostgreSQL	Database that holds carts, prices and orders	Stores the order with the client-supplied total
Payment provider	External service that charges the customer’s payment	Charges the amount the API sends, whatever the current price

Current architecture



A stale total of 349 is charged

A promotion ends at midnight while a customer is on the checkout page. The page still sends 349 for a cart that now costs 389, and ShopEasy accepts it.



2.3%
of orders differ from the recomputed total

3 inputs
price, promotion and tax change under open carts

0
orders charged other than the server total

Ways to stop trusting a total the client sends

Solution	What it is	What it does
Server total, client value ignored	The server computes the total and discards the total sent by the client	Prevents a forged total and always charges the server price
Server total + expected total check	The client sends the total it displayed as expectedTotal and the server computes its own and compares	A mismatch is refused with 409 and the new total, so the customer confirms the price
Price quote resource	A quote resource holds the server-computed total with an expiry and the order references it	The price is guaranteed until the quote expires

Computing the total is cheap, consent and a held price cost more

Solution	Gains	Costs
Server total, client value ignored	Client cannot set the price Smallest change One source of truth	Customer may pay more than shown Price change is silent, no consent Disputes after a price move
Server total + expected total check	Customer pays only a confirmed price Forged totals are rejected Change limited to one route	Price moves cause refused orders Client must show the new total Comparison must handle rounding
Price quote resource	Price held for a stated time Quote can be audited Fits holds and promotions	New resource and lifecycle Expiry and cleanup to build Extra round trip at checkout

Deciding signal: How often the displayed total and the server total differ and what a difference should do: measure the share of orders whose totals differ and whether the business promises a price for a period. A price promise (a held price for a stated time) picks the quote resource; without it, comparing the expected total is enough

Smallest safe move

Server computes the total, client confirms it

- **Scope:** Contract, behavior, clients: POST orders only
- **Change:** Server recomputes prices, promotions and tax, trusting no client total; a mismatch with expectedTotal gets 409 and the new total in problem details; an old total counts as expectedTotal
- **Safety net:** The silent variant is not used, it can charge more than shown; the provider gets the stored server total; money is a decimal string with a currency
- **Escape route:** A flag accepts the client total for one release, logging mismatches
- **Deferred:** Price quote resource when a price is promised for a period; pricing service when promotions need their own release cycle

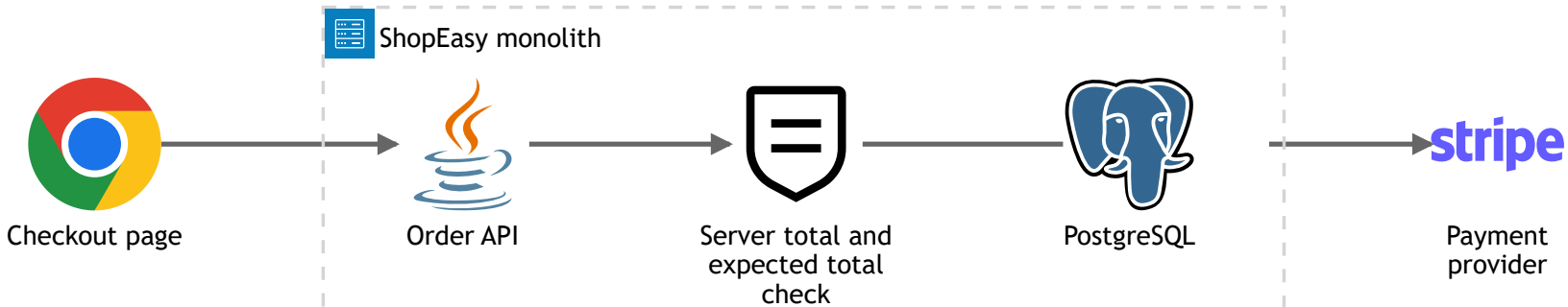
Verification: changed prices, old totals and a forged total

Check	Pass condition
Total unchanged since display	Order created; stored total equals the displayed total
Promotion ended before submit	409 with the new total; no order and no charge
Forged total of 1 sent	409; the payment provider receives no charge
Old client that sends total	Handled as expectedTotal with the same results
1,000 generated carts across tax rules	Recomputed total equals the pricing engine total to the cent in every cart
Fitness function	A contract test replays generated carts with changed prices and asserts that the charged amount equals the server total and that a mismatch never creates an order

Rollback: Routing rollback: the flag accepts the client total again, which restores the exposure. State rollback: orders already created with a client total stay as stored; recompute them from the audit data, charge or refund the difference and tell the customers.

Stop: When mismatches are refused, nothing is charged other than the server total and refusals stay rare, no quote resource or pricing service is added

Solution architecture



Same page, API, database and provider; the API now computes the total from current prices and compares it with the expected total before charging.

A refused total comes back with the new total

```
POST /customers/C7/orders HTTP/1.1
{ "cartId": "C9", "expectedTotal": { "value": "349.00", "currency": "PLN" } }

HTTP/1.1 409 Conflict
Content-Type: application/problem+json
{ "type": "/problems/total-changed", "status": 409,
  "title": "The total changed since it was displayed",
  "currentTotal": { "value": "389.00", "currency": "PLN" } }
```

The body is problem details; its extra member `currentTotal` lets the page show the new price for the customer to confirm.

Expected total sent	Answer	Effect
Equals the server total	Order created	Charged the server total
Differs after a promotion ended	`409` with the new total	No order and no charge; the customer confirms the new price
Forged total of 1	`409`	No order; the provider receives no charge

A price quote is warranted only when a price is promised

If the signal is	Choose	Why
No price promise, totals sometimes differ	Server total + expected total check	A refusal with the new total gives the customer the choice
A price is promised for a stated time	Price quote resource	The price is held until the quote expires and can be audited
The client total is only ignored	Not enough alone	Forgery is closed, but the customer may pay more than shown
Promotions need their own release cycle	Pricing service, later	Pricing changes then ship apart from checkout
Mismatches are refused and refusals stay rare	Nothing more	No quote resource or pricing service is added

Glossary

Used in this step

Term	Definition
Server total, client value ignored	The server computes the total and discards the client's value
Server total + expected total check	The server computes the total and refuses the order if it differs from the client's expected total
Price quote resource	A resource that holds a server-computed price with an expiry
Client-supplied total	A total taken from the request body instead of computed by the server
Expected total	The total the client displayed and asks the server to confirm
409 Conflict	Status code for a request that conflicts with the current state of the resource
Problem details	Standard machine-readable error body, application/problem+json
Money value	Decimal amount with a currency, never a floating-point number
Price promise	Commitment to charge a shown price until a stated time
Authoritative value	Value computed or owned by the system responsible for it

Related concepts

Term	Definition
Domain invariant	Business rule that must hold after every change
Consumer-driven contract test	Consumer expectations verified against a provider
Fitness function	Measurable or executable architectural quality rule
Idempotency Key	Stable identifier recognizing multiple attempts at the same logical command

API04 · Resources and HTTP semantics

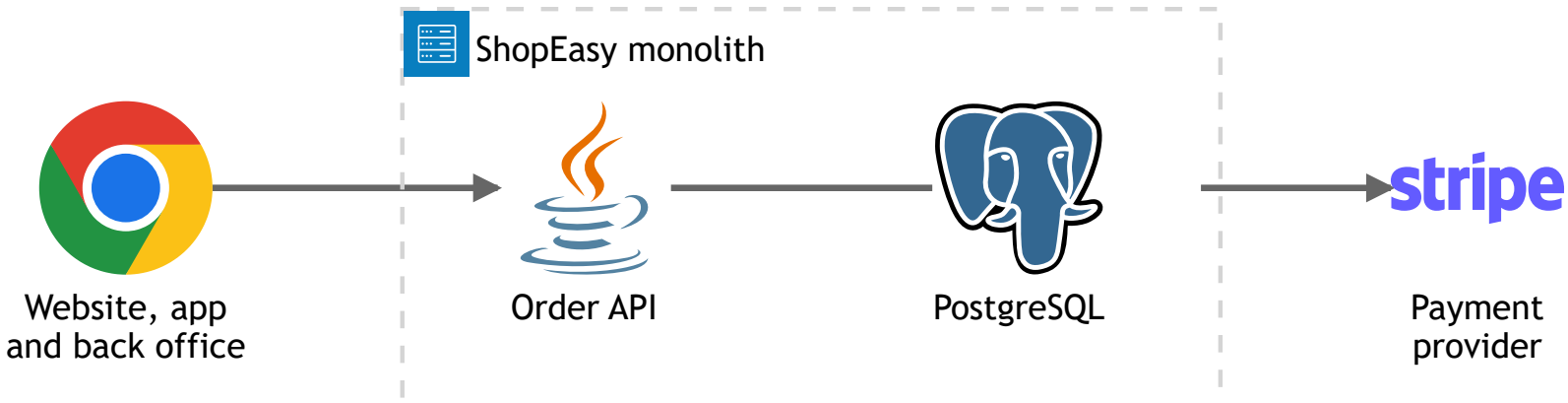
Replace status edits with server-owned transitions

Fraud review, partial capture and cancellation while pending make a client-set status wrong: 14 orders shipped as PAID without a captured payment in one week, against a target of 0 status changes without provider evidence.

Order status is a field that any client can patch

Component	What it is	What it does
Website, app and back office	The clients that mark an order paid	Send PATCH /orders/O8 with status PAID, also after their own payment call timed out
Order API	The PATCH, payment and stock reservation routes in the Java modular monolith	Accepts any status value a client sends, including PAID
PostgreSQL	Database that holds orders and payments	Stores the status as sent, whatever the provider confirmed
Payment provider	External service that captures payments and reports the outcome	Knows whether money moved; fraud review can reject after a capture request

Current architecture



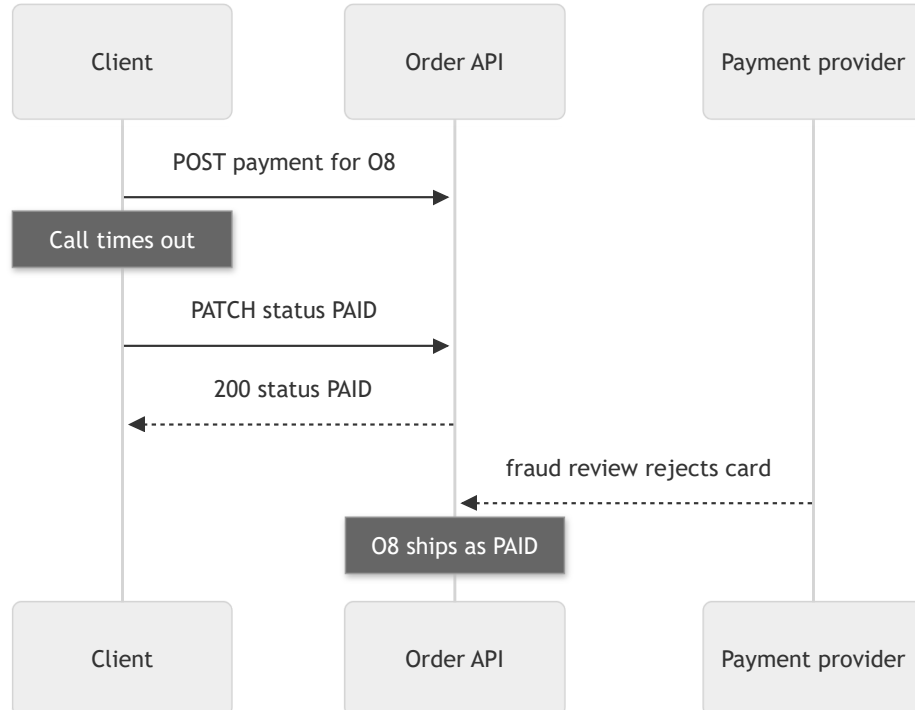
Status says what a client believed

A client marks an order PAID after its payment call timed out; fraud review later rejects the card. Another order, captured for 200 of 349, also shows PAID.

14 orders
shipped as PAID without captured payment in a week

4 writers
website, app, back office and provider patch status

0
status changes accepted without provider evidence



Ways to stop clients from asserting payment facts

Solution	What it is	What it does
Transition rules behind PATCH	A state machine in the PATCH handler allows only listed status changes	Rejects impossible transitions but still lets a client request PAID
Payment attempt resources	Each payment try is a resource created with POST; the provider result sets its status and the order status follows	Status is server-owned and set only from payment evidence
Command endpoints	One POST route per command, such as POST /orders/O8/pay and POST /orders/O8/cancel	The client asks, and the server decides and moves the status

Each option moves the decision about PAID a different distance

Solution	Gains	Costs
Transition rules behind PATCH	No new routes Invalid jumps are refused Quick to add	Client can still claim PAID Partial capture has no field Rules hide behind one verb
Payment attempt resources	Status comes from evidence Partial and repeated payments fit Each attempt is auditable	New resource and table Provider callbacks to handle Clients read status instead of setting it
Command endpoints	Clear intent per call Simple to document Fast to build	Routes multiply with the workflow Command result is not a resource Retry safety per route

Deciding signal: Who may assert a fact and what evidence backs it: count the status values that only the provider or fraud review can know, and the share of orders with several payments or partial captures. If a status needs evidence from another party the client must not set it; several attempts per order pick payment attempt resources

Smallest safe move

Payment attempts decide status; clients only read

- **Scope:** Contract, behavior, clients, data: read-only status, one new attempt route
- **Change:** Client PATCH of status gets 422; POST /orders/O8/payment-attempts creates an attempt; the provider callback sets its result; PAID needs captured equal to total; back office keeps PATCH one release
- **Safety net:** Transition rules stay a second check, not the design; each attempt carries an idempotency key so a retry cannot charge twice
- **Escape route:** A flag restores PATCH of status for the back office; attempts keep recording
- **Deferred:** Command endpoints for evidence-free actions such as cancellation until the outcome must be tracked; refund resource when partial refunds arrive

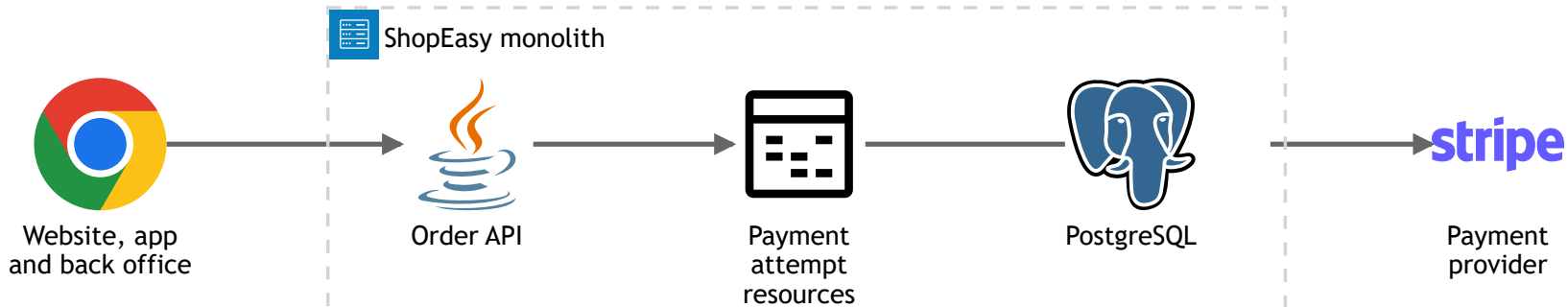
Verification: every PAID order has captured payments equal to its total

Check	Pass condition
Client sends PATCH status PAID	422 and no change
Provider confirms a full capture	Attempt captured; order PAID; amounts equal
Partial capture of 200 of 349	Order not PAID; the remaining amount is visible
Fraud review rejects after the capture request	Attempt failed; the order stays unpaid and never ships
Nightly query for PAID orders without captured payment	0 rows
Fitness function	A nightly query and a CI contract test assert that PAID orders have captured payments equal to the total; the build fails if any route lets a client write a provider-owned status

Rollback: Routing rollback: the flag restores client PATCH of status, which reopens the hole. State rollback: orders marked PAID without evidence stay in that state; find them with the nightly query, hold shipment and reconcile each with the provider before release.

Stop: When no order is PAID without a captured payment and clients only read status, no workflow engine or event-sourced order is added

Solution architecture



Same clients, API, database and provider; payment attempts are created through the API, the provider result is written from the callback, and clients only read the order status.

A payment attempt is requested and the status is derived

```
POST /orders/O8/payment-attempts HTTP/1.1
{ "methodId": "PM2", "amount": { "value": "349.00", "currency": "PLN" } }

HTTP/1.1 201 Created
Location: /payment-attempts/PA1

GET /orders/O8 HTTP/1.1
HTTP/1.1 200 OK
{ "status": "UNPAID", "total": "349.00", "captured": "200.00" }
```

The client never sends a status; it asks to pay and later reads what the provider evidence made of the order.

Step	Who writes	Result
`POST` payment attempt	Client	An attempt exists; the order status is unchanged
Provider callback	Provider evidence	The attempt result is set, here 200 of 349 captured
`GET` order	Nobody, read only	Status derived: PAID only when captured equals the total

Clients state intent, evidence and derived state stay with the server

Field or fact	Who may set it	Why
Items and the chosen payment method	Client	The client knows what the customer wants
A request to pay	Client, by POST on payment attempts	The client asks and the server decides
Attempt result: captured, declined, failed	Provider callback only	Only the provider knows whether money moved
Fraud review outcome	Provider or fraud review only	It can reject after payment was requested
Order status PAID	Nobody directly: derived from attempts	PAID only when the captured amount equals the total
Customer cancellation while pending	Client, as a command	It carries no evidence; tracked as a resource only when needed

Glossary

Used in this step

Term	Definition
Transition rules behind PATCH	A state machine that limits which status values a PATCH may set
Payment attempt resources	Each payment try is its own resource whose status comes from the provider
Command endpoints	One route per command; the client asks and the server decides the new state
Server-owned state	State only the server may change, from its own rules or evidence
State machine	Set of states and the transitions allowed between them
Payment attempt	One try to charge a payment method for an order
Partial capture	Charging less than the authorised amount
Provider callback	Request from the payment provider that reports an outcome
Derived status	Status computed from other records instead of set by hand
Fraud review	Check that can hold or reject an order after payment was requested
422 Unprocessable Content	Status code for a well-formed request the server will not accept

Related concepts

Term	Definition
REST is not CRUD	A resource can model a process or evidence, not only a stored row
Idempotency Key	Stable identifier recognizing multiple attempts at the same logical command
Reservation Pattern	Temporarily claim business resource pending workflow completion
Domain command	A named request to change the domain, such as pay or cancel
Webhook	HTTP callback that one system sends to another when an event happens

API05 · Resources and HTTP semantics

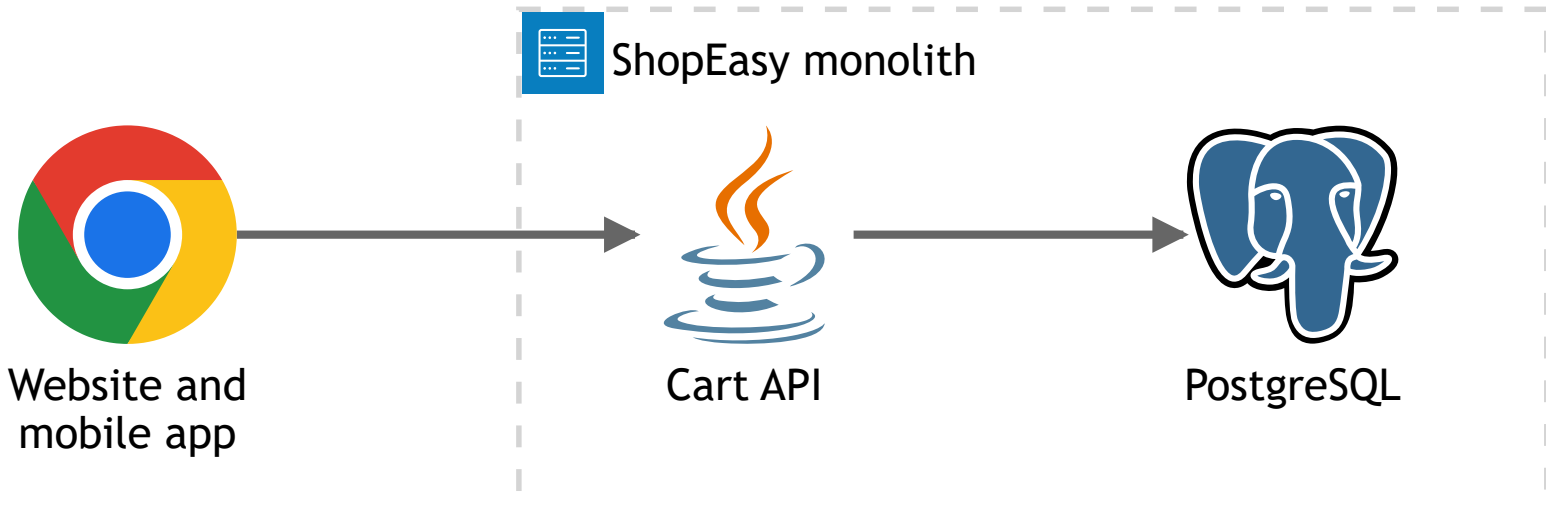
Give resources identity independent of their parent

Anonymous, shared and reassigned carts break URLs that name one customer, so 9% of cart requests fail with 404 after sign-in or sharing, against a target of 0 URL changes.

Nested URLs put one parent in every address

Component	What it is	What it does
Website and mobile app	The clients that build, share and store cart links	Request /customers/C7/carts/C9 and keep the link in e-mails and bookmarks
Cart API	The cart and order routes in the Java modular monolith	Finds a cart through the customer id in the path
PostgreSQL	Database that holds carts, orders, payments and refunds	Stores each cart with one customer id
Nested URLs	Paths such as /customers/C7/orders/O8 and /orders/O8/payments/P5/refunds/R2	Encode one parent in the address of every child

Current architecture



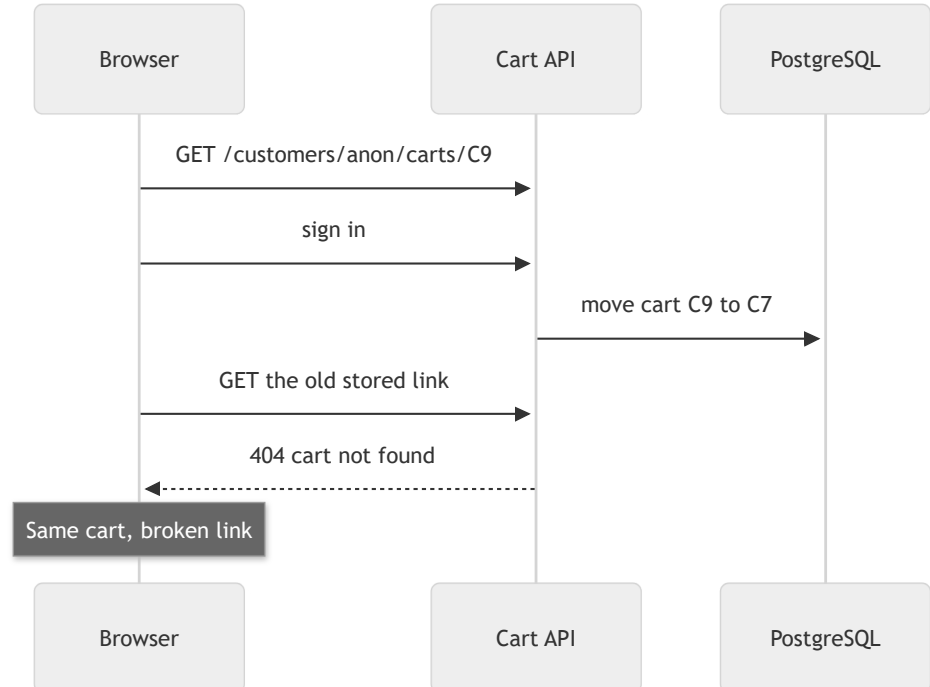
A cart changes owner and its URL breaks

A guest cart moves to customer C7 at sign-in, then a household member joins it. The URL holds one customer id, so stored links go stale.

9%
of cart requests fail with 404 after sign-in or sharing

3 changes
owner changes per cart, from guest to shared

0
URL changes on ownership change; old URLs resolve



Ways to address a resource whose owner changes

Solution	What it is	What it does
Canonical URL + contextual collection	Each resource has one top-level URL such as /carts/C9, and /customers/C7/carts lists the carts that customer can use	Identity stays when the owner changes; the nested view is only a filtered list
Placeholder owner in the nested path	Keep the nested path and put a placeholder owner such as anonymous or the session id in it	Anonymous carts get a path of the same shape
Nested path, parent ignored	Keep /customers/C7/carts/C9 but look the cart up by C9 alone	Any owner id in the path resolves to the same cart

A stable address costs a new route and a different access check

Solution	Gains	Costs
Canonical URL + contextual collection	Identity survives ownership change Links and caches stay valid Many owners fit	New top-level route Authorization on the resource, not the path Old nested URLs need redirects
Placeholder owner in the nested path	No new route Old clients unchanged Quick fix	URL changes at sign-in Shared carts have no single owner Caches and links break on reassignment
Nested path, parent ignored	No client change Nothing to migrate Works today	The path lies about ownership Authorization easily uses the wrong id Several URLs name one cart

Deciding signal: Whether the child can exist or move without its parent: count resources with more than one possible parent or a changing parent (anonymous, shared and reassigned carts, partial refunds) and the share of requests that use a stale ancestor path. A resource whose parent can change needs its own top-level identity; one whose identity depends on the parent, such as a line inside a statement, may stay nested

Smallest safe move

Canonical cart URL, nested path redirects

- **Scope:** Contract and clients: carts first, orders, payments and refunds later
- **Change:** Add /carts/C9 as the canonical resource; /customers/C7/carts lists the carts the customer may use; the old nested path answers 308 Permanent Redirect to the canonical URL
- **Safety net:** Authorization checks cart membership, so a redirect never grants access; links use the canonical URL; parent ignored is not used, the path would lie about ownership
- **Escape route:** Remove the redirect rule and serve both paths; the canonical route stays
- **Deferred:** Top-level /refunds/R2 when partial refunds or chargebacks need their own address; a statement line stays nested

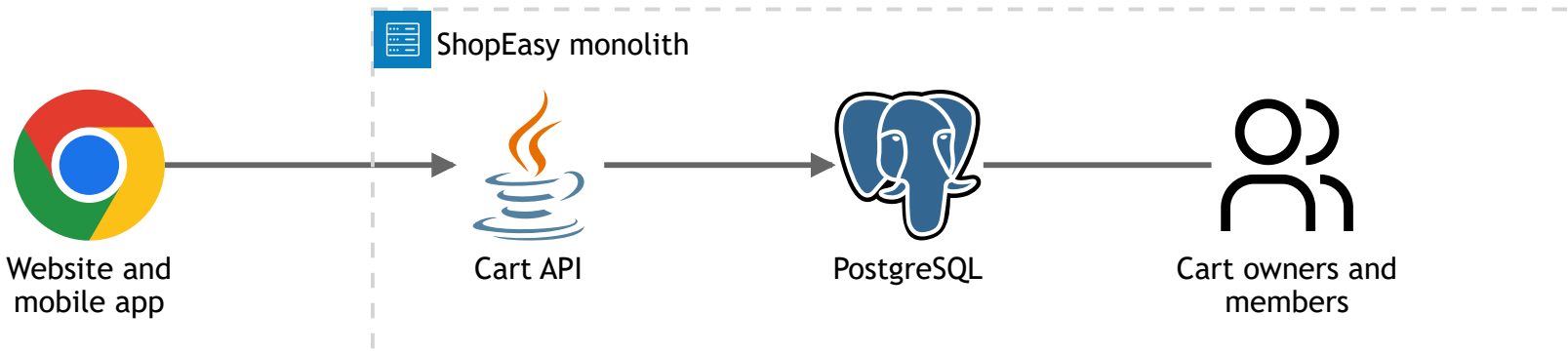
Verification: owner changes, every stored link still works

Check	Pass condition
Stored nested link used after an owner change	308 to /carts/C9, then 200 for an authorized user
Unauthorized user opens the canonical URL	403 or 404, as before the change
Anonymous cart signs in	Same URL before and after
Contextual collection requested by a member	Lists the shared cart
Every link in representations	All hrefs are canonical
Fitness function	A contract test asserts that every link in a representation is canonical and that an ownership change keeps the URL; an access-log query counts 308 hits to decide when the old paths can go

Rollback: Routing rollback: removing the redirect makes the nested paths serve directly again. State rollback: none needed for URLs; a cart reassigned during the change keeps its new owner and members.

Stop: When ownership changes leave the URL and every stored link intact, no URL-rewriting gateway or opaque-identifier scheme is added

Solution architecture



Same clients, API and database; the cart has one canonical URL, access is checked against its owners and members, and the customer path is only a list that links to it.

A stored nested link redirects to the canonical cart

```
GET /customers/C7/carts/C9 HTTP/1.1

HTTP/1.1 308 Permanent Redirect
Location: /carts/C9

GET /carts/C9 HTTP/1.1

HTTP/1.1 200 OK
```

- **308:** the old nested path moves for good and the method is kept.
- **Canonical URL:** `/carts/C9` stays the same whoever owns the cart; the final `200` goes to an authorized member.
- **Contextual collection:** `GET /customers/C7/carts` lists the carts the customer may use, each linking to its canonical URL.

A deep path is a smell only when ancestry is mistaken for ownership

Test: can the child exist or move without its parent? A deep path is not automatically un-RESTful; it becomes troublesome when its ancestry is confused with domain ownership, aggregate boundaries or physical storage.

Example URL	Verdict	Why
`/accounts/A1/statements/2026-10/lines/142`	Potentially good	Line 142 is identified within a particular statement
`/transfers/T1/ledger-postings`	Good	Exposes a meaningful relationship collection
`/customers/C7/accounts/A1/transfers/T1`	Suspicious	Transfer identity does not necessarily depend on one customer’s ownership
`/customers/C7/orders/O8/shipments/S3/tracking-events/E1`	Suspicious	Several ownership assumptions are encoded into one long path
`/orders/O8/payments/P5/refunds/R2`	Context-dependent	Valid as a contextual link, misleading as the refund’s canonical identity
`/products/P4/availability/warehouses/W2`	Context-dependent	Reasonable view if warehouse availability is independently addressable

Glossary

Used in this step

Term	Definition
Canonical URL + contextual collection	One stable address per resource, plus lists that show related resources
Placeholder owner in the nested path	A stand-in owner id kept in the path for resources without one
Nested path, parent ignored	A parent in the path that the server does not use to find the resource
Canonical URL	The one stable address of a resource
Contextual collection	List resource that shows related resources without owning them
Resource identity	What stays the same about a resource while its state and relations change
308 Permanent Redirect	Status code that moves a URL for good and keeps the method
Containment	A URL shape where the child's path sits under its parent's path
Authorization on the resource	Permission check against the resource's members, not against its path
Partial refund	One of several refunds against one payment

Related concepts

Term	Definition
Aggregate	Cluster of records changed together under one consistency rule
Bounded Context	Boundary inside which a domain model and its language stay consistent
Link relation	Name that says why one resource links to another
REST	Resource-oriented interface style using uniform HTTP semantics

API06 · Resources and HTTP semantics

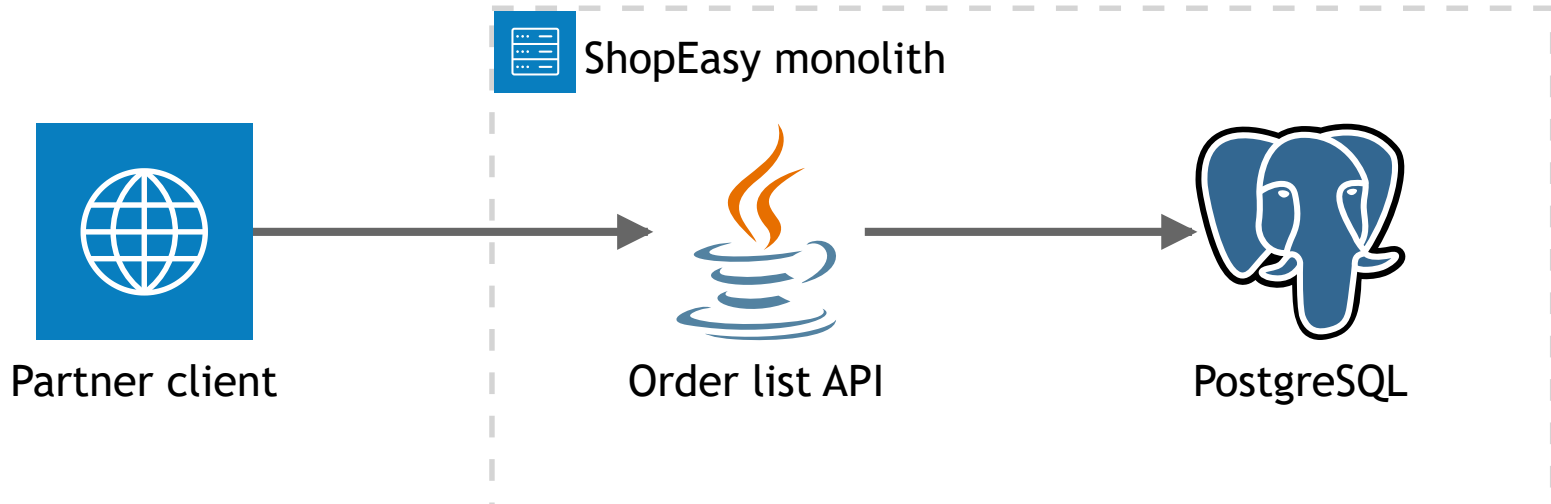
Page large collections without skipping or repeating rows

Deep offset pages get slower and rows are skipped or repeated while orders keep arriving: p95 of 1.2 s on a deep page against a target of ≤ 100 ms at any depth.

A partner system walks the order list page by page

Component	What it is	What it does
Partner client	A partner system that syncs ShopEasy orders	Syncs orders by walking page 1, 2, 3 and so on
Order list API	GET /orders and /customers/C7/orders in the Java monolith	Returns a slice by OFFSET from page and perPage, sorted by createdAt
PostgreSQL	Database that holds orders	Counts and discards every skipped row before it returns a page
Incoming orders	Checkout traffic that creates orders all day	Inserts new rows at the top of the list while a walk is running

Current architecture



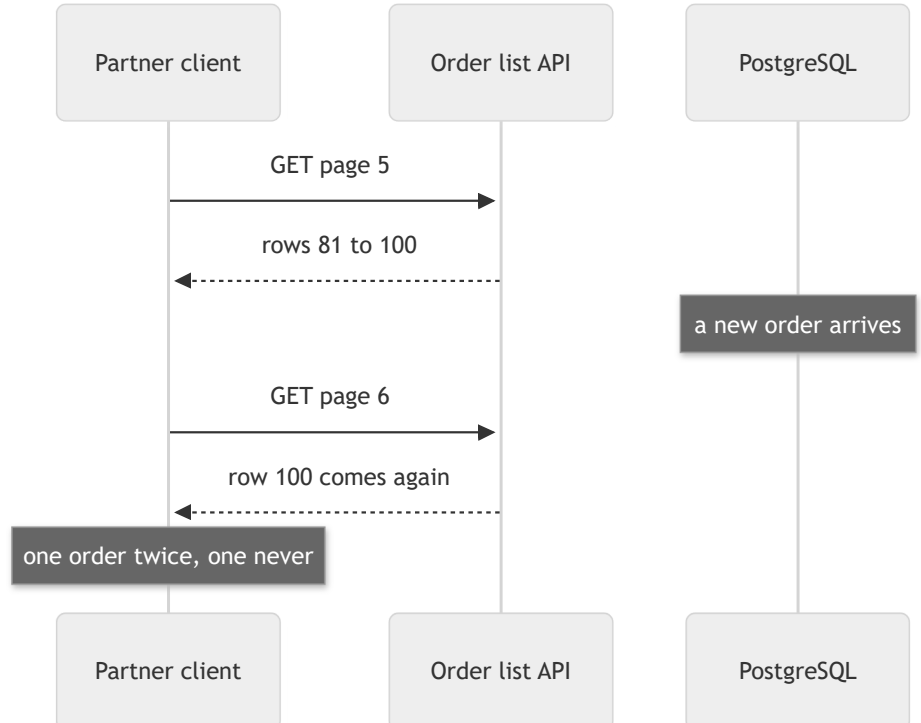
New orders shift pages under a client

A partner syncs orders by walking pages. New orders shift every row, so some appear twice and one never appears, and deep pages take over a second.

1.2 s
p95 of a deep offset page

20 per second
new orders arrive while a client walks

≤ 100 ms
p95 at any depth, no skipped or repeated rows



Ways to continue a list from where the client stopped

Solution	What it is	What it does
Page + perPage	A page number and a size select a slice by offset	Lets a user jump to any page
Cursor + limit	An opaque cursor marks the last row seen and the next call continues after it	Reads the next rows with an indexed range scan on a stable key
Sort + from/to	Clients ask for a time or id range of rows	Returns every row in the range, in order

Each paging style suits one kind of client and fails another

Solution	Gains	Costs
Page + perPage	Friendly page numbers Jump to any page Trivial SQL OFFSET	Deep pages grow slower Inserts shift rows: gaps and repeats Page 5 names no fixed set
Cursor + limit	Cost flat at any depth No skipped or repeated rows Page size bounded	No jump to page 500 Cursor must hold the whole sort key Reverse paging and expiry need design
Sort + from/to	Natural for logs and time series Easy to resume a sync Ranges are cacheable	Large ranges still need batching Inclusive bounds must be defined Ties need a unique tie-breaker

Deciding signal: What the client needs: browse recent rows, jump to a numbered page, or walk everything exactly once. Measure p95 by depth and the share of walks with gaps or repeats while writes continue; if users never go past page 20, offset paging with a depth cap is enough, and a sync or export client forces a cursor or a range

Smallest safe move

Continue after the last key seen

- **Scope:** Contract, clients, data: the order list and one composite index
- **Change:** Add cursor and limit (default 50, cap 100) ordered by (createdAt, id) with a next link; page and perPage serve the first 20 pages, then answer 400
- **Safety net:** The cursor holds the whole ordering tuple, never an offset; limit is capped; filters run before the cut; no total count
- **Escape route:** A flag hides the cursor route; page and perPage stay
- **Deferred:** Sort + from/to when sync clients need a time window; a snapshot when one walk needs one consistent view

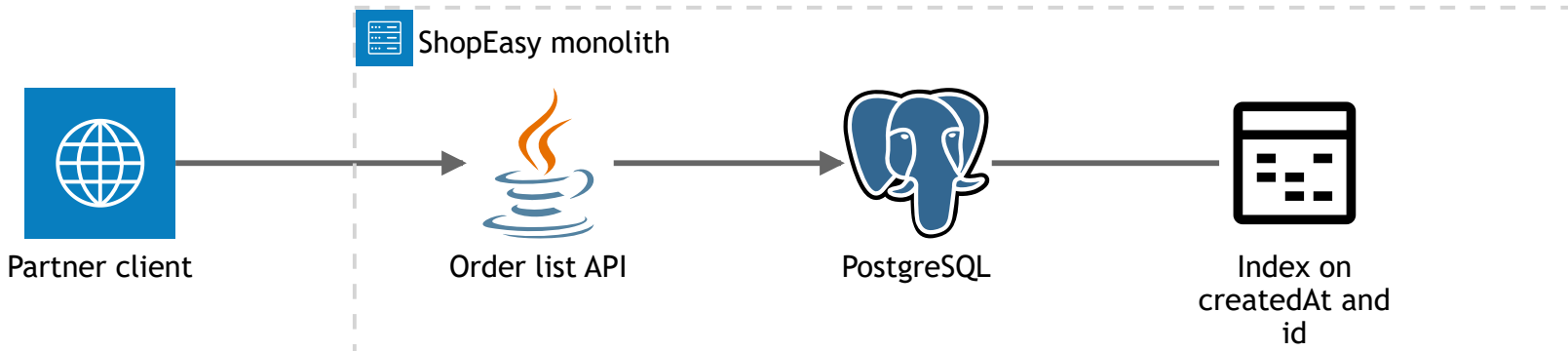
Verification: walk the whole list while orders keep arriving

Check	Pass condition
Walk of 2,000,000 orders with the cursor while 20 inserts per second continue	Every order present at the start is seen exactly once; none repeated
p95 at depth 1, 100,000 and 2,000,000	≤ 100 ms at every depth
Request with limit=1000000	Capped to 100 or rejected
Rows with an equal createdAt at a page boundary	Order is stable by id; none skipped
Old client calls page 5 with perPage 20	Same result as before the change
Fitness function	A nightly walk test against a copy with concurrent writes asserts exactly-once and latency by depth; a dashboard shows p95 by cursor depth

Rollback: Routing rollback: a flag hides the cursor route and clients use page and perPage again. State rollback: none; the change only reads, and the composite index can be dropped.

Stop: When walks see every order once at the target latency, no search engine, snapshot context or separate read store is added

Solution architecture



Same client, API and database; the API now continues after the last key seen, so every page is an index range scan and a walk is not shifted by new orders.

The client decides which question the list must answer

Request	Question it answers	Watch for
`?page=5&perPage=20`	Browse recent rows, or jump to a numbered page	Page 5 names no fixed set; deep pages cost more
`?limit=20&offset=80`	The same question written as a raw offset	Skipped rows are still processed; inserts cause gaps and repeats
`?cursor=abc&limit=20`	Walk everything exactly once	No jump to page 500; the cursor keeps the whole sort key
`?sort=createdAt&from=…&to=…`	Which rows fall in this time window	Large ranges still need batching; equal values need a tie-breaker

Pagination is a contract about ordering, consistency, navigation and resource cost, not only slicing a collection.

A cursor request and a response with a next link

```
GET /orders?cursor=MjAyNi0xMS0wMlQxMDowMDowMFp8TzQy&limit=50 HTTP/1.1

HTTP/1.1 200 OK

{ "items": [ { "id": "O43" }, " ... ", { "id": "O92" } ],
  "links": { "next": "/orders?cursor=MjAyNi0xMS0wMlQxMDowNDoxMFp8Tzky&limit=50" } }
```

Part	What it does
`cursor`	Opaque to the client; encodes the last row seen as `(createdAt, id)`, never an offset
`limit`	Page size, default 50 and capped at 100
`next`	The client follows it until it is absent; no total count is computed

Glossary

Used in this step

Term	Definition
Page + perPage	Page number and size selecting a slice of a collection by offset
Cursor + limit	Opaque position token plus a page size, continuing after the last row seen
Sort + from/to	A sorted range of rows selected by lower and upper bound
Offset pagination	Skips a number of rows, then returns the next ones
Keyset pagination	Continues after the last key seen using an indexed range scan
Cursor	Opaque token that marks where the next page starts
Tie-breaker	Unique field added to a sort key so that equal values keep a fixed order
Composite index	Index on several columns in the same order as the sort
Stable ordering	Sort order that does not change between two requests of the same data
Snapshot	Consistent view of the data as of one moment

Related concepts

Term	Definition
Total count trap	Counting a filtered set can cost more than fetching the page
Pagination versus streaming	Browsing, batch export and event consumption need different continuation rules
Rate Limiter	Controls the number of requests allowed within a time window.
HTTP caching	Responses define cacheability, freshness and validators

Phase 2

Workflow resources and contracts

Steps in this phase

Step	Forcing constraint
API07 Model approval as a resource with evidence	Payments above PLN 10,000 need an additional approval, business accounts may need two authorized representatives, and each approval is verified with the customer’s security method and kept with outcome, person and time
API08 Advertise available actions	Mobile and partner clients release separately, but hard-coded URLs and status rules force a coordinated release for every workflow change
API09 Return errors clients can act on	Failures come back as HTTP 200 with success false, so clients and intermediaries cannot tell a retryable failure from a permanent one
API10 Let caches serve statements safely	The same closed statements are fetched again and again while personal data must never reach a shared cache
API11 Evolve a representation without breaking old clients	Joint accounts change holder to holders while old app versions stay in use

API07 · Workflow resources and contracts

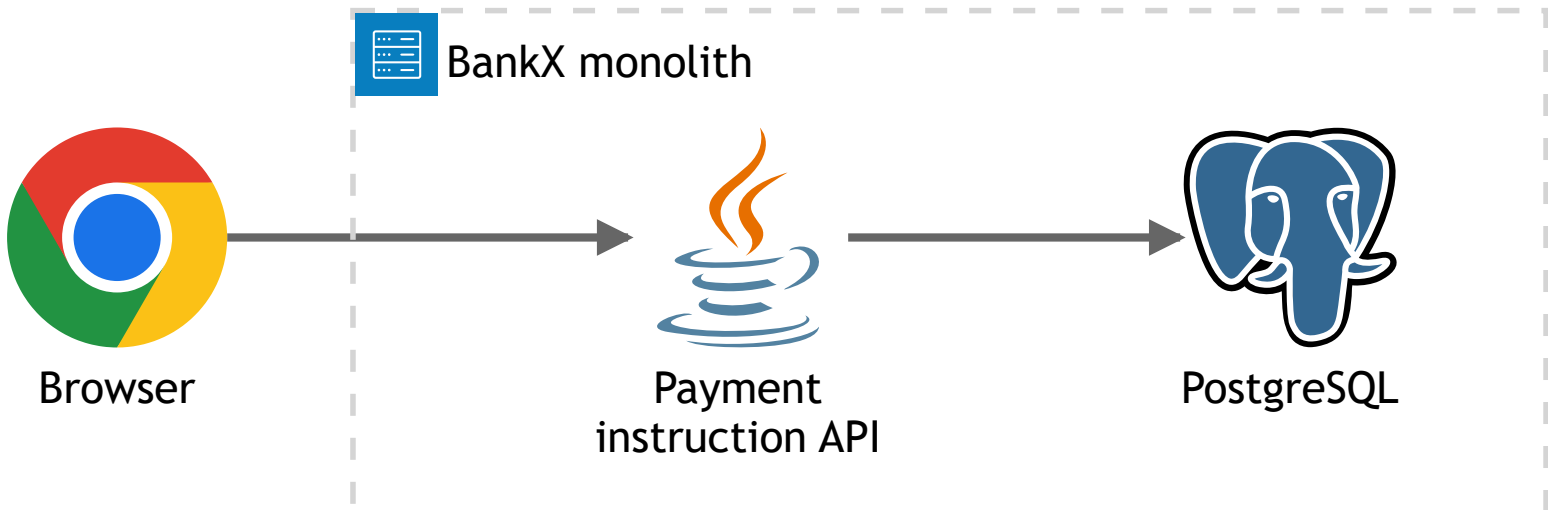
Model approval as a resource with evidence

A status field cannot say who approved a payment, under which challenge or whether a second approval is missing: 0 of 6 audit questions answered against a target of 100% for payments above PLN 10,000.

An instruction is approved by setting its status field

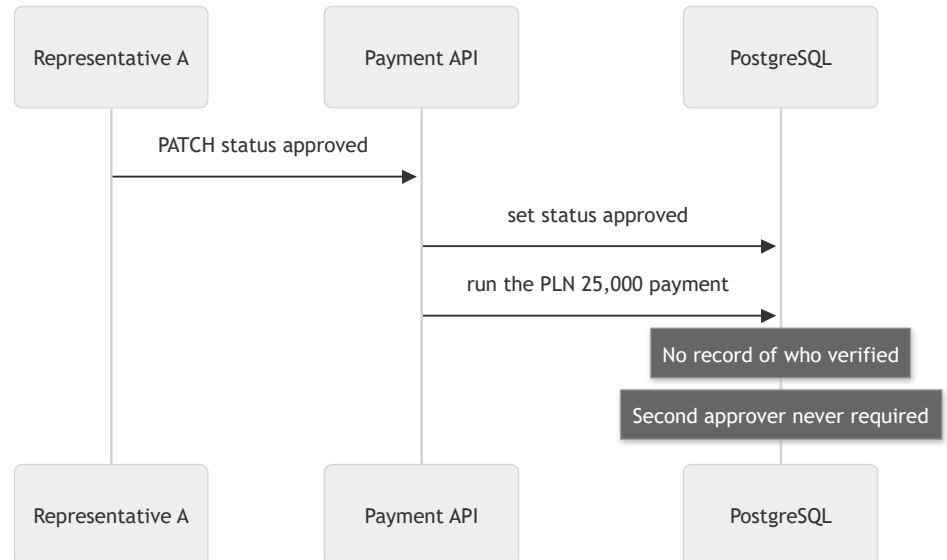
Component	What it is	What it does
Browser	BankX web app of an authorized representative	Sends PATCH /payment-instructions/PI1 with status approved
Payment instruction API	The instruction routes in the Java monolith	Accepts the PATCH and stores the new status without asking who approved
PostgreSQL	Database that holds accounts, beneficiaries and instructions	Keeps one status column per instruction
Payment instruction	A customer’s request to pay a beneficiary from an account	Waits in awaiting-approval until a client sets approved

Current architecture



A status field cannot say who approved

A company payment of PLN 25,000 needs two representatives. A status field cannot say who approved, under which challenge, which version they saw, or whether the second approval is missing.



0 of 6
audit questions a status field can answer

PLN 10,000
above this, an additional approval is required

100%
of payments above it execute only with verified approvals

Ways to record an approval

Solution	What it is	What it does
PATCH status approved	The client sets the instruction's status to approved	Moves the instruction forward with one field change
Approvals embedded in the instruction	The instruction carries an approvals list that a PATCH appends to	Records who approved inside the instruction document
Approval sub-resource	Evidence is POSTed to /payment-instructions/PI1/approvals and creates an approval with its own URL and status	Gives each approval identity, evidence and lifecycle, and the instruction derives its state from them

The more evidence an approval keeps, the more structure it needs

Solution	Gains	Costs
PATCH status approved	One route, no new resource Familiar CRUD shape Quick to build	No person, challenge or version Cannot express two approvers Client asserts a verified fact
Approvals embedded in the instruction	No new URL Approvals travel with the instruction One conditional update	No identity or lifecycle per approval Hard to expire, reject or audit one Two approvers contend for one record
Approval sub-resource	Person, evidence and time kept Fits two approvers and expiry Outcome is retrievable and auditable	New resource and table Verification flow to build Clients follow Location and read status

Deciding signal: Whether an approval needs its own identity, evidence and lifecycle: count approvals per instruction (one or two), the states an approval can reach (verified, rejected, pending-verification, expired) and the audit questions the record must answer. One approver and no evidence needs only a field; two approvers or per-approval expiry need a resource

Smallest safe move

Each approval becomes a resource with evidence

- **Scope:** Contract, behavior, data: one route and one table for approvals
- **Change:** POST /payment-instructions/PI1/approvals (challenge id, proof) returns 201 Created, Location /approvals/AP1, status verified, rejected or pending-verification; the instruction advances on enough verified approvals; PATCH status answers 422
- **Safety net:** The proof comes from a bank-defined challenge, never client text; the server verifies it, binds it to the instruction version, checks permission; verified is not executed
- **Escape route:** A flag re-enables PATCH approval for single-approver accounts below the threshold
- **Deferred:** Approvals embedded in the instruction only for a single-approver product without evidence; revocation and expiry when a policy requires them

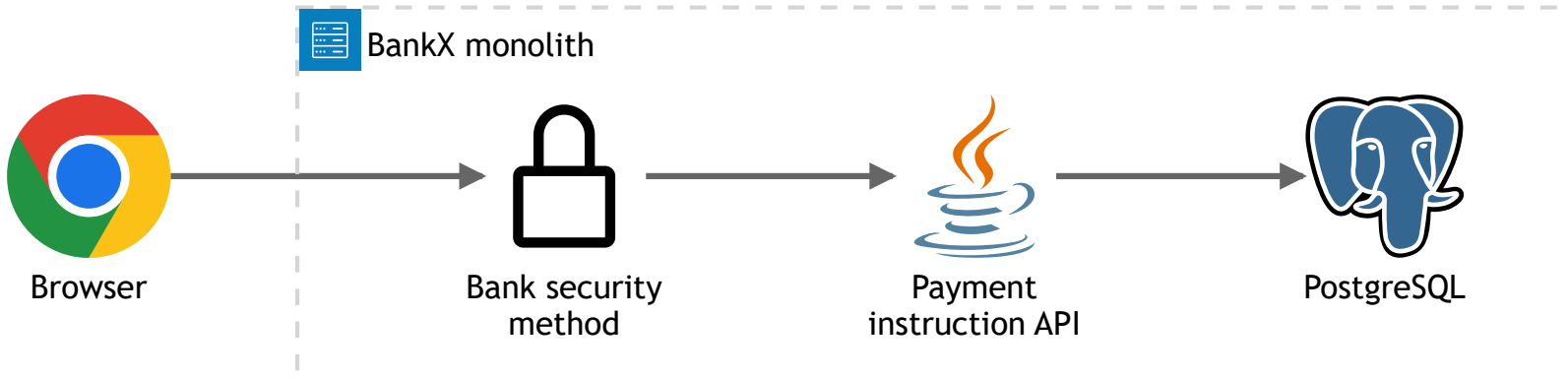
Verification: two representatives, a wrong proof and an expired challenge

Check	Pass condition
First of two representatives approves with a valid proof	201; approval verified; the instruction still awaits the second approval
Second representative approves	201; the instruction leaves awaiting-approval
Wrong or expired proof	Approval rejected and the instruction unchanged
Same representative approves twice	Second approval refused; only one counts
Audit query on executed payments above PLN 10,000	Each has the required verified approvals with person, time and outcome
Client sends PATCH status approved	422 and no change
Fitness function	A nightly audit query and a CI test assert that no instruction above the threshold reaches execution without the required verified approvals

Rollback: Routing rollback: the flag restores the PATCH shortcut for single-approver accounts and the approvals route stays off for them. State rollback: approvals already recorded are evidence and are never deleted; an approval made by mistake is revoked by a new record, not edited.

Stop: When every executed payment above the threshold traces to verified approvals with person, time and outcome, no approval workflow engine is added

Solution architecture



Same client, API and database; the bank’s security method now issues the challenge and verifies the proof, and each approval is stored as its own record with person, time and outcome.

The client sends evidence and receives the address of an approval

```
POST /payment-instructions/PI1/approvals HTTP/1.1
Content-Type: application/json

{ "challengeId": "CH1", "proof": "<bank-issued-authorization-proof>" }

HTTP/1.1 201 Created
Location: /approvals/AP1

{ "id": "AP1", "status": "verified", "instructionId": "PI1" }
```

Part	What it means
`challengeId` and `proof`	Evidence from a bank-defined challenge, never text typed by the user
`201 Created` and `Location`	The approval has its own URL; `GET /approvals/AP1` returns it later
`status`	`verified`, `rejected` or `pending-verification`; verified does not mean the transfer is executed

An approval record answers the six audit questions that a status field cannot

Audit question	Kept on the approval record
Who provided authorization?	The authorized representative and the time
Under which authentication challenge?	The challenge id and the verification outcome
Which version of the instruction did they authorize?	The instruction version the approval is bound to
Was the authorization accepted?	The status: verified or rejected
Can it expire or be revoked?	An expired state; revocation is a new record
Is an additional approver required?	Verified approvals counted against the required number

An approval ends in one of four states: verified, rejected, pending-verification or expired. Only a verified approval counts toward the required number.

Glossary

Used in this step

Term	Definition
PATCH status approved	A client request that sets the instruction's status field to approved
Approvals embedded in the instruction	Approval entries kept as a list inside the instruction document
Approval sub-resource	Approval as its own resource, created under the instruction it authorizes
Payment instruction	A customer's request to pay a beneficiary from an account
Approval	Verified authorization of one instruction by one authorized person
Authorized representative	Person allowed to act for a business account under its agreement
Challenge and proof	Bank-issued test and the customer's answer to it
Evidence	Recorded facts that show how and by whom something was authorized
Audit history	Reviewable record of instructions, approvals and decisions
201 Created with Location	Reply that names the URL of the resource just created

Related concepts

Term	Definition
REST is not CRUD	A resource can model a process or evidence, not only a stored row
Hypermedia	Responses that advertise the links and actions available next
Four-eyes principle	Rule that two people must agree before a critical action
Domain command	A named request to change the domain, such as approve or cancel
Bounded Context	Boundary inside which a domain model and its language stay consistent

API08 · Workflow resources and contracts

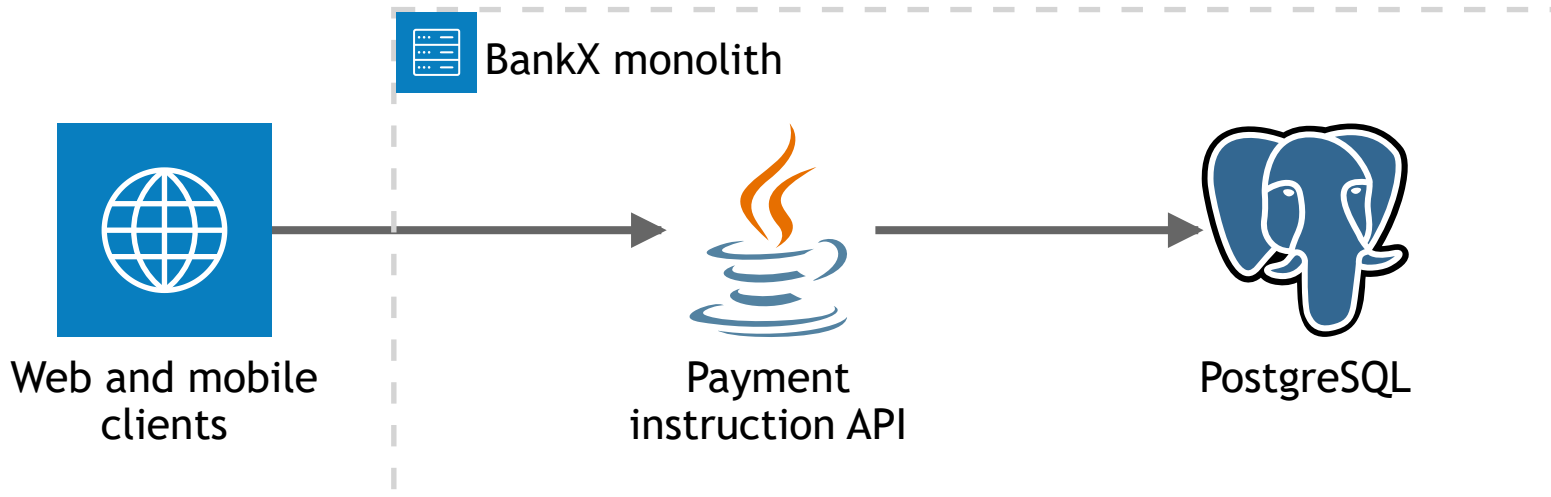
Advertise available actions

Hard-coded URLs and status rules in separately released clients force a coordinated release for every workflow change: 6 weeks of lead time against a target of 0 coordinated client releases.

Clients read the status, choose a button and build the URL

Component	What it is	What it does
Web and mobile clients	BankX apps released on their own schedules	Read an instruction, pick buttons from its status and build the approvals URL
Payment instruction API	The instruction and approvals routes in the Java monolith	Returns the instruction as plain JSON with a status and no actions
PostgreSQL	Database that holds instructions and approvals	Stores the status that every client interprets
Status table in client code	Rules that map each status to an allowed action and a URL	Must change in every client when the workflow changes

Current architecture



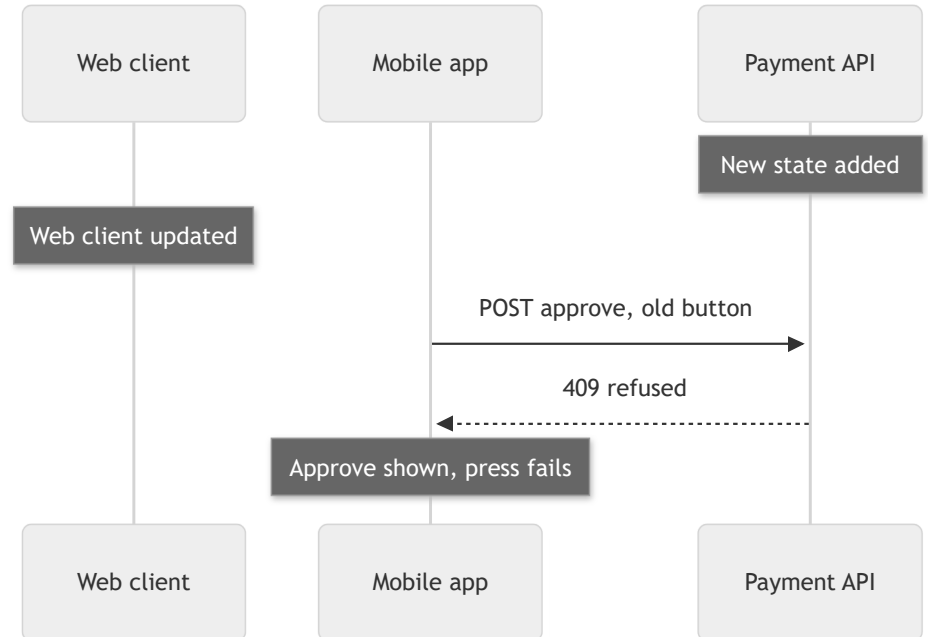
Each workflow change waits for the slowest client

A new state, awaiting-second-approval, is added. The web client is updated, but the mobile app still shows Approve for approved instructions, and every press fails.

6 weeks
set by the slowest client release

3 states
workflow states that clients hard-code

0
coordinated releases for an added or removed action



Ways to tell a client what it may do next

Solution	What it is	What it does
Documented status table in clients	Clients hard-code which status allows which action and build each URL	Works while every client ships the same table
Allowed-actions list	The representation lists the names of actions currently allowed, such as approve	Clients stop guessing from status but still build the URL
Hypermedia actions	The representation advertises each available action with its name, method, URL and input fields, Siren-style	Clients follow what the response offers and render forms from it

The more the response says, the less the client has to know

Solution	Gains	Costs
Documented status table in clients	No server change Simple clients Works today	Workflow change needs client releases Every client repeats the rules Old apps show stale buttons
Allowed-actions list	Small additive change No new media type Hides stale buttons	Client still knows the URLs Method and fields are not described Private vocabulary
Hypermedia actions	Clients need no paths or transitions Workflow evolves on the server Self-descriptive messages	Larger representations Clients need a generic action handler Action names still need documented meaning

Deciding signal: How many independently released clients depend on the workflow and how often it changes: count client releases forced per workflow change and the time to ship the slowest. One client released with the server can keep a documented status table; several independent clients push toward advertised actions, and when actions gain input fields the full hypermedia format pays off

Smallest safe move

The response lists the actions it allows

- **Scope:** Contract, clients: the instruction representation only, via content negotiation
- **Change:** Serve application/vnd.siren+json on request with class, properties, links and actions (approve: POST, href, fields challengeId and proof), shown only while the state allows it; plain JSON stays default
- **Safety net:** The server still validates permission, state, challenge and concurrency on every POST, because hiding an action is not authorization; actions come from the same rules
- **Escape route:** A flag stops serving Siren; clients use plain JSON again
- **Deferred:** Allowed-actions list when a client cannot parse Siren; other resources as their workflows change; a shared library when several teams build clients

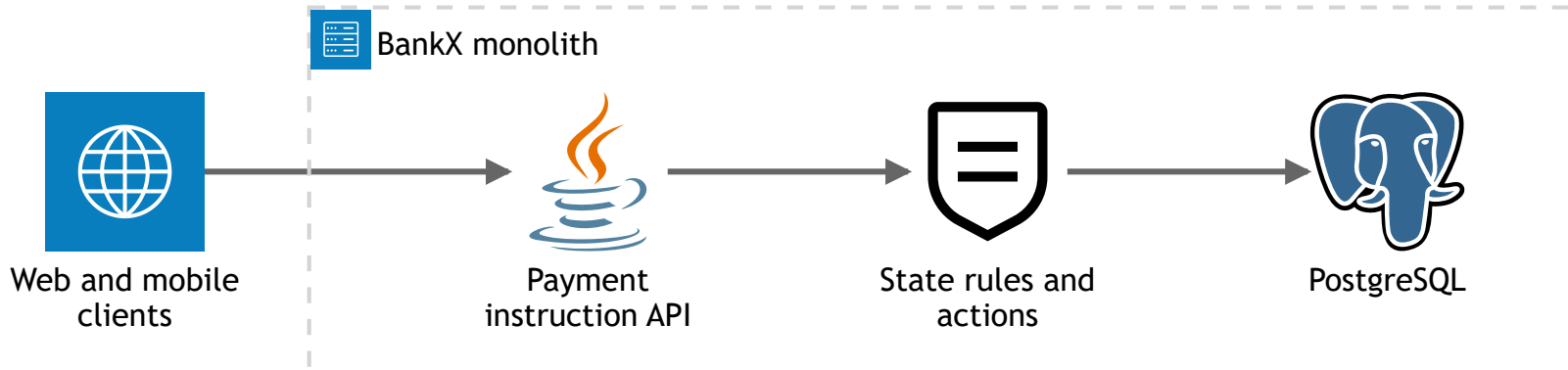
Verification: a state added on the server reaches an unreleased client

Check	Pass condition
Instruction awaiting approval is requested	approve action present with method, href and two fields
Instruction already approved or settled	No approve action; links to the related resources are present
New state added on the server only	Unreleased client shows the right buttons without a release
POST to approve when it is not advertised	409 or 403 from the server, by the same rules as when advertised
Client that asks for application/json	Same response as before the change
Fitness function	A contract test compares the advertised actions with the server’s state rules for every state, and a test client that only follows actions runs the whole approval flow

Rollback: Routing rollback: the flag stops serving the Siren media type and clients use the plain JSON they used before. State rollback: none; actions are computed on each read and no data changed.

Stop: When a workflow change reaches unchanged clients through advertised actions, no generic hypermedia framework or SDK generator is added

Solution architecture



Same clients, API and database; the API now computes the allowed actions from the same state rules that guard each POST and lists them in the response, so clients follow them instead of hard-coding status rules.

The response lists the approve action with method, URL and fields

```
{
  "class": ["payment-instruction"],
  "properties": { "id": "PI1", "status": "awaiting-approval" },
  "links": [ { "rel": ["self"], "href": "/payment-instructions/PI1" } ],
  "actions": [ {
    "name": "approve", "method": "POST",
    "href": "/payment-instructions/PI1/approvals",
    "fields": [ { "name": "challengeId" }, { "name": "proof" } ]
  } ]
}
```

Part	What the client does with it
`actions`	Shows Approve only while it is listed; after approval the list is empty
`method` and `href`	Sends the form there instead of building the URL
`fields`	Renders the inputs; the meaning of `approve`, `challengeId` and `proof` is still documented

Hiding an action is not authorization: four checks run on every POST

Check	What the server asks	Why the advertised action cannot replace it
Permission	May this representative approve for this account?	The button may reach the wrong person, or the call may be made directly
State	Is the instruction still in a state that can be approved?	The state may change after the representation was read
Challenge	Is the proof valid for a bank-defined challenge?	The client can send any text
Concurrency	Is this still the version the client saw?	Another approver may have acted in between

The advertised action is computed from the same rules that guard the POST, and a stale or hand-made request meets the same checks.

Glossary

Used in this step

Term	Definition
Documented status table in clients	Client code that maps each status to the allowed actions and URLs
Allowed-actions list	Names of the actions currently allowed, without how to call them
Hypermedia actions	Actions advertised with name, method, URL and fields in the response
Hypermedia	Responses that advertise the links and actions available next
HATEOAS	Hypermedia as the engine of application state: the response communicates available transitions
Siren	Hypermedia media type with properties, links and actions
Action	A named operation a client may request next, with method, URL and input fields
Link relation	Name that says why one resource links to another
Content negotiation	Client and server agree on a representation through Accept and Content-Type
Independent evolvability	Clients and server change on separate release schedules

Related concepts

Term	Definition
Uniform interface	Standard method meanings that generic clients and intermediaries understand
Richardson Maturity Model	Levels of HTTP API design; not a formal REST compliance test
Consumer-driven contract test	Consumer expectations verified against a provider
Backend for Frontend (BFF)	API layer tailored to a specific frontend or client.

API09 · Workflow resources and contracts

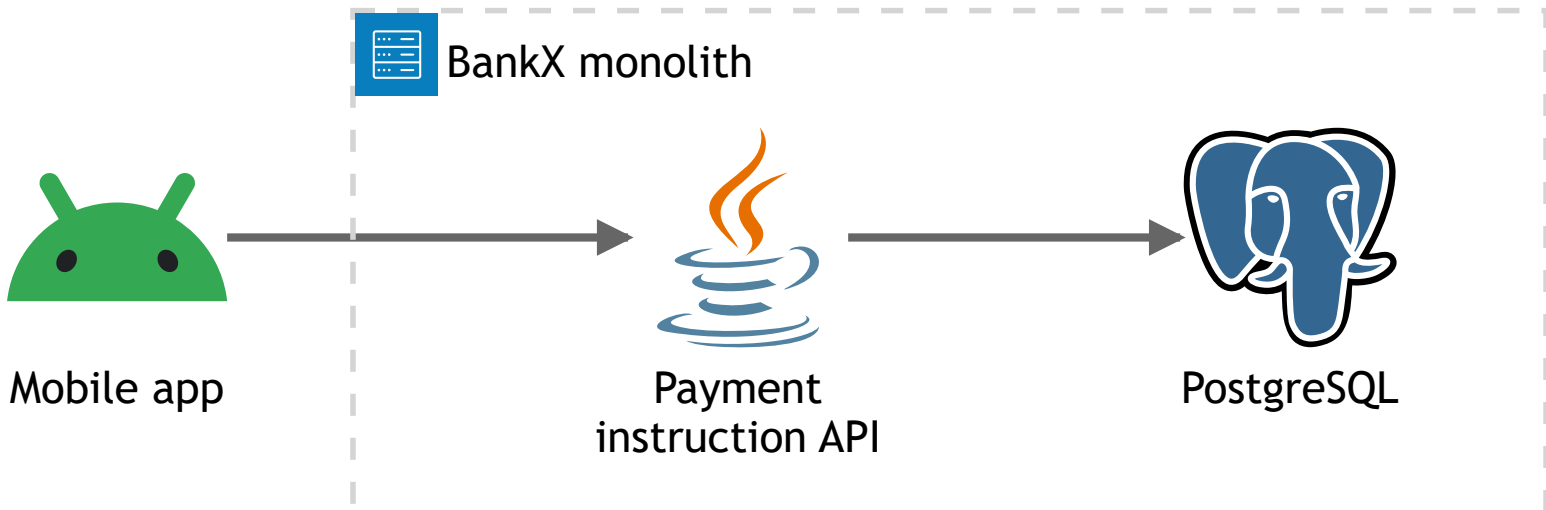
Return errors clients can act on

Every failure comes back as HTTP 200 with success false, so clients retry what cannot succeed: 31% of retried calls fail again against a target of 0 failures answered with 200.

Every outcome is answered with HTTP 200

Component	What it is	What it does
Mobile app	BankX customer app	Reads success from the body and retries when it is false
Payment instruction API	The instruction and approval routes in the Java monolith	Answers 200 with success false and a message for every failure
PostgreSQL	Database that holds instructions and approvals	Holds the state that decides whether a request can succeed
Monitoring	Dashboards and alerts built on HTTP status	Counts the 200 responses and sees no errors

Current architecture



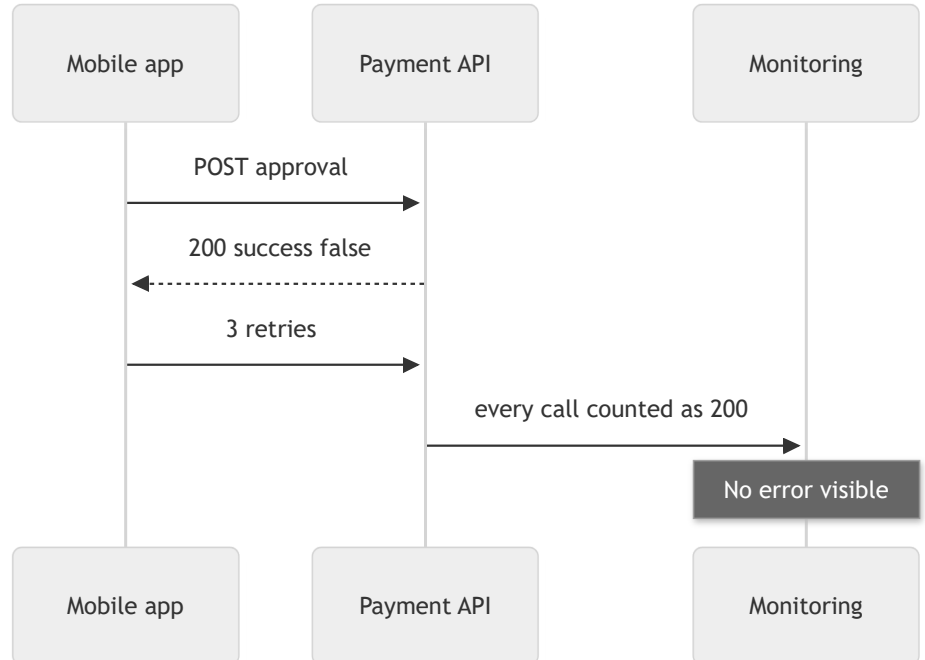
A failure that looks like success is retried

An approval fails because the instruction was cancelled. The API answers 200 with success false, the app retries three times, and monitoring sees no error.

31%
of retried calls fail again; the error cannot succeed

1 status
200 answers every kind of outcome

0
failures answered with 200



Ways to report a failure to a client

Solution	What it is	What it does
Status codes + problem details	Use the HTTP status that names the failure class and a problem details body with type, title, detail and fields	Clients, proxies and monitoring read the class, and clients read the type to decide what to do
200 + error code in body	Keep 200 and add a stable errorCode and a retryable flag to the body	Clients read the code, so the app can choose whether to retry
Custom error envelope	A fixed wrapper around every response	One shape that clients always unwrap

The failure class must reach every place that decides to retry or alert

Solution	Gains	Costs
Status codes + problem details	Generic tools understand status Retry rules follow the class Monitoring sees real error rates	Every route needs a mapping Clients change their error handling Detail must not leak sensitive data
200 + error code in body	No transport change Old clients unaffected Fast to add	Proxies and dashboards see success Every client parses every body Private codes no tool knows
Custom error envelope	One parser Predictable shape Room for metadata	Status still says 200 Success payloads wrapped too Tools cannot read results

Deciding signal: Who must act on the failure and what they need to know: measure the share of retries that fail again and the share of failures visible to monitoring. If retry decisions or alerts depend on the class of failure, the status code must carry it, because a body code alone does not reach proxies, caches or dashboards

Smallest safe move

One status per failure class, one problem body

- **Scope:** Contract, clients: five failure classes on instruction and approval routes
- **Change:** Return 409 for conflicts, 422 for invalid evidence, 401, 403 and 429 with Retry-After, each with an application/problem+json body (type, title, status, code); the old success flag stays one release
- **Safety net:** Details never reveal why a proof failed; 409, 422 and 429 stay apart from a retryable 503; clients get a retry code table
- **Escape route:** A flag returns 200 with the old body to clients with a legacy header
- **Deferred:** Error code as a problem details extension member when clients need finer classes; no custom envelope planned

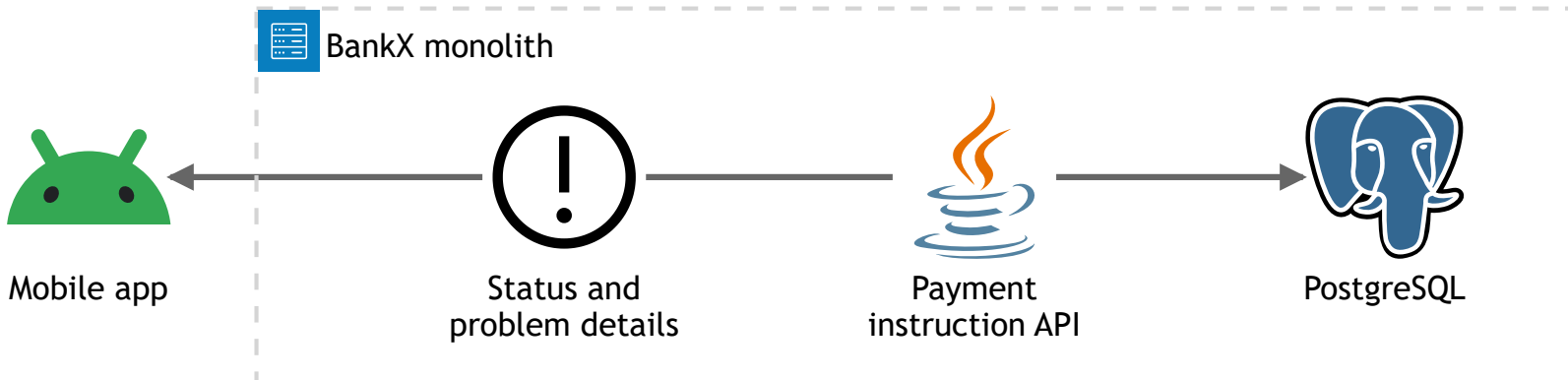
Verification: each failure class provoked and read by client and dashboard

Check	Pass condition
Instruction already cancelled	409 with a problem type; client refreshes and shows the state; no retry
Invalid challenge proof	422 with a generic detail, no sensitive reason; no retry
Missing or expired token	401; client reauthenticates once
Caller above its limit	429 with Retry-After; client waits at least that long
Dashboard error rate during a conflict test	Counts the 409 responses; no failure appears as 200
Fitness function	A contract test asserts that no route answers 200 with an error body and that every 4xx and 5xx has a problem details body with a registered type; the dashboard alerts on the 5xx rate

Rollback: Routing rollback: the legacy-header flag restores 200 with success false for old clients and their monitoring gain is lost. State rollback: none; only the response format changed.

Stop: When no failure is answered with 200 and retries of non-retryable errors fall to near zero, no custom error envelope or code registry beyond problem types is added

Solution architecture



Same client, API and database; every failure now leaves the API with its own status and a problem details body, so the app, proxies and dashboards can tell what to do.

Each status tells the client whether to retry, refresh or stop

Status	When it is returned	What the client does
`401 Unauthorized`	User is not authenticated	Reauthenticates according to the documented scheme
`403 Forbidden`	Authenticated user lacks permission	Does not retry unchanged
`409 Conflict`	Instruction already cancelled, settled or unapprovable	Refreshes the resource and its advertised actions
`412 Precondition Failed`	A conditional header such as `If-Match` no longer matches	Refreshes state before deciding whether to retry
`400 Bad Request`	Malformed JSON	Corrects the request syntax
`422 Unprocessable Content`	Semantically invalid authorization evidence	Follows the bank's challenge flow; no sensitive detail leaks
`429 Too Many Requests`	Caller is throttled	Observes `Retry-After`

A 409 body that names the failure and keeps the reason safe

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{ "type": "https://api.bankx.example/problems/instruction-cancelled",
  "title": "The instruction can no longer be approved",
  "status": 409,
  "detail": "The instruction was cancelled before the approval arrived.",
  "code": "instruction-cancelled" }
```

Member	What the client does with it
`type` and `status`	Read the failure class; refresh the instruction and do not retry
`detail`	Shown to the customer; it never carries a sensitive reason
`code`	A stable value the app can switch on

Glossary

Used in this step

Term	Definition
Status codes + problem details	The status names the failure class and the body states its type and detail
200 + error code in body	Success status with the failure reported only inside the body
Custom error envelope	A private wrapper around every response that holds data or an error
Problem details	Standard machine-readable error body, application/problem+json
Status code class	First digit of a status: 2xx success, 4xx client error, 5xx server error
Retry-After	Header that tells the client how long to wait before the next try
409 Conflict	Status code for a request that conflicts with the current state of the resource
422 Unprocessable Content	Status code for a well-formed request the server will not accept
429 Too Many Requests	Status code for a caller that sent too many requests
Retryable failure	Failure that may succeed if the same request is sent again
Self-descriptive message	Message whose status, headers and body tell a generic client how to read it

Related concepts

Term	Definition
Retry with Backoff	Repeats failed operations with increasing delays.
Idempotency Key	Stable identifier recognizing multiple attempts at the same logical command
Circuit Breaker	Stops requests to failing dependencies temporarily.
Rate Limiter	Controls the number of requests allowed within a time window.

API10 · Workflow resources and contracts

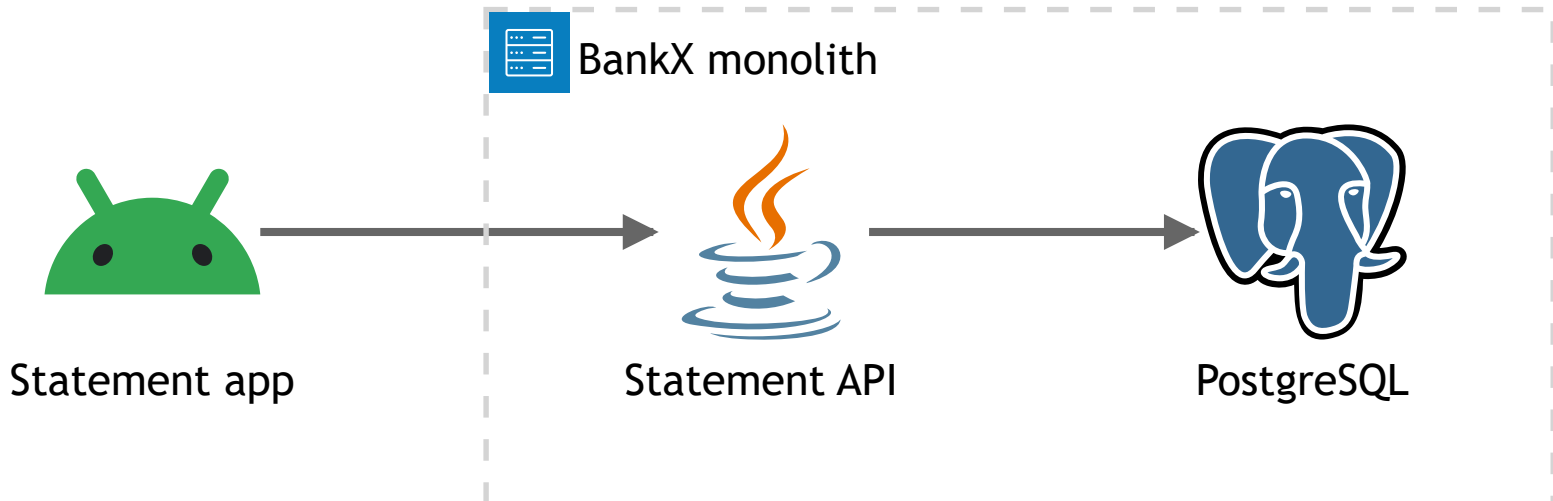
Let caches serve statements safely

Every statement is fetched again on each open although closed months do not change: 70% of requests return an unchanged statement, against a target of ≥ 90% of repeat requests served from the client’s own cache or with 304.

Every statement response is no-store

Component	What it is	What it does
Statement app	BankX customer app on the phone	Opens up to 24 monthly statements and fetches each one again
Statement API	GET /accounts/A1/statements/2026-10 in the Java monolith	Returns each statement with Cache-Control: no-store
PostgreSQL	Database that holds accounts and statements	Is read on every request, although closed months do not change
Cache-Control no-store	Response header set on all sensitive financial data	Forbids the client and every cache to store the response

Current architecture



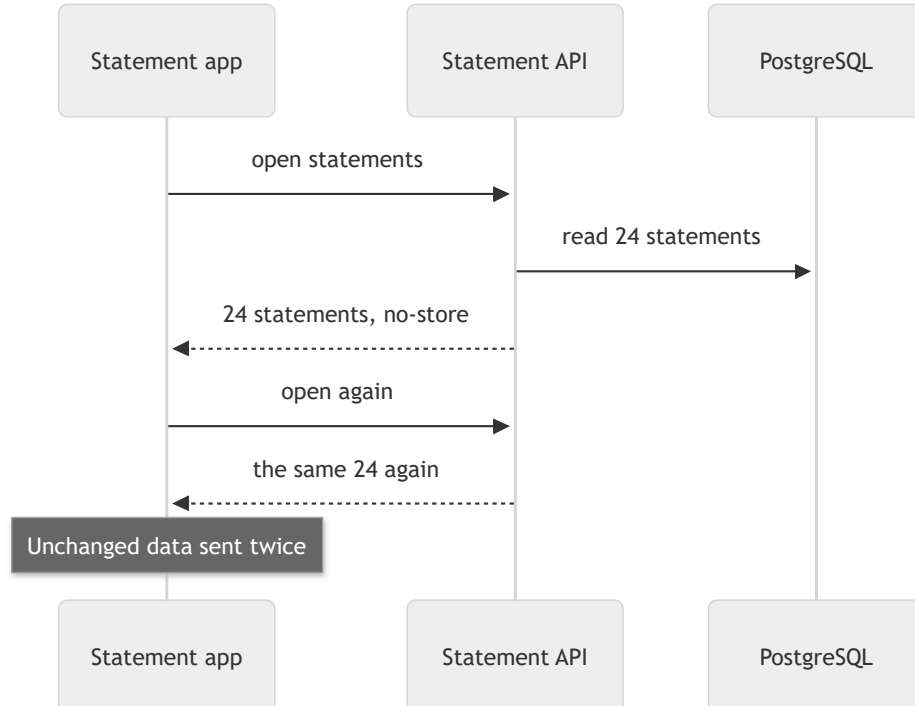
Closed statements are fetched again and again

At month-end each open downloads up to 24 monthly statements again, although closed months do not change. Caching them publicly would let a shared cache serve one customer's statement to another.

70%
of statement requests return an unchanged statement

24 months
of statements a customer may fetch per open

≥ 90%
of repeat requests answered by own cache or 304



Ways to let a client or a cache reuse a statement

Solution	What it is	What it does
no-store everywhere	Cache-Control no-store on every statement response	Nothing is stored by the client or by any cache
Private cache + ETag validators	Cache-Control private with an ETag; clients revalidate with If-None-Match and get 304 when unchanged	Only the customer's own client stores the statement and asks cheaply whether it changed
Public shared cache	Cache-Control public with a max-age on a CDN or proxy	A shared cache answers repeat requests without reaching the API

Reuse saves load, and every shared copy of personal data is a risk

Solution	Gains	Costs
no-store everywhere	Safest default No stale statement Nothing to configure	Every open downloads again Load peaks at month-end No benefit from validators
Private cache + ETag validators	Unchanged statement costs a 304 Personal data stays private Open month stays fresh	Clients must implement revalidation A tag per statement to compute Device holds sensitive data
Public shared cache	Largest load reduction Works for every client Offloads the API	Customer data in a shared cache One key mistake leaks a statement A hit bypasses authorization

Deciding signal: Whether the representation is personal and how often it changes: measure the share of unchanged responses and the hit rate a private cache would reach. Personal data rules out shared caching; closed statements that do not change favor a long private max-age with validators, and the open month needs revalidation on every use

Smallest safe move

Private caching with validators for statements

- **Scope:** Contract, clients, behavior: statements and the list only; other money endpoints keep no-store
- **Change:** Closed statements send Cache-Control private, max-age=86400 and a strong ETag; open month: private, no-cache with an ETag; clients send If-None-Match and get 304; Vary: Authorization
- **Safety net:** private keeps shared caches out, Vary stops cross-user reuse; authorization runs on every revalidation; a corrected statement gets a new ETag; the public variant is not used
- **Escape route:** A flag returns no-store at once; clients simply fetch again
- **Deferred:** Shared caching only for non-personal content such as exchange rates; a CDN for exports behind signed URLs when they grow

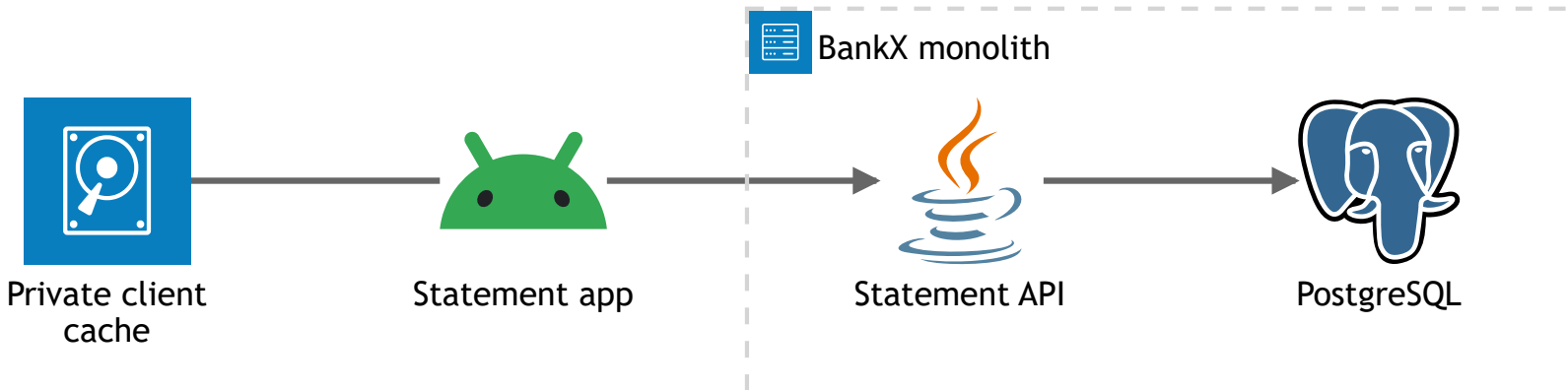
Verification: a repeat open costs a 304 and no shared cache holds a statement

Check	Pass condition
Second open of 24 closed statements	24 responses of 304 or served from the client cache; no statement body downloaded
Statement corrected after it closed	New ETag; the client receives 200 with the new body
Request through a shared proxy test cache	Never stored: no hit, private and Vary honored
User who lost access sends If-None-Match	403, not 304
Headers of balances and instructions	Still no-store
Fitness function	A header test in CI fails when any personal endpoint lacks private or no-store; a month-end metric tracks the 304 ratio and statement bytes per open

Rollback: Routing rollback: the flag returns no-store at once and load rises again. State rollback: copies already stored on devices live until max-age ends; send no-store or a shorter max-age to clear them, and a corrected statement uses a new ETag so no wrong copy survives revalidation.

Stop: When repeat opens cost a 304 or no request and no personal data reaches a shared cache, no CDN or edge cache is added for statements

Solution architecture



Same app, API and database; the app now keeps its own private copy of each statement and asks the API with a validator whether it changed, so a repeat open costs a 304.

Each cache directive says who may store a response and when to reuse it

Directive	What it does	For personal data
`no-store`	No client or cache stores the response	The safe default; kept for balances and instructions
`private`	Only the end client's cache may store it	Required for any personal response; keeps shared caches out
`no-cache`	A stored copy is reused only after revalidation	Fits the open month, which can still change
`max-age`	Seconds a stored copy is fresh without asking	Fits closed months, together with `private`
`public`	A shared cache may store and reuse it	Never: a hit bypasses authorization
`Vary`	Names the request headers that change the response	`Vary: Authorization` stops reuse across users
`ETag` and `304`	A version tag lets a client ask whether its copy is current	A cheap check; authorization still runs each time

A repeat open costs a 304 and no statement body

```
GET /accounts/A1/statements/2026-10 HTTP/1.1
Authorization: Bearer <access-token>
If-None-Match: "st-2026-10-v1"

HTTP/1.1 304 Not Modified
ETag: "st-2026-10-v1"
Cache-Control: private, max-age=86400
Vary: Authorization
```

Part	What happens
`If-None-Match`	The client sends the tag of its stored copy; authorization still runs
`304 Not Modified`	No body; the client reuses the statement it holds
Corrected statement	A new `ETag` means `200` with the new body
User who lost access	`403`, not `304`

Glossary

Used in this step

Term	Definition
no-store everywhere	Every statement response forbids storing, so nothing is reused
Private cache + ETag validators	Only the client's own cache stores it, and a tag shows whether it changed
Public shared cache	A cache used by many users that may store and reuse the response
Cache-Control	Response header that sets how a response may be cached
no-store	Directive that forbids storing the response anywhere
private	Directive that lets only the end client's cache store the response
Validator	Value, such as an ETag, that lets a client ask whether a stored response is current
304 Not Modified	Reply that says the stored copy is still valid and sends no body
Conditional GET	GET that sends If-None-Match or If-Modified-Since
Freshness	How long a stored response may be reused without asking
Shared cache	Cache serving many users, such as a CDN or proxy

Related concepts

Term	Definition
CDN (Content Delivery Network)	Distributes cached content closer to users.
Reverse Proxy	Intermediary that forwards client requests to backend services.
ETag	Opaque version tag of a representation, returned in a response header
Vary header	Tells caches which request headers change the response

API11 · Workflow resources and contracts

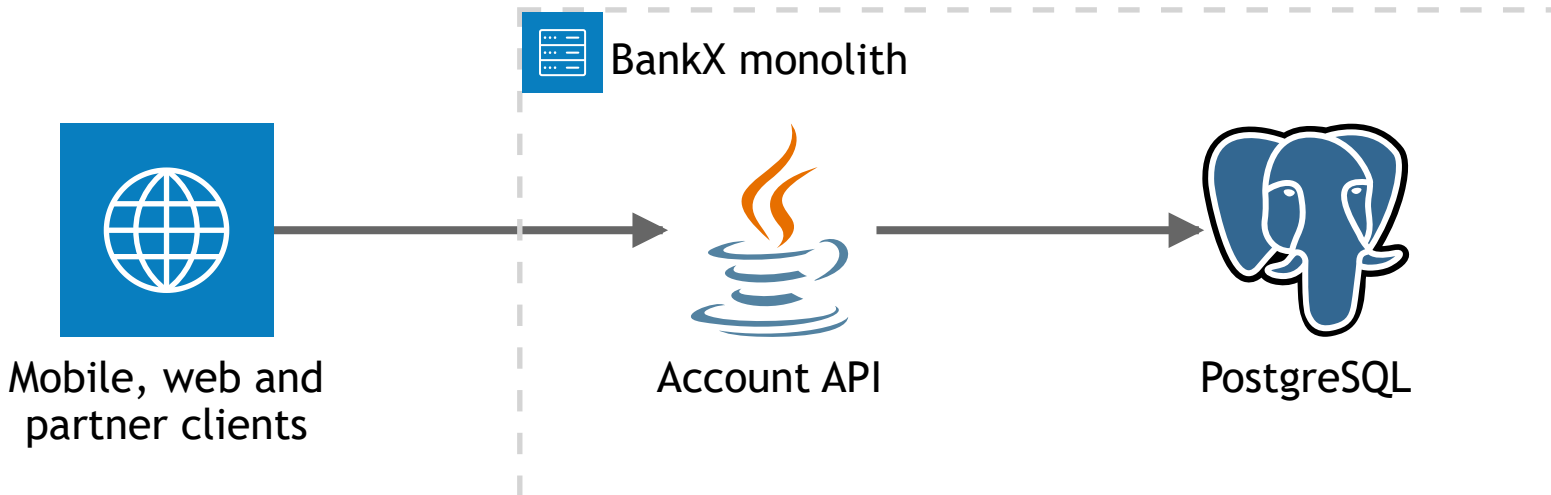
Evolve a representation without breaking old clients

Joint accounts replace holder with holders while old app versions stay in use: 18% of sessions read the single holder field, against a target of 0 errors in old clients.

Three clients read one holder from the account response

Component	What it is	What it does
Mobile app	BankX customer app, released through app stores	Reads holder from the account response; old versions stay in use for months
Web app	BankX web client on its own release schedule	Reads holder from the account response
Partner integration	External client on its own release schedule	Reads holder and validates responses against its schema
Account API	GET /accounts/A1 in the Java monolith	Returns id, number, currency, type, holder with id and name, and status
PostgreSQL	Database that holds accounts and their holders	Stores each account with its single holder

Current architecture



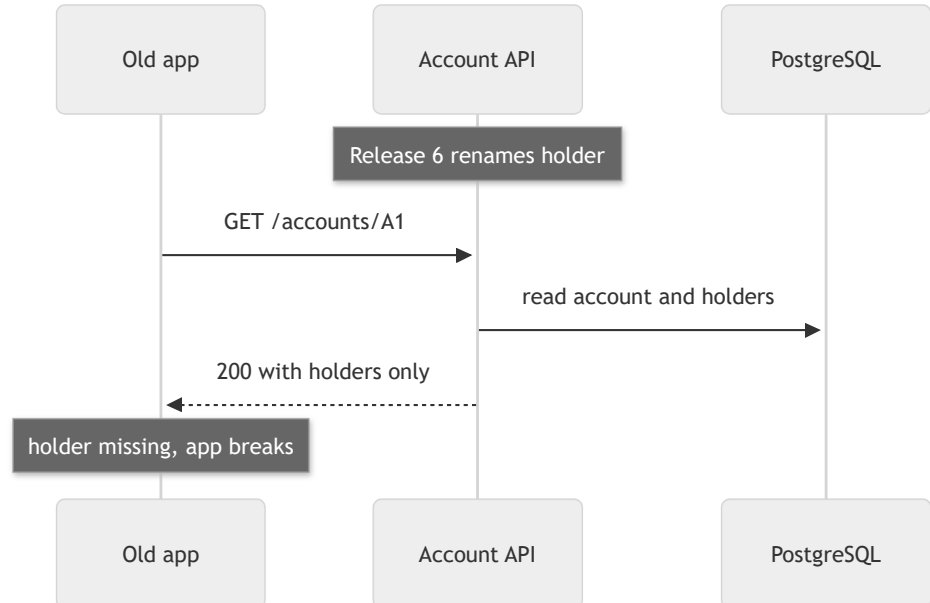
Joint accounts remove the holder old clients read

Release 6 adds joint accounts, so holder becomes holders. Old app versions stay in use for months, still read holder and crash or show an empty name.

18%
of sessions run versions that read holder

3 clients
mobile, web and partner, released separately

0
errors in old client versions after the change



Ways to change a contract that clients already read

Solution	What it is	What it does
Additive change	Add holders next to holder; holder keeps showing the primary holder	Old clients read what they always read and new clients read holders
URL version	Publish /v2/accounts with holders and keep /v1 until clients move	Old and new clients each use their own contract
Media type version	Clients ask for application/vnd.bankx.account.v2+json through Accept	One URL serves each client the version it asks for
Rename in place	Replace holder with holders in the existing response	One model for every client

Every way to change a contract leaves a cost with someone

Solution	Gains	Costs
Additive change	Old clients unchanged One URL Expand now, contract later	Two fields to keep in step holder now means primary holder Removal needs a plan
URL version	Clean new model Cut-over per client Old contract untouched	Two APIs to run and test Links must name a version Every resource copied or versioned
Media type version	URLs stay stable Version per resource Fits content negotiation	Caches need Vary Harder to try by hand Server keeps both renderers
Rename in place	No duplication Simplest code Clean model at once	Breaks old clients at once Needs a coordinated release No way back after rollout

Deciding signal: How long old clients stay in use and whether they can be forced to upgrade: measure the share of traffic from old versions per week and the time until it reaches zero. If old versions stay for months and cannot be forced, the change must be additive or versioned; if the only client ships with the API, renaming is cheap. Versioning pays only when the change cannot be additive

Smallest safe move

Add holders, keep holder as the primary holder

- **Scope:** Contract, clients, data: the account representation and a holders table
- **Change:** Add holders (id, name, role, permissions), keep holder as the primary holder; a deprecation header announces removal once old-version traffic falls to the agreed share
- **Safety net:** Rename in place is not used; contract tests from old app versions run before each release; permissions are enforced on the server
- **Escape route:** A flag drops holders from the response; holder was never removed
- **Deferred:** URL version when a field changes meaning; media type version when clients need different views of one URL

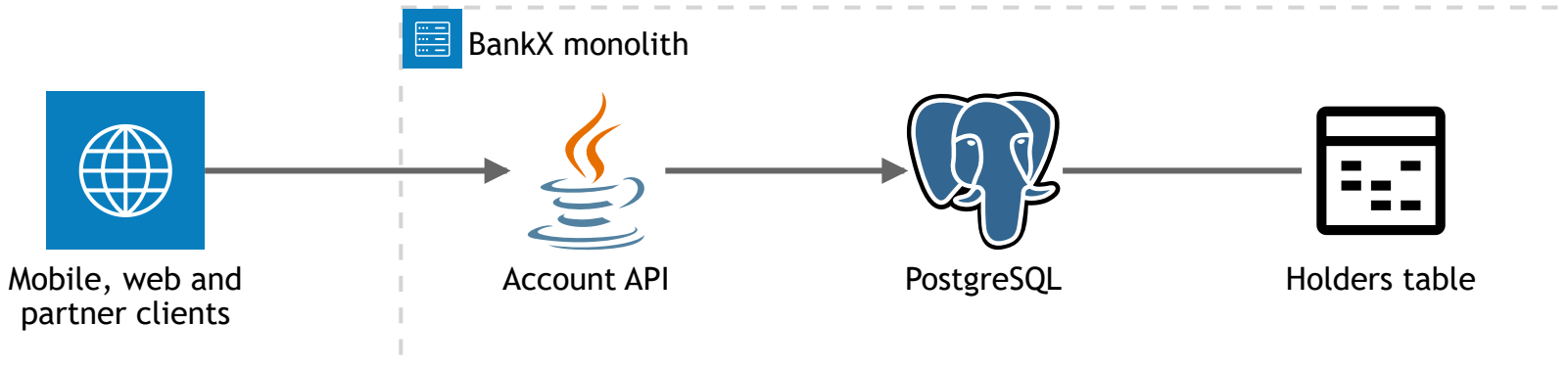
Verification: the oldest clients replayed against the new API

Check	Pass condition
Oldest app version reads an account with two holders	Renders without error and shows the primary holder
New client reads the same account	Sees both holders with their permissions
Partner client's schema validation against the new response	Passes, because only fields were added
Single-holder account	holder and holders agree
Share of traffic on old versions	Reported weekly; removal blocked above the agreed share
Fitness function	Consumer-driven contract tests from each supported client version run in CI and block a release that changes or removes a field they use

Rollback: Routing rollback: the flag removes holders from responses and old clients never depended on it. State rollback: the holders table keeps the data and joint accounts created during the change stay valid; nothing is rolled back.

Stop: When old clients keep working on the additive contract and old-version traffic falls to the agreed share, no versioned API, gateway translation layer or custom media type is added

Solution architecture



Same clients, API and database; the account response now carries both the old holder field and the new holders list, backed by a holders table.

One response that old and new clients both read

```
GET /accounts/A1 HTTP/1.1

HTTP/1.1 200 OK
{ "id": "A1", "currency": "PLN", "status": "active",
  "holder": { "id": "C7", "name": "Anna Nowak" },
  "holders": [
    { "id": "C7", "name": "Anna Nowak", "role": "owner", "permissions": ["view", "pay"] },
    { "id": "C9", "name": "Jan Nowak", "role": "co-holder", "permissions": ["view"] }
  ] }
```

Release 6 adds `holders` and leaves `holder` in place; no URL, method or status code changes.

Reader	What it uses	What happens
Old app version	`holder`	Shows the primary holder, as before
New client	`holders`	Shows both holders with their permissions
Partner schema check	Fields it already knows	Passes, because only fields were added

Each way to change a contract fits one situation

Style	Fits when	Who pays
Additive change	Old clients stay for months and cannot be forced to upgrade	The server keeps two fields in step until removal
URL version	A field changes meaning, so no additive form exists	The server runs two APIs, and links must name a version
Media type version	Different clients need different representations of one URL	The server keeps both renderers, and caches need Vary
Rename in place	The only client ships with the API	Every old client at once, with no way back

Versioning pays only when the change cannot be additive. The measurement that decides is the share of traffic from old versions per week.

Glossary

Used in this step

Term	Definition
Additive change	A change that only adds fields or routes and removes or renames nothing
URL version	A version number in the path, such as /v2, that selects the contract
Media type version	A version carried in the media type that a client asks for
Rename in place	Changing an existing field in the live response without a transition
Expand-Contract	Add compatible representation, migrate consumers, then remove old representation
Backward compatibility	New server versions keep working with existing clients
Breaking change	A change that makes an existing client fail or misbehave
Consumer-driven contract test	Consumer expectations verified against a provider
Deprecation	Announced end of life of a field or version
Joint account	Account with several holders, each with individually defined permissions
Primary holder	The holder shown by the old single-holder field

Related concepts

Term	Definition
Hypermedia	Responses that advertise the links and actions available next
OpenAPI document	Machine-readable description of an HTTP API's routes and messages
Content negotiation	Client and server agree on a representation through Accept and Content-Type
Semantic versioning	Version numbers whose parts signal the compatibility of changes

Phase 3

Synchronous and asynchronous interaction

Steps in this phase

Step	Forcing constraint
API12 Make a lost response safe to retry	The client times out after the server committed, and the retry creates a second payment
API13 Stop slow work from blocking the request	A statement export covering up to 24 months outlasts the gateway timeout
API14 Hand over the finished result and expire it	Finished exports must stay retrievable for a defined time and then disappear without breaking clients
API15 Learn the result without hammering the server	50,000 clients require notice within 2 seconds
API16 Represent a transfer that completes days later	Clearing for transfers to other banks follows clearing schedules and can finish days after the HTTP exchange or fail, while customers read accepted as sent
API17 Cancel while clearing is running	A cancellation request races with execution: it can arrive after the transfer was submitted to clearing, and both can win
API18 Deliver status webhooks to partner systems	Partner endpoints time out after 20 seconds or fail, so one event is delivered up to 5 times, out of order, and any caller can forge one
API19 Slow callers down instead of failing	1000 msgs/s arrive; only 500 msgs/s processed; backlog grows 500/s

API12 · Synchronous and asynchronous interaction

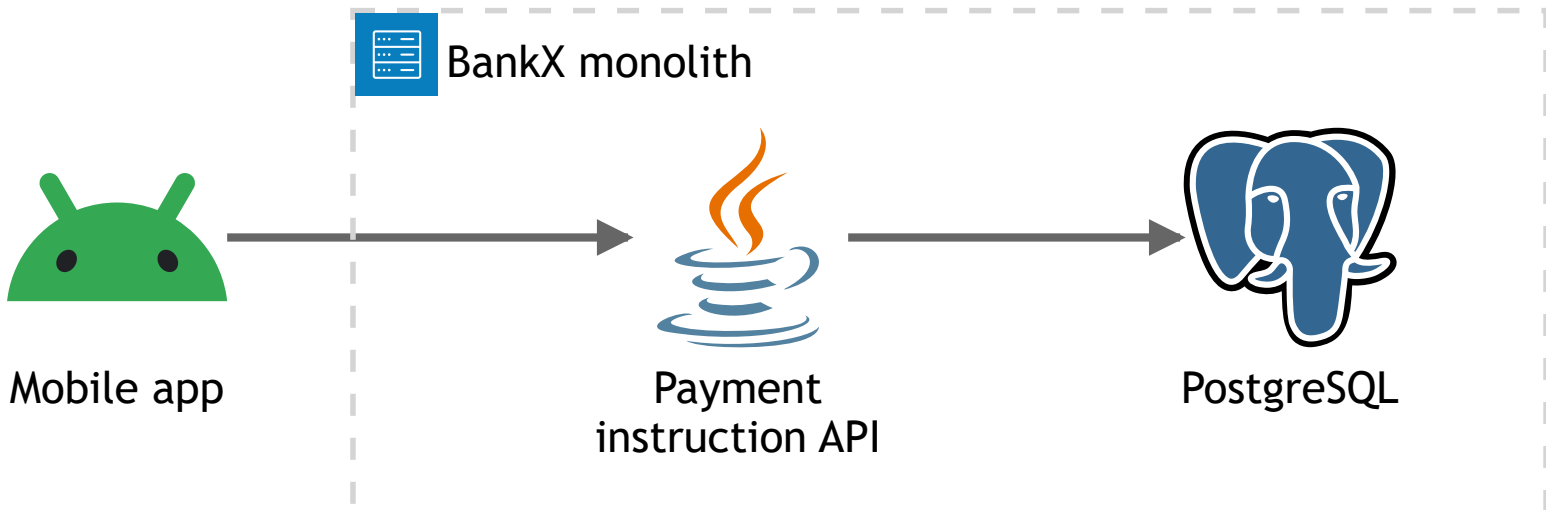
Make a lost response safe to retry

A response lost after the server committed makes the client retry, and BankX creates a second payment instruction: 12 duplicates per week against a target of 0.

A submission creates an instruction and answers 201

Component	What it is	What it does
Mobile app	BankX customer app on the customer’s phone	Sends POST /payment-instructions with a 10 s timeout and sends it again after a timeout
Payment instruction API	The POST and GET routes for payment instructions in the Java monolith	Validates the request, creates the instruction and answers 201 with a Location
PostgreSQL	Database that holds accounts, beneficiaries and instructions	Stores the new instruction in one transaction
Mobile network	The connection between the phone and the API	Can drop a response after the server has already committed

Current architecture



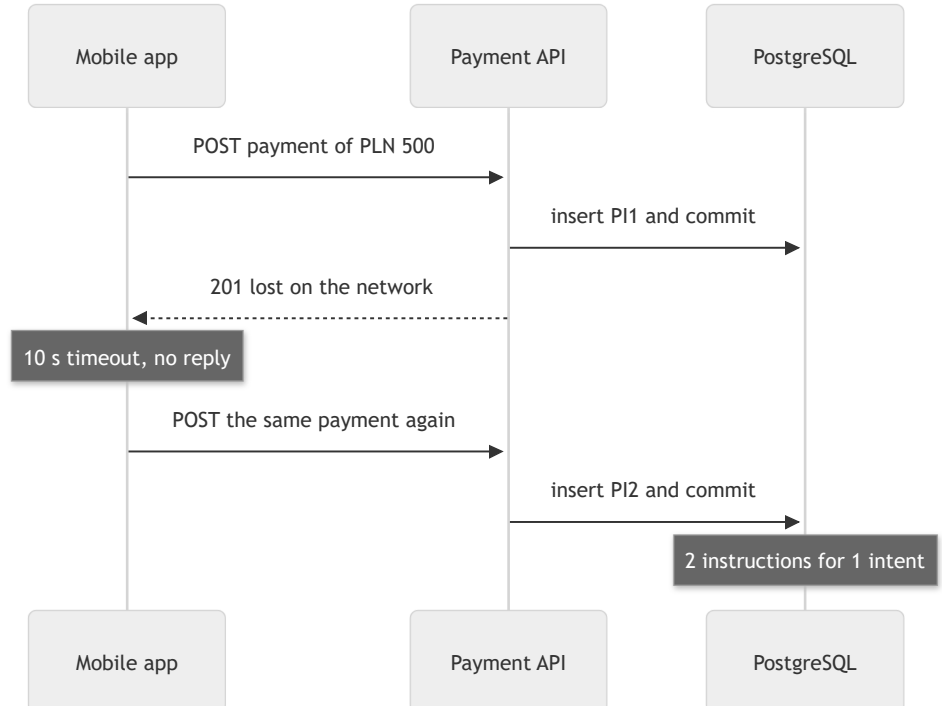
The reply is lost and the retry pays twice

A customer submits PLN 500 over a weak mobile connection. The server commits the instruction and replies, but the reply is lost. The app times out and sends the request again, and the bank now holds two instructions for the same rent.

12 per week
duplicate instructions created by retries after a lost response

10 s
client timeout, shorter than the slowest 1 in 200 server responses

0
duplicate instructions in a test that drops every response and retries



Ways to make a repeated request harmless

Solution	What it is	What it does
Idempotency-Key header	The client generates a key per intent and sends it in a header; the server stores the key, a fingerprint of the request and the response	A retry with the same key returns the stored result and creates nothing
Unique client reference	The payment reference sent by the client is unique per account and enforced by a unique constraint	A second request with the same reference is refused or returns the first
Check before retry	After a timeout the client asks the server whether the payment exists and sends the POST again only if not	Avoids a duplicate when the first call really failed

Each guarantee has to live somewhere, and it costs something there

Solution	Gains	Costs
Idempotency-Key header	Safe retries for any POST Works for any payload Stored response is replayed	Key scope, expiry and storage to define Same key with other body must be refused Client keeps a key per intent
Unique client reference	No header or key store Uses data already sent Database enforces it	Customers reuse references legitimately A refusal looks like an error Business meaning mixed with retry safety
Check before retry	No server change Simple to explain Uses the existing GET	First request may still be running Client may lack a lookup key Extra round trip on every timeout

Deciding signal: The cost of a duplicate and whether the client can name its intent: measure duplicates per week after lost responses and how often a legitimate repeat (same reference, same amount) occurs. A duplicate that moves money needs a server-side guarantee, so the choice is between a key per intent and a unique business reference; the reference works only if a repeat with the same value is never legitimate

Smallest safe move

One key per intent, stored with the instruction

- **Scope:** Contract, behavior, clients, data: POST /payment-instructions and a key table kept 24 hours
- **Change:** Require an Idempotency-Key header; store key, body hash and response with the instruction; a repeat returns the stored 201; another body with the same key gets 422
- **Safety net:** Key and effect commit together; keys are per user and route and outlive the retry window; check before retry is not used, it races
- **Escape route:** A flag makes the header optional again; stored keys stay
- **Deferred:** Unique client reference when a customer reference is a business rule; retry budget and circuit breaker when retries amplify load

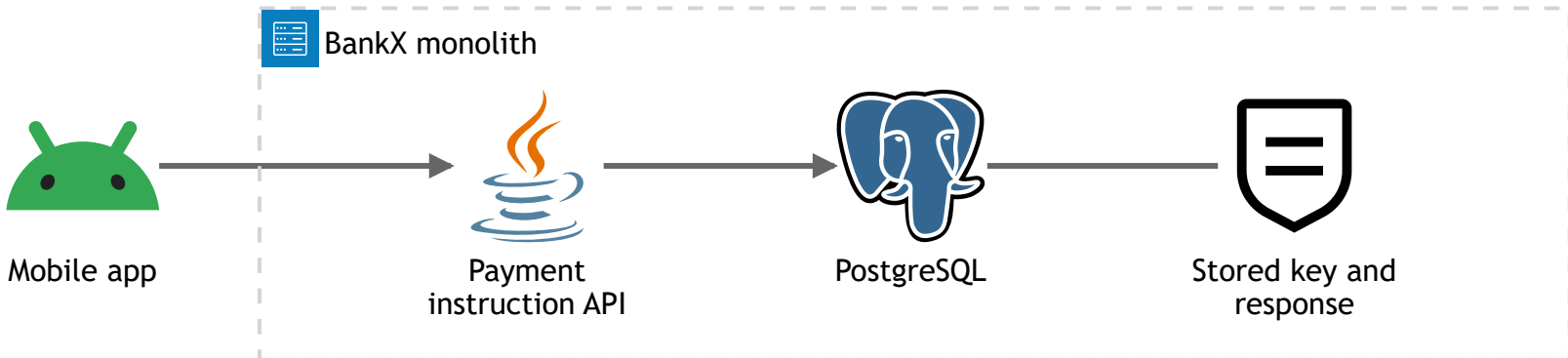
Verification: every response dropped, every request repeated

Check	Pass condition
Response dropped after commit, client retries with the same key	One instruction; the retry returns the stored 201 and Location
100 concurrent requests with one key	Exactly one instruction
Same key with a different body	422 and no new instruction
Server crash between key write and response	No instruction without a key and no key without an instruction
Key used after the retention period	Treated as a new request, as documented
Fitness function	A fault-injection test in CI drops responses and fires parallel duplicates, and fails on more than one instruction; a metric counts replayed keys per day

Rollback: Routing rollback: the flag makes the header optional again, and duplicates are possible for clients that send no key. State rollback: duplicates created before the change stay; find pairs with the same account, beneficiary, amount and a short time window, and cancel the extra instruction before execution.

Stop: When the duplicate test creates one instruction and clients retry with the same key, no distributed lock, exactly-once messaging platform or client-side deduplication service is added

Solution architecture



Same client, API and database; the database now keeps each key with its response, so a repeated request returns the stored result instead of creating a second instruction.

A retry that returns the stored result

```
POST /payment-instructions HTTP/1.1
Idempotency-Key: 4a8d1d6c-illustrative-retry-key

{ "amount": { "value": "500.00", "currency": "PLN" } }

HTTP/1.1 201 Created
Location: /payment-instructions/PI1
```

The first attempt creates PI1. The retry carries the same key and body and receives the stored reply.

Request	What the server does
Same key, same body	Returns the stored `201` and `Location`; no second instruction
Same key, other body	Answers `422`; nothing is created
New key	Treats it as a new intent and creates a new instruction

What the key contract has to state

Rule	In this API	Why
Meaning	One key names one intent, not one attempt	Every retry of an intent carries the same key
Scope	Bound to one user and one route	A stored response never replays to another caller
Other body	Same key with another body gets 422	The request fingerprint differs
Still running	Wait or answer 409 until the first ends	Two inserts must not race
Retention	Longer than the retry window, here 24 hours	A late retry still finds its key
Commit	One transaction for key, response and instruction	A crash leaves neither half

Glossary

Used in this step

Term	Definition
Idempotency-Key header	Request header that carries a client-chosen key naming one logical command
Unique client reference	A client-supplied business reference that must be unique per account
Check before retry	Client asks whether the operation happened before sending it again
Idempotency Key	Stable identifier recognizing multiple attempts at the same logical command
Request fingerprint	Hash of the request stored with the key to spot a different request
Lost response	The server did the work but the client never received the reply
Client timeout	Time after which the client stops waiting for a response
Retry with Backoff	Repeats failed operations with increasing delays.
Jitter	Randomized delay around retries
Key retention	How long stored keys are kept to recognize late retries
Exactly-Once Effect	Application-visible effect occurs once despite repeated attempts under defined guarantees

Related concepts

Term	Definition
At-Least-Once Delivery	Delivery semantics allowing duplicate deliveries in exchange for retries
Circuit Breaker	Stops requests to failing dependencies temporarily.
Retry budget	Cap on retries relative to normal calls, so retries cannot multiply the load
Unique constraint	Database rule that rejects a second row with the same key

API13 · Synchronous and asynchronous interaction

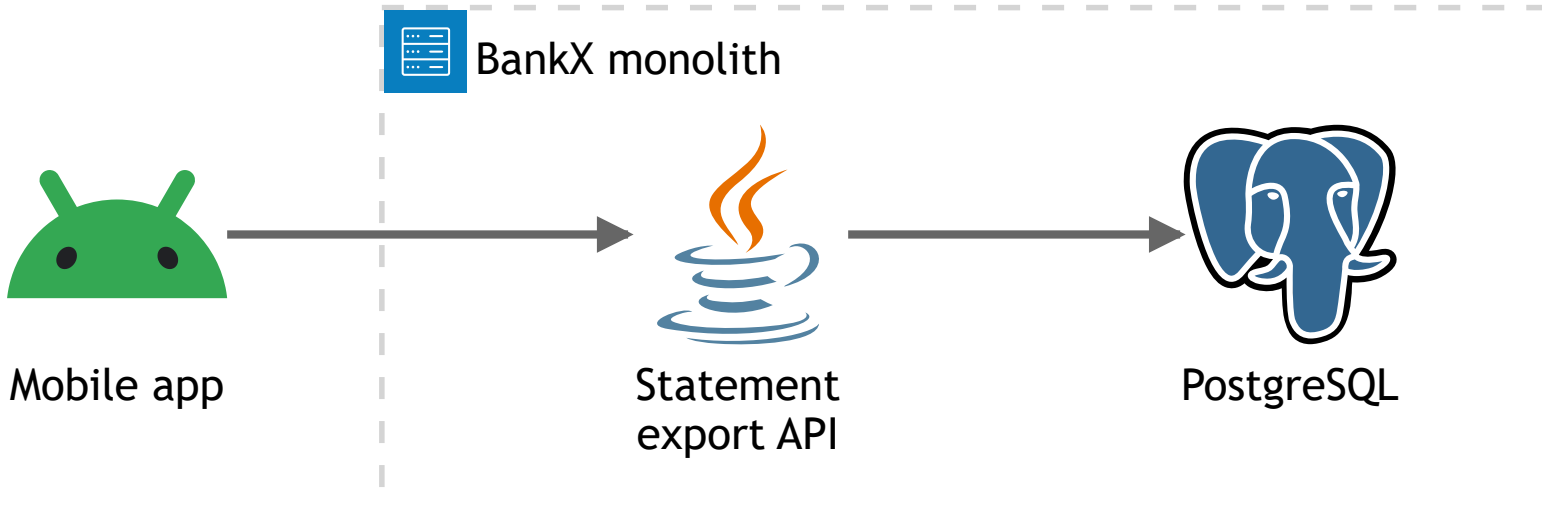
Stop slow work from blocking the request

A statement export covering up to 24 months outlasts the 30 s gateway timeout: 38% of long exports time out, against a target of acknowledging the request in ≤ 1 s.

An export is built inside the request and returned when ready

Component	What it is	What it does
Mobile app	BankX customer app on the customer’s phone	Posts the export request and sends it again after a timeout
Gateway	Front door that cuts requests at 30 s	Gives up before a long export is ready
Statement export API	POST /statement-exports in the Java monolith	Builds the export inside the request and returns it when it is ready
Request workers	Threads that serve one request at a time	Each export holds one worker for the whole build
PostgreSQL	Database that holds accounts and statement lines	Is read to build the export

Current architecture



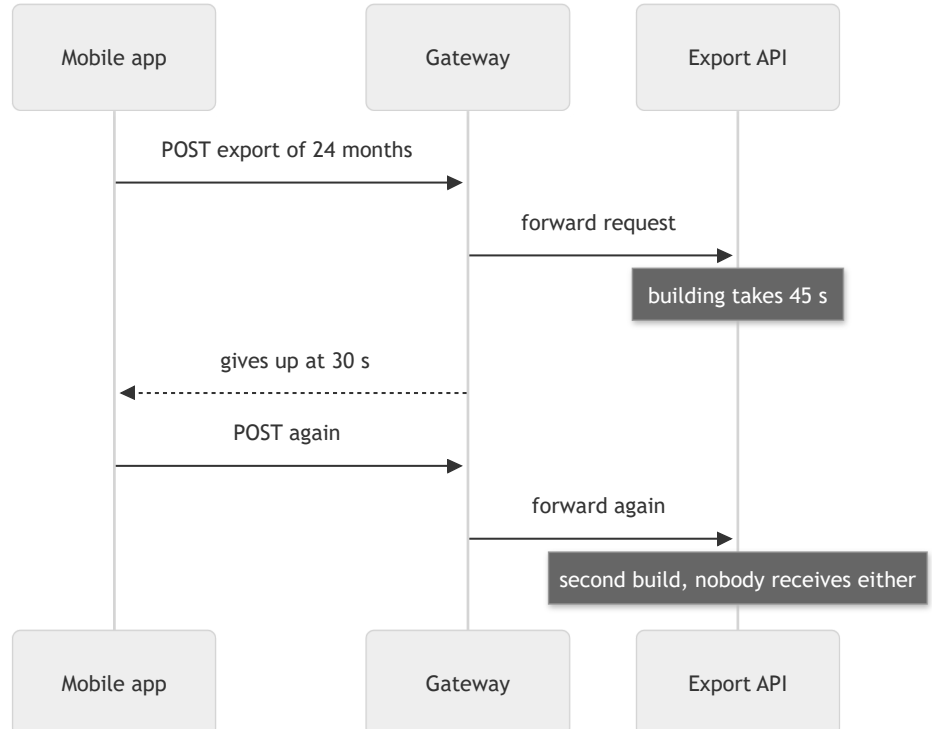
The gateway gives up before the export is ready

A customer requests 24 months of statements. Building the file takes 45 s, the gateway gives up at 30 s, the app retries, and request workers stay busy producing exports nobody receives.

38%
of 24 month exports time out at the gateway

45 s
to build a 24 month export, held by one request worker

≤ 1 s
to acknowledge the request, with the result delivered later



Ways to handle work that outlasts a request

Solution	What it is	What it does
Raise timeouts	Raise the gateway and client timeouts above the longest export	The request stays open until the file is ready
202 Accepted + operation resource	POST answers 202 with the URL of an operation resource that reports progress	The request is accepted at once and the client checks the operation
201 Created + pending resource	POST answers 201 with the export resource itself in a pending state	The export is its own status resource and is read until it is ready

The wait has to go somewhere: connections, workers or a new resource

Solution	Gains	Costs
Raise timeouts	No API change Result in one call Quick to apply	Workers held for the whole build Longer limits for every route Connections drop on mobile networks
202 Accepted + operation resource	Fast acknowledgement Progress and failure are visible Known pattern for long work	Client must poll or be told New resource and lifecycle Result needs a second hop
201 Created + pending resource	One resource, no extra type Natural when the export is an entity Clear URL for the result	201 may suggest the file exists Status and content share one lifecycle Poor fit for work with no result entity

Deciding signal: How long the work takes against the time a request may be held, and how many workers it blocks: measure export duration by period length, the share above the gateway timeout and worker occupancy (arrival rate times duration). If most work finishes well inside the limit, raising it is enough; if the tail passes the limit or occupancy approaches the worker count, accept the work and finish it later

Smallest safe move

Accept now, build later in a worker

- **Scope:** Contract, behavior, clients: statement exports only; other routes unchanged
- **Change:** POST /statement-exports stores the request and answers 202 with Location /operations/OP1 and Retry-After; a worker builds the export; GET /operations/OP1 shows queued, processing, failed or completed
- **Safety net:** 202 promises acceptance, not completion; the request is stored before 202; the idempotency key protects the POST; no half-built file is offered
- **Escape route:** A flag sends periods under one month to the synchronous path; the operation route stays
- **Deferred:** 201 with a pending resource if exports become managed entities; raised timeouts only where the slowest requests stay under the limit

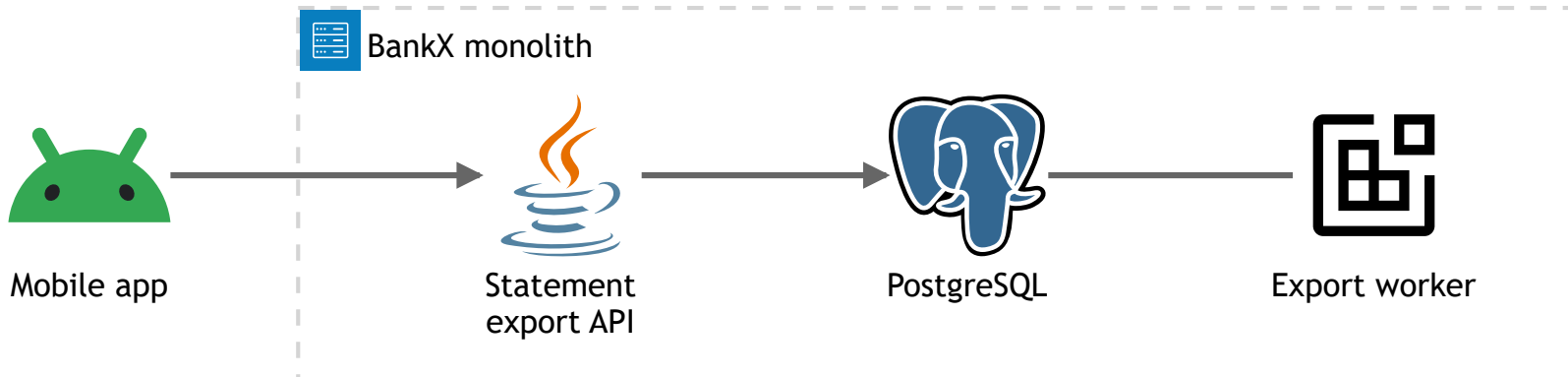
Verification: long exports under load with the gateway cut at 30 s

Check	Pass condition
24 month export requested	202 with Location within 1 s; no request is held beyond the acknowledgement
Operation read while the worker builds	200 with processing
Worker crashes mid-build	Operation returns to queued or failed; no half file offered
Same request sent twice with one key	One operation
100 exports requested at once	Request workers stay free for other routes; acknowledgements stay within 1 s
Fitness function	A load test requests exports above worker capacity and fails if any request waits longer than the acknowledgement budget or if other routes slow down

Rollback: Routing rollback: the flag sends small exports back to the synchronous route, and the long ones fail at the gateway again. State rollback: operations and files already created stay; unfinished operations are completed or marked failed by the worker, and no data other than export records changed.

Stop: When exports are acknowledged within the budget and the request workers stay free, no job platform or workflow engine is added

Solution architecture



Same client, API and database; the API only stores the request and answers 202, and a worker builds the export outside the request.

An export request answered at once with an operation to watch

```
POST /statement-exports HTTP/1.1
Idempotency-Key: K1

{ "account": "/accounts/A1", "period": "2026-10" }

HTTP/1.1 202 Accepted
Location: /operations/OP1

GET /operations/OP1  →  200 { "status": "processing" }
```

The `Retry-After` header on the `202` tells the client when to look again.

Message	What the client may conclude
`202` with `Location`	Accepted for later processing; no completed statement is promised
`200` with `processing`	The operation exists and is still running; read it again later
Same `POST` with the same key	One operation, not a second export

POST does not always answer 201

Status	What it says	What the client does
`200`	The request was handled; the body is the answer	Reads the body, or the operation again while it runs
`201`	A resource was created; `Location` names it	Follows `Location`; a pending resource may lack the file
`202`	Accepted for later; no completed statement is promised	Follows the operation link; does not assume completion
`303`	The result is in another resource, named by `Location`	Follows `Location` with GET to read the result

Glossary

Used in this step

Term	Definition
Raise timeouts	Lengthen the limits so a slow request can finish inside one call
202 Accepted + operation resource	Accept now, and report progress and outcome through a separate resource
201 Created + pending resource	Create the result resource at once in a pending state
202 Accepted	Status code that says the request was accepted for processing, not that it finished
Operation resource	Resource that reports the progress and outcome of a long-running request
Gateway timeout	Time after which the gateway stops waiting for the API
Request worker	Thread that serves one request at a time; a blocked worker serves nobody else
Worker occupancy	Workers busy at once: arrival rate times the time each call holds a worker
Retry-After	Header that tells the client how long to wait before the next try
Statement export	Downloadable statement covering a selected period, up to 24 months
Durable acceptance	The request is stored before 202 is sent

Related concepts

Term	Definition
Message Queue	Buffers messages for asynchronous processing by consumers.
Idempotency Key	Stable identifier recognizing multiple attempts at the same logical command
Polling	Repeated requests to determine whether data changed
Dead Letter Queue (DLQ)	Stores messages that could not be processed successfully.

API14 · Synchronous and asynchronous interaction

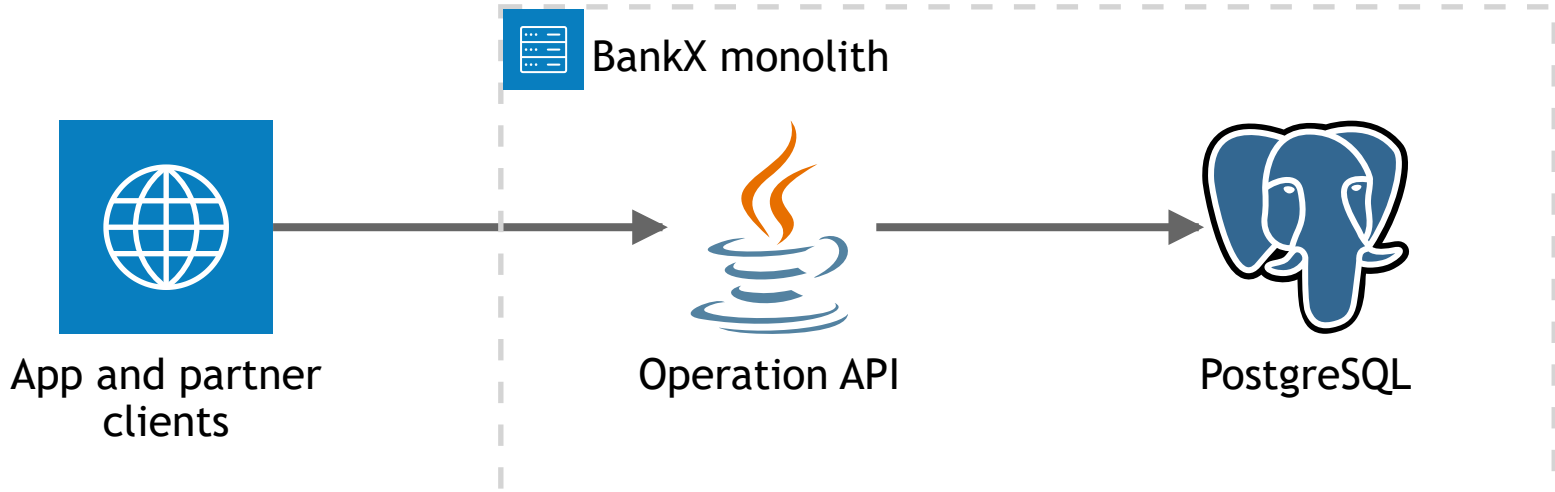
Hand over the finished result and expire it

Finished exports must stay retrievable for 7 days and then expire cleanly: every poll of a completed operation returns 40 MB, against a target of 0 clients that get a bare 404 for an expired export.

A completed operation carries the export file itself

Component	What it is	What it does
Mobile app	BankX customer app on the customer’s phone	Polls the operation after it completes and receives the file each time
Partner integration	External client that stores operation URLs	Opens a stored operation URL weeks later
Operation API	GET /operations/OP1 in the Java monolith	Reports processing and then completed, with the export file attached
PostgreSQL	Database that holds operations and, for now, the finished files	Keeps every file until a clean-up that nobody owns

Current architecture



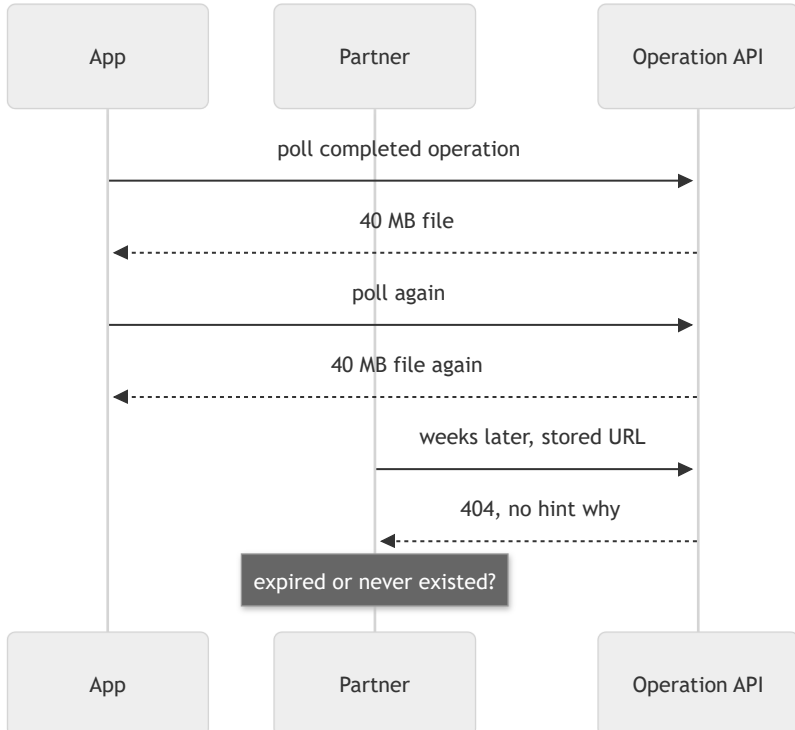
Every poll returns the file; expiry answers 404

After completion every poll returns the 40 MB file again. A partner stores the operation URL and weeks later gets a bare 404 with no hint that the export expired.

40 MB
returned by every poll of a completed operation

7 days
how long a finished export must stay retrievable

0
clients that get a bare 404 for an expired export



Ways to hand over a finished result

Solution	What it is	What it does
Result inside the operation	The completed operation response carries the file or its content	One URL answers both status and result
303 See Other to a result resource	A finished operation answers 303 with Location /statement-exports/EX1	The result is its own resource with its own cache rules and lifetime
Signed storage URL	The operation links to a time-limited signed URL on object storage	The client downloads directly from storage, not through the API

The result’s size, lifetime and audience decide where it lives

Solution	Gains	Costs
Result inside the operation	One call after completion No second resource Simple for clients	Every poll can carry megabytes Status and file share one lifetime Result cannot be handed to another party
303 See Other to a result resource	Status stays small Result has its own URL and headers Standard redirect semantics	One more request Clients must follow the redirect Two resources to authorize and expire
Signed storage URL	Offloads large downloads Expiry built into the URL API stays small	The URL is a bearer secret Cannot be revoked early Authorization moves out of the API

Deciding signal: Result size and lifetime against the status call: measure bytes per poll of a completed operation, how long clients need the result and whether anyone other than the requester fetches it. Large results or different lifetimes need a separate result resource; very large files with simple access rules can go straight to storage

Smallest safe move

Redirect to a result resource that expires

- **Scope:** Contract, clients, data: finished operations and one result route
- **Change:** A completed operation answers 303 with Location /statement-exports/EX1; the file is private and a clean-up job deletes it after 7 days; later requests get 410 Gone with a link to request again
- **Safety net:** The result route checks the requester; 410 tells expired from unknown; signed URLs are not used for personal data without a short expiry
- **Escape route:** A flag returns the file inside the operation for small exports
- **Deferred:** Signed storage URL when results exceed what the API should stream; longer retention where customers need it

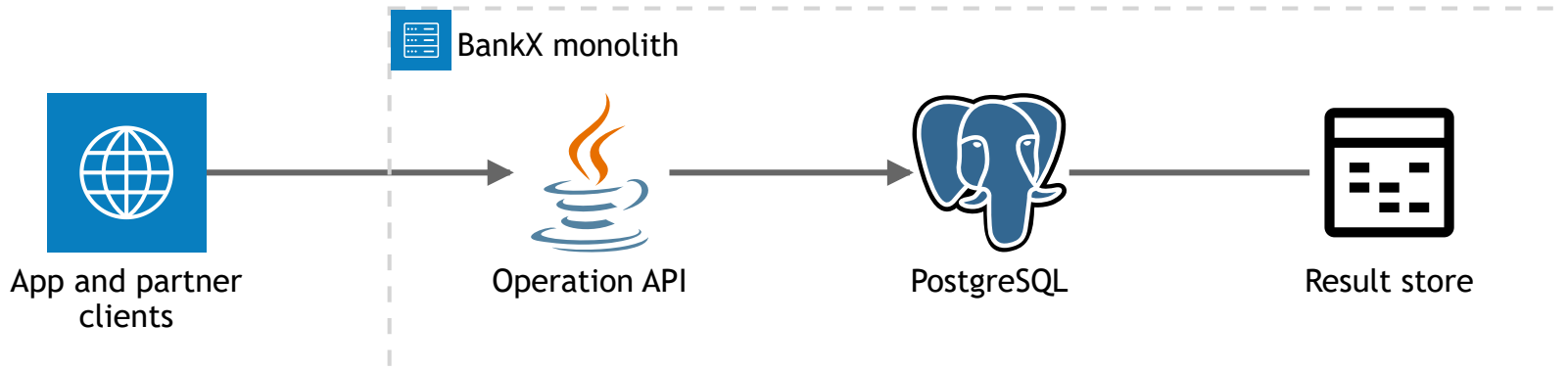
Verification: read before and after expiry, and as another user

Check	Pass condition
Poll after completion	303 with Location; status body of 1 KB or less
Follow the redirect as the requester	200 with the file and private cache headers
Read as another customer	403 or 404 with no content
Read after 7 days, the retention period	410 Gone with a link to request again
Clean-up job run	No stored result older than 7 days remains
Fitness function	A scheduled job asserts that no stored result is older than the retention period and a contract test asserts 410 for expired results; storage size is tracked

Rollback: Routing rollback: a flag returns the file inside the operation for small exports. State rollback: deleted results cannot return; a customer who needs one requests a new export, and the operation record stays to show what was asked.

Stop: When finished exports are fetched through the result URL and expire on schedule, no download service or signed-link storage layer is added

Solution architecture



Same clients, API and database; the finished operation now points to a separate result in a result store, and the result expires on schedule.

Four requests hand over the export, and a later one meets the expiry

```
POST /statement-exports      →  202 Accepted, Location /operations/OP1
GET  /operations/OP1         →  200, status processing
GET  /operations/OP1         →  303 See Other, Location /statement-exports/EX1
GET  /statement-exports/EX1  →  200, the export
GET  /statement-exports/EX1  →  410 Gone, after 7 days
```

The operation keeps a small status; the file sits under its own URL, cache rules and lifetime.

Answer	What the client may conclude
`303` with `Location`	The work is finished and the result is another resource
`200` with the export	The result is handed over to the requester
`410 Gone`	The export existed and expired; request a new one

403, 404 and 410 answer three different questions about an export

Status	The question it answers	What the client does
`403`	May this caller read it? No, not the requester	Does not retry unchanged; the caller lacks authorization
`404`	Does it exist for this caller? No such export	Checks the URL, or requests a new export
`410`	Did it exist and expire? Yes, it is gone	Follows the link in the problem details to request it again

410 separates expired from unknown, which a 404 cannot. For another customer’s export, 403 or 404 both pass, provided no content is returned.

Glossary

Used in this step

Term	Definition
Result inside the operation	The finished operation carries the result in its own response
303 See Other to a result resource	The finished operation redirects the client to a separate result resource
Signed storage URL	Time-limited link that lets a client download straight from storage
303 See Other	Status code that directs the client to another resource for the result
Result resource	Resource that holds the outcome of a finished operation
410 Gone	Status code that says the resource existed and is permanently gone
Retention period	How long a result is kept before it is deleted
Clean-up job	Scheduled process that deletes results after their retention period
Bearer secret	A URL or token that grants access to whoever holds it
Requester check	Server rule that only the customer who asked may read the result

Related concepts

Term	Definition
Object Storage	Service that stores large files as objects addressed by key
Cache-Control	Response header that sets how a response may be cached
Operation resource	Resource that reports the progress and outcome of a long-running request
Problem details	Standard machine-readable error body, application/problem+json

API15 · Synchronous and asynchronous interaction

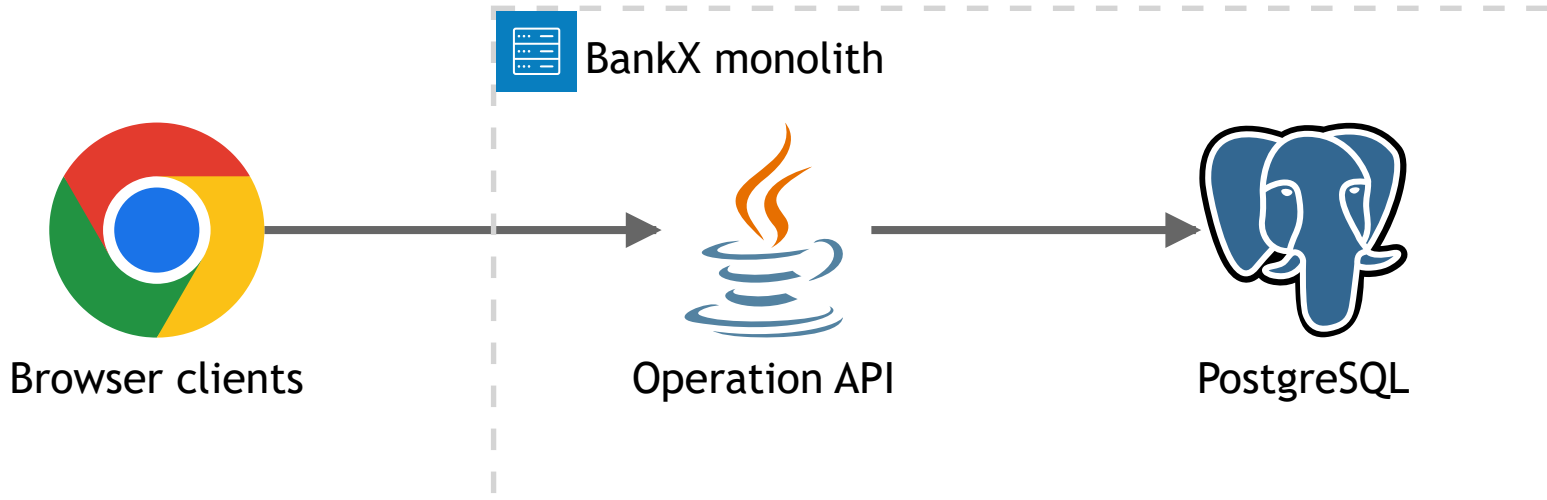
Learn the result without hammering the server

At month-end 50,000 clients wait for exports, and polling every 2 seconds costs 25,000 req/s, while the target is notice in ≤ 2 s without that load.

Browsers poll the operation every 2 seconds until it completes

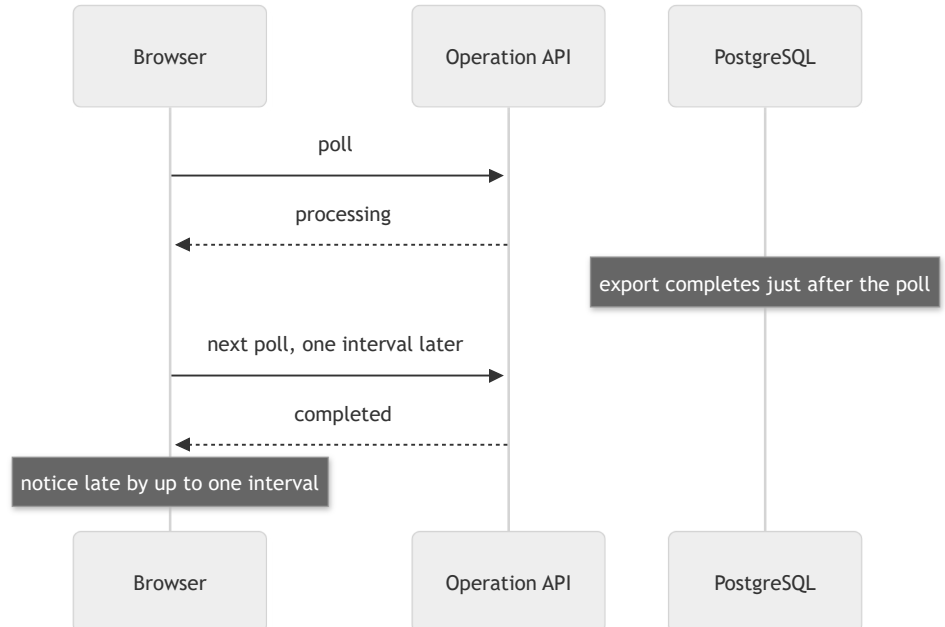
Component	What it is	What it does
Browser	BankX web app in the customer's browser	Polls GET /operations/OP1 every 2 seconds until it completes
Waiting customers	50,000 clients waiting for exports at month-end	Produce 25,000 req/s of polling, almost all answering processing
Operation API	GET /operations/OP1 in the Java monolith	Answers processing until the export completes, then redirects to the result
PostgreSQL	Database that holds the operations	Is read on every poll

Current architecture



Fast notice for 50,000 clients costs a flood of status requests

At month-end 50,000 customers wait on their export. Polling every 2 seconds keeps notice within the bound and costs 25,000 req/s, almost all answering processing. Polling every 30 seconds is cheap and leaves customers waiting.



25,000 req/s
status requests per second for 50,000 clients, almost all answering processing

50,000 clients
customers waiting for an export at month-end

≤ 2 s
from completion to client notice

Ways to tell a waiting client that the work is done

Solution	What it is	What it does
Polling with Retry-After	Clients poll the operation and the server tells each client when to ask next	Spreads and slows requests while keeping the pull model
SSE on the operation	A server-to-client event stream over HTTP, one stream per waiting client	Pushes the status change when it happens
Webhook callback	The client registers a URL and the server calls it when the operation completes	No client waits on the server

Pull costs requests and delay, push costs connections and a client that can receive

Solution	Gains	Costs
Polling with Retry-After	No new protocol Works through any proxy Easy to troubleshoot	Notice is late by up to the interval Requests that find nothing Clients must honor Retry-After
SSE on the operation	Push within the target Plain HTTP with automatic reconnect Operation stays the record	50,000 open connections to hold One-way only Resume needs event ids
Webhook callback	No open connection Instant notice Fits servers that expose an endpoint	Browsers cannot receive callbacks Delivery retries and security to build Client endpoint may be down

Deciding signal: Client type, client count, update frequency and the required delay: client count divided by the polling interval gives the request rate, and the required notice time bounds the interval. At 50,000 clients and a 2 s bound polling needs 25,000 req/s; if that exceeds capacity, browsers need push, and webhooks apply only to clients that can run an endpoint

Smallest safe move

Stream operation status, keep polling as fallback

- **Scope:** Contract and clients: operation status only; routes unchanged
- **Change:** GET /operations/OP1/events starts with the current status, streams changes and a final completed event with the result link; events carry ids; polling stays
- **Safety net:** The operation stays the record, events only announce; clients re-read on reconnect, with heartbeats and jitter; no webhooks for browsers
- **Escape route:** A flag turns the stream off; clients poll with Retry-After
- **Deferred:** Polling with Retry-After if the target is 30 seconds or more; webhook for partners with an endpoint; WebSocket if clients must send messages

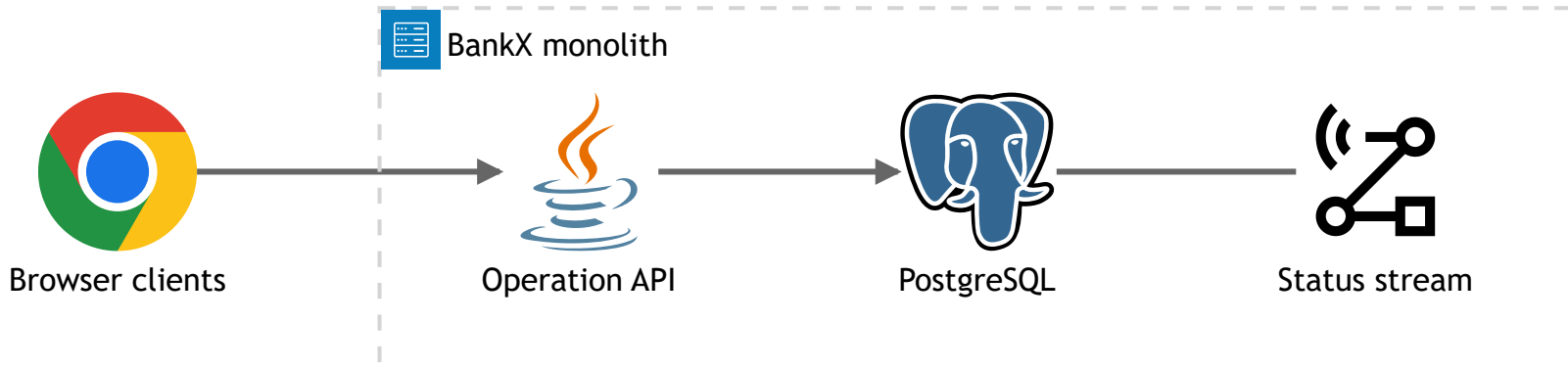
Verification: 50,000 clients held open, delay measured at completion

Check	Pass condition
50,000 streams open for the whole test	Held within memory and file-descriptor limits
Completion to client notice	p99 ≤ 2 s, maximum reported
Status requests per second from connected clients	Near zero
Server restart	All clients reconnect with jitter and resync from the operation within the target
Stream blocked by a proxy	Clients fall back to polling and still see the outcome
Fitness function	A load test with simulated clients fails when delivery delay exceeds 2 s or streams drop; an alert tracks open streams and the status request rate

Rollback: Routing rollback: a flag turns the stream off and clients poll again; the stream holds no state. State rollback: none; the operation resource was always the record of the outcome.

Stop: When delivery stays within 2 s at 50,000 clients, no WebSocket service, message broker or webhook infrastructure is added for browsers

Solution architecture



Same clients, API and database; a status stream now pushes completion to every waiting client, and the operation stays the record.

A stream announces the change and the operation stays the record

```
GET /operations/OP1/events
Accept: text/event-stream

id: 7
data: {"status":"processing"}

id: 8
event: completed
data: {"result":"/statement-exports/EX1"}
```

- **First event:** the current status, so a client that connects late starts from the truth.
- `id` : lets a client resume after a reconnect; the stream holds no state of its own.
- `completed` : carries the result link, which the client fetches with GET.
- **Polling stays:** `GET /operations/OP1` still returns the outcome to a client that was offline or blocked.

Client count divided by the polling interval gives the request rate

Clients	Polling interval	Status requests	What follows
50,000 clients	2 seconds	25,000 req/s	Meets the notice bound; almost every answer is processing
50,000 clients	30 seconds, illustrative	about 1,667 req/s	Cheap, and notice is late by up to 30 seconds
50,000 clients	none, one stream each	near zero	Push within the bound; 50,000 open streams to hold

The required notice time bounds the interval. When the resulting rate exceeds capacity, browsers need push, and a webhook applies only to clients that can run an endpoint.

Glossary

Used in this step

Term	Definition
Polling with Retry-After	Repeated status requests paced by the server's Retry-After header
SSE on the operation	A server-sent event stream that announces changes of one operation
Webhook callback	The server calls a client-registered URL when the operation completes
Polling	Repeated requests to determine whether data changed
SSE	Server-to-client event stream over HTTP
Retry-After	Header that tells the client how long to wait before the next try
Webhook	HTTP callback that one system sends to another when an event happens
Event stream	Open HTTP response that sends events as they happen
Heartbeat	Periodic message that keeps an idle connection open
Event id	Number that lets a client resume a stream after reconnecting
Jitter	Randomized delay around retries

Related concepts

Term	Definition
WebSocket	Persistent bidirectional client-server communication
Long Polling	Keep an HTTP request open until new information or a timeout
Operation resource	Resource that reports the progress and outcome of a long-running request
Graceful Degradation	Retain core capability when optional dependencies fail

API16 · Synchronous and asynchronous interaction

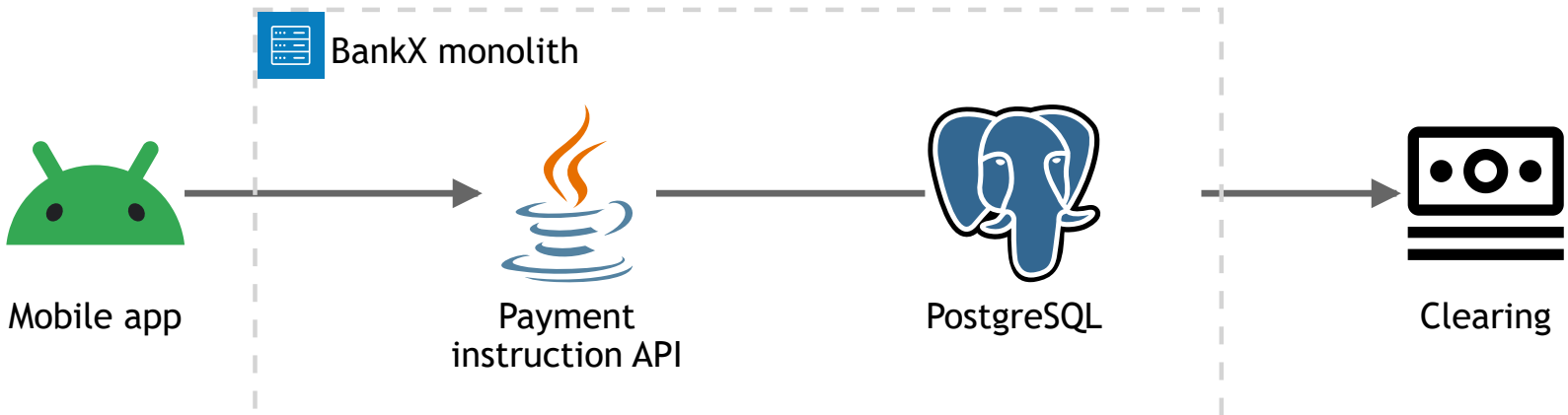
Represent a transfer that completes days later

Clearing for other-bank transfers can finish up to 2 business days after the HTTP exchange or fail, while the app reads accepted as Sent: 30% of support contacts ask where a payment is, against a target of 0 payments shown as sent before settlement.

One status field feeds the word Sent in the app

Component	What it is	What it does
Mobile app	BankX customer app on the customer’s phone	Shows the instruction status and displays executed as Sent
Payment instruction API	The instruction routes in the Java monolith	Exposes one status field: awaiting-approval, approved or executed
PostgreSQL	Database that holds accounts, instructions and approvals	Stores the instruction with its single status
Clearing	Exchange and settlement of transfers to other banks on clearing schedules	Settles up to 2 business days later, or the receiving bank rejects

Current architecture



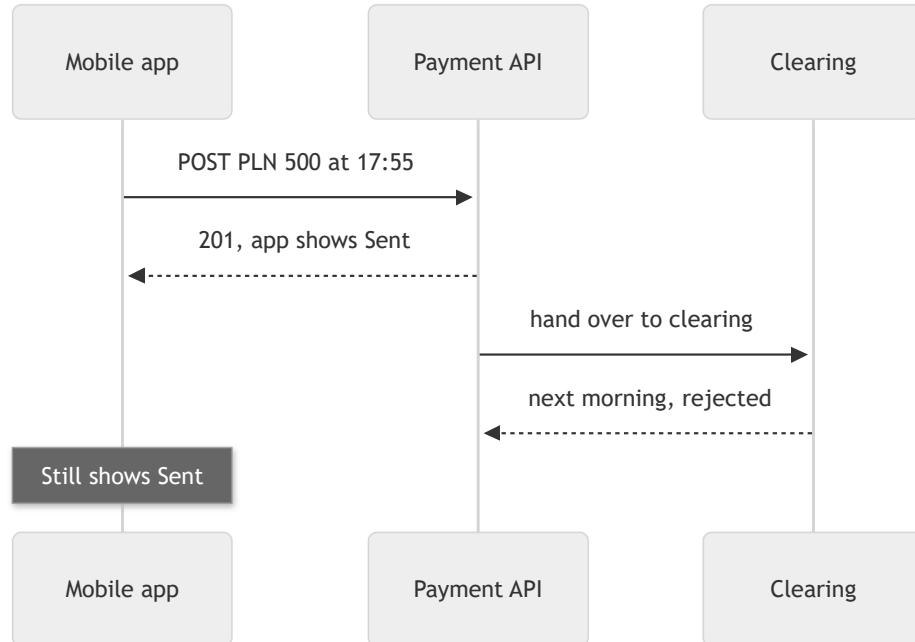
The app says Sent and clearing rejects it

A customer sends PLN 500 to another bank at 17:55 and the app shows Sent. Clearing runs next morning and the receiving bank rejects it, yet the customer believes the rent was paid.

30%
of support contacts ask where a payment is

up to 2 business days
between acceptance and settlement, other-bank transfers

0
payments shown as sent before settlement



Ways to show how far a payment has come

Solution	What it is	What it does
Richer status on the instruction	More values in the instruction's status: accepted, submitted, settled, failed	One field shows how far the payment has come
Separate transfer resource	Execution is a transfer resource with its own status, created when the instruction is approved and linked from it	Instruction and transfer move independently and each states what it knows
Event history resource	A read-only list of dated events on the instruction, such as accepted, sent to clearing, settled, rejected	Shows the path and the dates without a second state machine

Where the status lives decides who is allowed to change it

Solution	Gains	Costs
Richer status on the instruction	No new resource Simple for clients Quick to ship	Two lifecycles tangled in one field Failure after approval has no home One field with several owners
Separate transfer resource	Each lifecycle has one owner Failed transfer on an approved instruction Ledger postings hang off the transfer	New resource and table Clients follow a link Two statuses to explain
Event history resource	Shows dates and reasons Append-only and auditable Existing status untouched	Clients derive state from events Order and duplicates to handle Still no owner of the current state

Deciding signal: Whether instruction and execution can disagree and who changes each: count the combinations that must be shown (approved but failed, settled later) and the parties that write status (customer side, clearing, ledger). If a different party owns the outcome, a separate resource is needed; if one party owns everything, a richer status is enough, and a history helps when dates and reasons matter

Smallest safe move

Execution gets its own transfer resource

- **Scope:** Contract, behavior, data: external transfers only; internal transfers keep one status
- **Change:** Create /transfers/T1 on full approval (accepted, submitted-to-clearing, settled or failed with a reason), linked from the instruction as transfer; the instruction stops at approved; clearing updates it
- **Safety net:** Accepted, submitted and settled stay distinct in contract and UI; settled comes only from clearing evidence, never a client; a 201 means only accepted
- **Escape route:** A flag hides the transfer link and the app shows the old status
- **Deferred:** Event history resource when customers need dates and reasons per step; richer status for internal transfers that settle at once

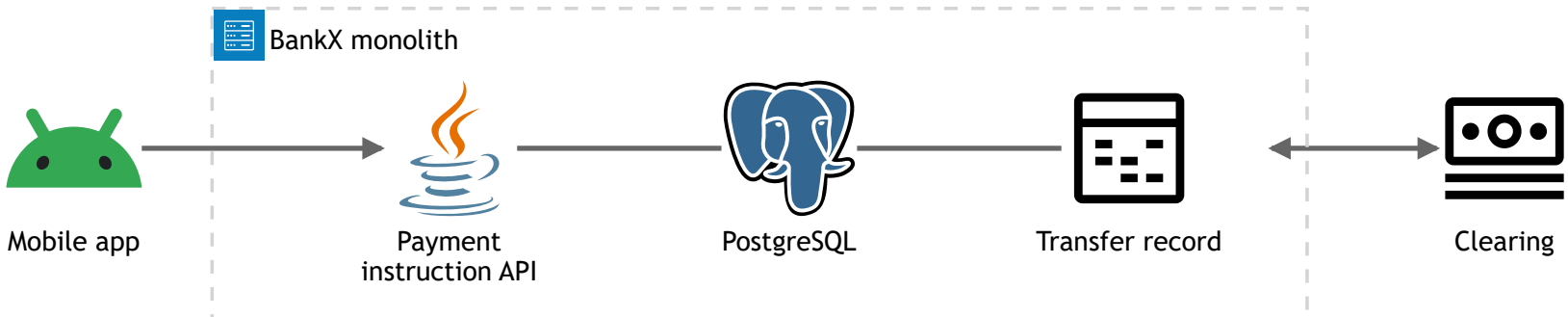
Verification: acceptance, settlement, rejection and a cut-off delay

Check	Pass condition
Instruction approved	Transfer created as accepted; instruction shows approved, not sent
Clearing confirms settlement	Transfer settled with a settlement date and postings linked
Clearing rejects after approval	Transfer failed with a reason; instruction stays approved; customer sees the failure
Transfer read after the cut-off time	Shows submitted-to-clearing with the expected date
Query for settled transfers without clearing evidence	0 rows
Fitness function	A nightly job compares every transfer's status with the clearing file and fails on any settled transfer that has no clearing evidence; UI text tests forbid the word sent for any status before settled

Rollback: Routing rollback: the flag hides the transfer link and clients show the old status. State rollback: transfers already created and settled keep their records; an API change rolls back no money; a failed transfer is repaired by the clearing process, never by editing a status.

Stop: When no payment is shown as sent before settlement and failed transfers are visible, no event-sourced transfer model or workflow engine is added

Solution architecture



Same client, API, database and clearing; clearing results now update a separate transfer record that the API exposes as its own resource, and the instruction stops at approved.

Only settled lets the app say Sent

Word	Set by	What the client may say
accepted	The server, when the instruction is fully approved	Accepted for processing, not sent
submitted-to-clearing	The server, on hand-over to clearing	Submitted with the expected date, not sent
settled	Clearing evidence only, never a client	Settled on its settlement date; only now Sent
failed	The clearing result, with the reason reported	Failed, with the reason and any account adjustment

A status is never edited by hand: a failed transfer is repaired by the clearing process, and the instruction stays approved.

A transfer links to its instruction and its postings

```
GET /transfers/T1  →  200 OK
Content-Type: application/vnd.siren+json

{ "class": ["transfer"],
  "properties": { "id": "T1", "status": "settled", "settledOn": "2026-10-02" },
  "links": [
    { "rel": ["payment-instruction"], "href": "/payment-instructions/PI1" },
    { "rel": ["ledger-postings"], "href": "/transfers/T1/ledger-postings" }
  ] }
```

The instruction links to this transfer by the relation `transfer` and stays at approved; the transfer carries the clearing outcome.

Part	What the client may conclude
`status` and `settledOn`	Finally completed on that date; only now may the app say Sent
Link to the instruction	The approved instruction that produced this transfer
Link to the postings	The accounting records hang off the transfer, not the instruction

Glossary

Used in this step

Term	Definition
Richer status on the instruction	More status values on one field that several parties would write
Separate transfer resource	Execution modelled as its own resource with its own status
Event history resource	Read-only list of dated events that happened to a resource
Transfer	Execution of an authorized instruction between accounts, within BankX or to an external bank
Clearing	Exchange and settlement of payments between banks on a schedule
Settlement	Final completion of a transfer between banks
Cut-off time	Latest time a payment is accepted for the next clearing run
Lifecycle	The states a resource passes through and who may change them
Ledger posting	Record of a financial movement with account, amount, currency, booking date and reference
Accepted versus settled	Accepted means stored for processing; settled means finally completed

Related concepts

Term	Definition
Data Reconciliation	Compare systems using keys, checksums and domain invariants to find divergence
Saga Pattern	Coordinates distributed business transactions using local transactions and compensations.
Domain Event	Statement that an important business fact occurred
Operation resource	Resource that reports the progress and outcome of a long-running request

API17 · Synchronous and asynchronous interaction

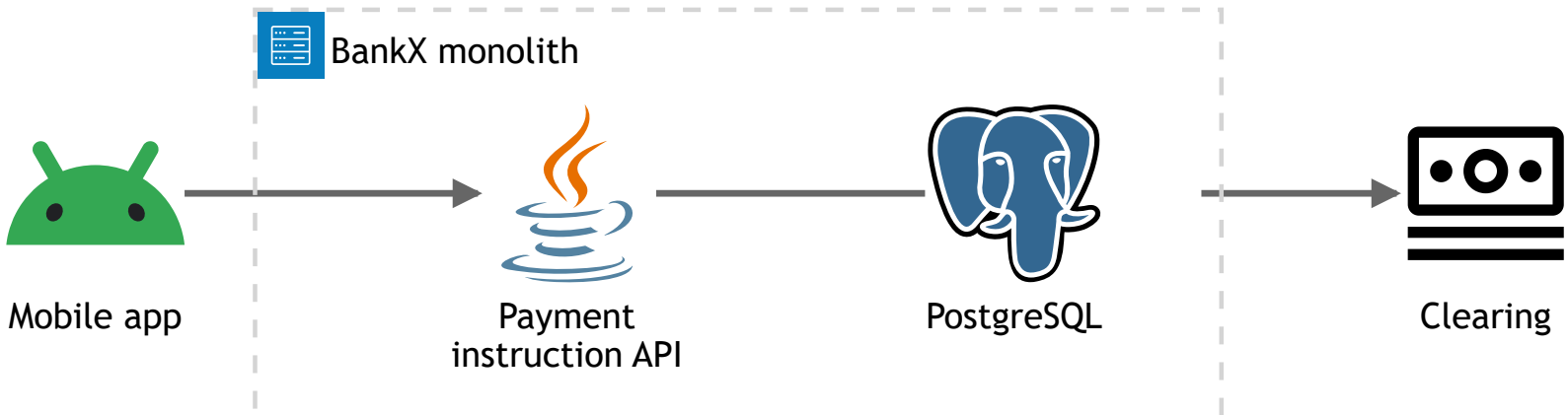
Cancel while clearing is running

A cancel that arrives after the instruction was submitted to clearing is answered 204 while the money still leaves: 4% of cancel requests arrive that late, against a target of 0 instructions both cancelled and executed.

A cancel deletes the instruction and answers 204

Component	What it is	What it does
Mobile app	BankX customer app on the customer’s phone	Sends DELETE /payment-instructions/PI1 and shows Cancelled after the 204
Payment instruction API	The payment instruction routes in the Java monolith	Removes the instruction on DELETE and answers 204 in every state
PostgreSQL	Database that holds instructions and their processing stage	Stores the stage that the cancel and the clearing submission both change
Clearing	The system that executes and settles transfers between banks	May already hold the instruction and settles it next day

Current architecture



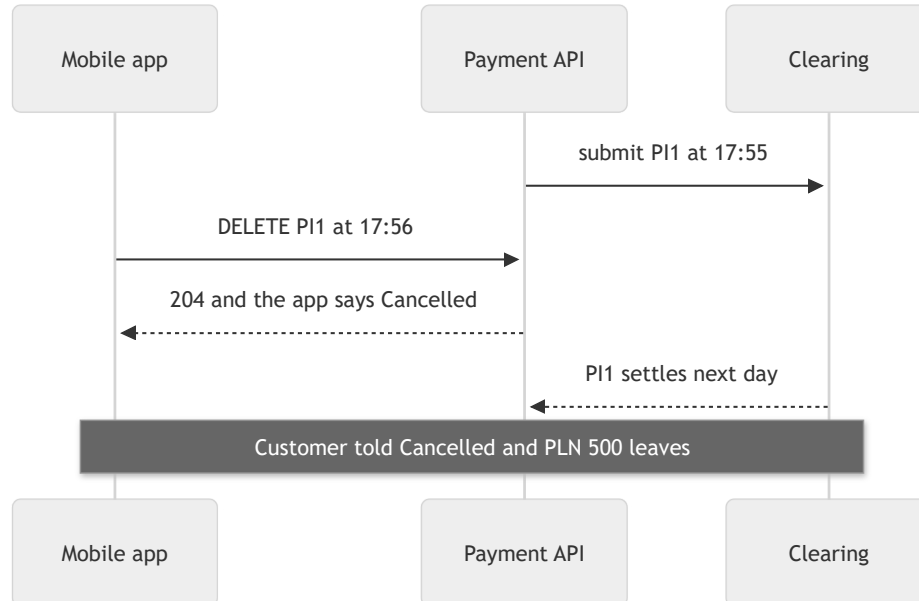
A cancel races with clearing

A customer cancels a PLN 500 payment at 17:56 and the app says Cancelled. Clearing received it at 17:55 and settles next day, so the money leaves.

4%
of cancels arrive after submission to clearing

2 writers
cancel and clearing submit change one instruction

0
instructions both cancelled and executed



Ways to answer a request to stop a payment

Solution	What it is	What it does
DELETE with immediate cancel	DELETE removes the instruction and answers 204	Cancels in one call
Cancellation request resource	POST /payment-instructions/PI1/cancellation-requests creates a request that ends accepted or rejected-too-late	The server decides against the current stage and the customer reads the outcome from the request
Compensating reversal	After settlement a reversal or recall request asks the other bank to return the money	Handles a cancellation that arrives after the transfer left

An honest outcome needs its own record and its own wording

Solution	Gains	Costs
DELETE with immediate cancel	One call, familiar verb Simple for clients No new resource	No outcome other than done Races with the clearing submit 204 promises what clearing may refuse
Cancellation request resource	The outcome has a place Race decided in one atomic step Fits stage-dependent rules	New resource and lifecycle Client reads the outcome Customer text for each outcome
Compensating reversal	Covers settled payments Real banking process Original record kept	Other bank may refuse Takes days Needs its own resource and rules

Deciding signal: At which stage the instruction can still be stopped and how often a request arrives past that point: measure the share of cancel requests per stage (before approval, queued, submitted, settled) and the time between submission and settlement. Before submission an atomic state change suffices; once submitted only a request with an uncertain outcome or a reversal is honest

Smallest safe move

Cancellation as a request with an outcome

- **Scope:** Contract, behavior, clients: cancelling payment instructions; DELETE answers 405 once submitted
- **Change:** POST /payment-instructions/PI1/cancellation-requests with If-Match creates CR1; one transaction cancels only before submission, else rejected-too-late; the customer reads the outcome
- **Safety net:** One atomic compare-and-set decides cancel against submit; cancelled is never reported early; If-Match refuses a stale cancel; the action is advertised only where it can succeed, yet POST checks again
- **Escape route:** A flag restores DELETE for instructions not yet approved; the request route stays
- **Deferred:** Compensating reversal when customers must recall settled payments; a tracked cancellation for other resources only when their cancel can fail

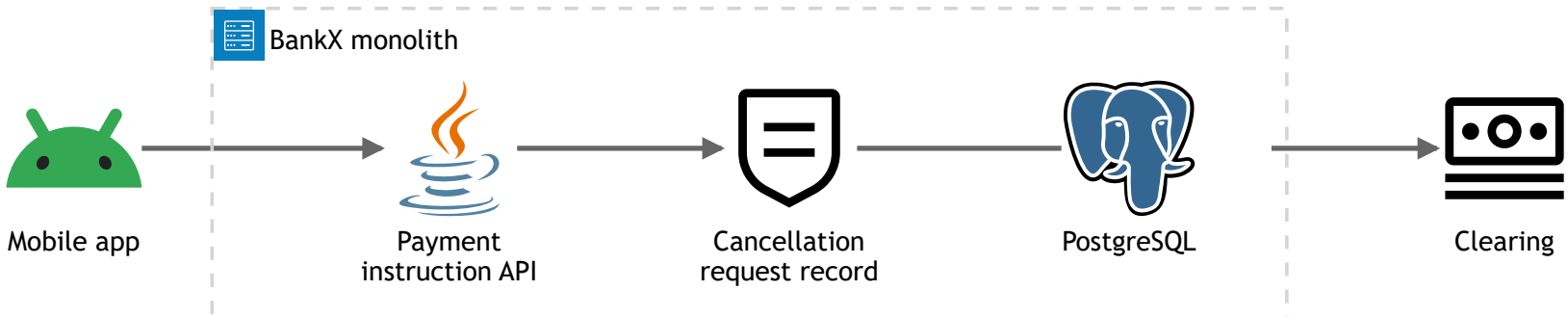
Verification: every stage raced, cancel against submission

Check	Pass condition
Cancel before submission	Request accepted; instruction cancelled; never submitted
Cancel after submission	Request rejected-too-late; the instruction proceeds; the customer is told
1,000 cancel and submit pairs fired concurrently	Exactly one wins in every pair; no cancelled instruction has a clearing record
Cancel sent with a stale ETag	412 and no change
Nightly query for cancelled instructions with a clearing submission	0 rows
Fitness function	A concurrency test races cancel against submit and fails if both succeed; the nightly query alerts on any cancelled instruction that has a clearing record

Rollback: Routing rollback: the flag restores DELETE for unsubmitted instructions and the race returns for them. State rollback: cancellations already decided stay recorded; an instruction wrongly cancelled is recreated by the customer; an executed transfer cannot be cancelled and needs a reversal.

Stop: When the race test yields exactly one winner and customers always see the true outcome, no reversal service or workflow engine is added

Solution architecture



Same client, API, database and clearing; a cancellation is now a record that the database decides in one transaction against the instruction's stage.

A cancellation that is recorded and then read

```
POST /payment-instructions/PI1/cancellation-requests HTTP/1.1
If-Match: "v3"

{ "reason": "Customer request" }

HTTP/1.1 201 Created
Location: /cancellation-requests/CR1

GET /cancellation-requests/CR1  →  200 { "status": "rejected-too-late" }
```

The `201` says the request exists; the verdict is read from `CR1`.

Message	What the client may conclude
`GET` shows `accepted`	The instruction is cancelled and was never submitted
`GET` shows `rejected-too-late`	The instruction proceeds; the customer is told
Stale `If-Match`	`412`; nothing changes

The stage decides what a cancellation can honestly report

Stage	Request ends	Instruction afterwards	Customer is told
Before approval	accepted	Cancelled	The payment is cancelled
Queued	accepted	Cancelled, never submitted	The payment is cancelled
Submitted	rejected-too-late	Proceeds to clearing	The payment was already sent and cannot be stopped
Settled	rejected-too-late	Executed	The money has left; recovering it needs a reversal

The stage is read in the same transaction that changes the state, so Cancelled is never reported before it is true.

Glossary

Used in this step

Term	Definition
DELETE with immediate cancel	DELETE on the instruction treated as a finished cancellation
Cancellation request resource	A request to cancel whose outcome is its own resource
Compensating reversal	A later request that undoes a completed transfer
Cancellation request	A customer's request to stop an instruction, whose outcome depends on its stage
Race condition	Outcome depends on which of two concurrent actions happens first
Compare-and-set	Change a value only if it still equals the value that was read
Processing stage	Where an instruction is in its lifecycle, such as approved or submitted
Reversal	Request to the other bank to return a settled payment
If-Match	Request header that makes a write conditional on the current tag
405 Method Not Allowed	Status code for a method the resource does not support
204 No Content	Success status that carries no body

Related concepts

Term	Definition
Saga Pattern	Coordinates distributed business transactions using local transactions and compensations.
Optimistic concurrency	Detect conflicts at write time instead of locking in advance
Reservation Pattern	Temporarily claim business resource pending workflow completion
Domain Event	Statement that an important business fact occurred

API18 · Synchronous and asynchronous interaction

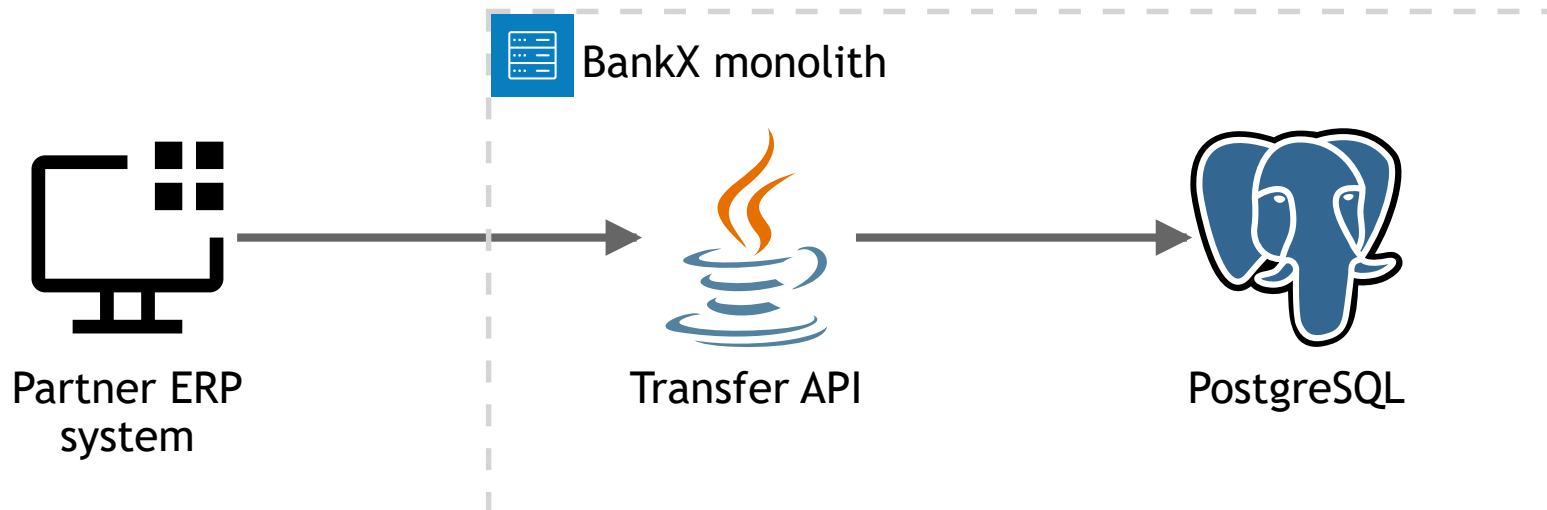
Deliver status webhooks to partner systems

A partner endpoint that takes 20 s or fails makes BankX deliver one event in up to 5 deliveries, out of order, and any caller can forge one: the target is 1 business effect per event.

A partner polls for transfer status and asks to be called back

Component	What it is	What it does
Partner ERP system	The business customer’s system that releases goods when a transfer is settled	Polls GET /transfers every minute and asks BankX to call it back instead
Transfer API	The GET /transfers routes in the Java monolith	Serves transfer status to pollers and has no way to call a partner
PostgreSQL	Database that holds transfers and their status changes	Records every status change and keeps no delivery record
Partner endpoint	The HTTPS address the partner would give BankX for callbacks	Can take 20 s or be down, which turns one event into 5 deliveries

Current architecture



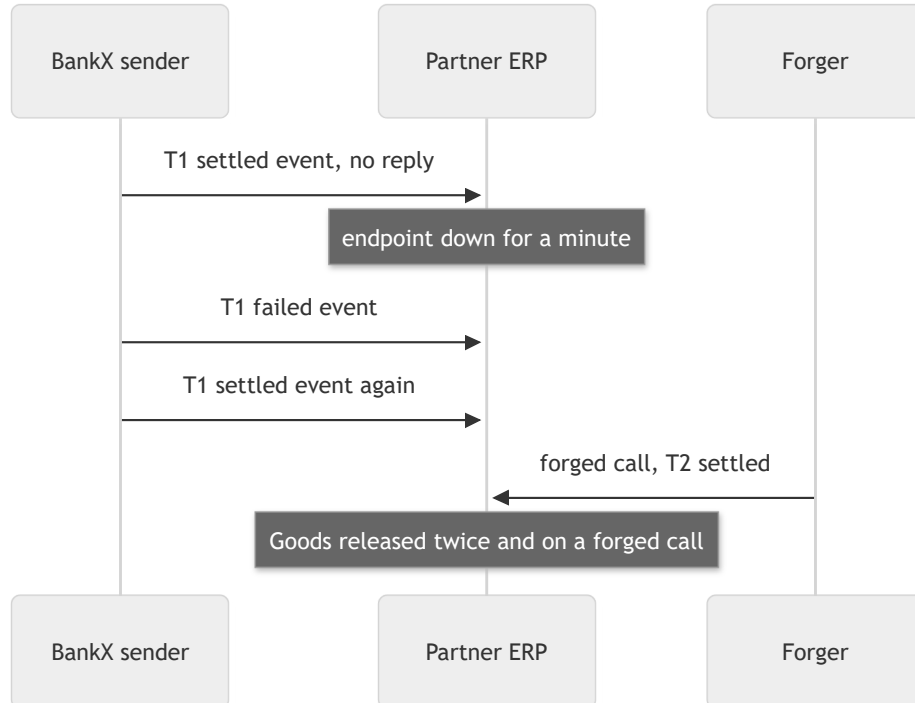
Callbacks repeat, reorder and can be forged

A partner ERP releases goods on transfer settled. After a one-minute outage BankX retries: settled arrives twice, after a later failed event, and a forged call marks another transfer settled.

5 deliveries
of one event when the endpoint fails

20 s
a slow partner endpoint holds a worker

1
business effect per authentic, latest event



Ways to tell a partner that a transfer changed

Solution	What it is	What it does
Signed full event with id and sequence	The callback carries the transfer state, an event id, a sequence number and a signature	The partner verifies the signature, ignores duplicates and ignores older sequence numbers
Thin notification + GET	The callback only names the changed resource and links to it; the partner fetches the current state	The partner always reads the latest authoritative state
Fire once, no retry	One delivery attempt per event, with no retries and no signature	The sender never repeats an event

Each option moves ordering and trust to a different party

Solution	Gains	Costs
Signed full event with id and sequence	No call back needed Partner can deduplicate by id Signature proves origin	Partner must implement ordering State in the event may be stale Payload fields become contract
Thin notification + GET	Order and staleness solved by fetching Small payload, no state leaked Signature still proves origin	One extra request per event Partner needs API credentials Load spike when many events arrive
Fire once, no retry	No duplicates Simplest sender No queue of pending events	Any outage loses events Anyone can forge a call Partner must poll to recover

Deciding signal: What a duplicate or a stale event costs the partner and whether it can read the resource: measure deliveries per event, the out-of-order rate and failed deliveries during partner outages. If acting on a stale status is costly and partners can call the API, a thin notification removes the ordering problem; if partners cannot call back or need the state at that moment, a full event with a sequence number is needed

Smallest safe move

Signed hint, partner reads the truth

- **Scope:** Contract and clients: transfer status changes for registered partners
- **Change:** Partners register an endpoint and secret; each change sends a signed notification (event id, type, transfer link, timestamp) with at-least-once retries and backoff for 24 hours; the partner fetches the transfer
- **Safety net:** Signature and timestamp stop forgery and replay; event id removes duplicates; resource is the truth; slow endpoints block no worker; fire once is not used
- **Escape route:** A flag pauses a partner, who polls again; pending events replay later
- **Deferred:** Full event with sequence number for partners that cannot fetch; event streaming platform when many partners need replay

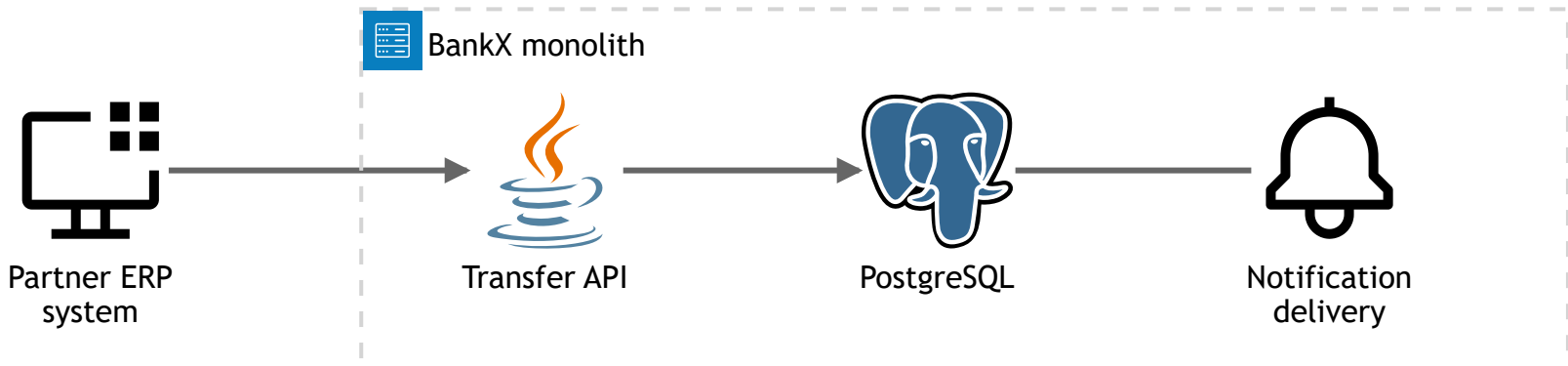
Verification: endpoint down, slow, repeated, reordered and forged

Check	Pass condition
Partner endpoint down for 10 minutes	Every event is delivered after recovery; none lost
Same event delivered 5 times	One business effect at the partner (event id deduplication)
Settled and failed events arrive in reverse order	Partner state equals the transfer resource after it fetches
Forged call, or a replay older than the allowed clock skew	Signature or timestamp check fails and nothing changes
Slow partner endpoint (20 s)	Sender workers stay free; deliveries queue instead of blocking
Fitness function	A contract test suite runs these cases against a partner sandbox endpoint; metrics track delivery lag, retries per event and the failure rate per partner

Rollback: Routing rollback: the flag pauses callbacks for a partner and the partner polls as before. State rollback: events already delivered cannot be recalled; a wrong event is corrected by a new event and the partner reads the transfer resource to resynchronize.

Stop: When every event is delivered at least once, duplicates and forgeries change nothing and partners read the truth from the resource, no event streaming platform or partner SDK is added

Solution architecture



Same partner system, API and database; a delivery component now sends a signed notification for each transfer status change, and the partner reads the transfer from the API.

A notification that names the change and nothing more

```
POST /bankx-events HTTP/1.1
BankX-Timestamp: 2026-11-03T10:15:00Z
BankX-Signature: sha256=9f2c41d8
Content-Type: application/json

{ "id": "E42", "type": "transfer.settled", "transfer": "/transfers/T1" }

HTTP/1.1 200 OK
```

The body carries an event id, a type and a link; the state itself is read from `/transfers/T1`.

Partner reply	What the sender does
`200`	Records the delivery and stops retrying
An error status or no reply	Retries with backoff and jitter; the same event id is sent again
A slow endpoint	Holds no worker; the delivery waits and is retried

A receiver that survives redelivery follows five rules

Rule	What the receiver does	Check that proves it
Verify	Checks the signature and rejects an old timestamp	Forged call or old replay changes nothing
Deduplicate	Ignores an event id it has already handled	Same event delivered 5 times gives one business effect
Fetch	Reads the transfer for the current state	Reversed settled and failed events end with the transfer’s state
Reply fast	Answers before slow work starts	Sender workers stay free when the endpoint takes 20 s
Tolerate redelivery	Treats a repeat or a late event as normal	Endpoint down for 10 minutes loses no event

Glossary

Used in this step

Term	Definition
Signed full event with id and sequence	Callback with full state, an event id, a sequence number and a signature
Thin notification + GET	Callback that names the change and links to the resource the partner then reads
Fire once, no retry	One delivery attempt per event, with no retries and no signature
Webhook	HTTP callback that one system sends to another when an event happens
At-Least-Once Delivery	Delivery semantics allowing duplicate deliveries in exchange for retries
At-Most-Once Delivery	Delivery semantics not retrying possibly lost messages
HMAC signature	Keyed hash that proves a message came from the holder of a shared secret
Replay attack	Re-sending a captured valid message
Event id	Number that lets a receiver recognize an event it has already handled
Sequence number	Counter that orders the events of one resource
Retry with Backoff	Repeats failed operations with increasing delays.

Related concepts

Term	Definition
Idempotency Key	Stable identifier recognizing multiple attempts at the same logical command
Exactly-Once Effect	Application-visible effect occurs once despite repeated attempts under defined guarantees
Dead Letter Queue (DLQ)	Stores messages that could not be processed successfully.
Publish/Subscribe (Pub/Sub)	Distributes messages to multiple interested subscribers.

API19 · Synchronous and asynchronous interaction

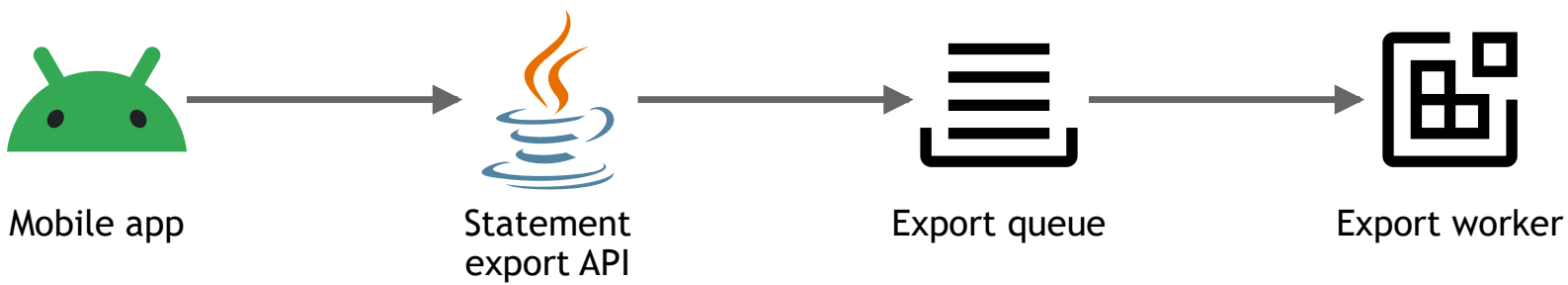
Slow callers down instead of failing

The API accepts 1000 msgs/s at month-end but only 500 msgs/s are processed, so the backlog grows 500/s: the target is an oldest queued request of ≤ 60 s.

Every export request is accepted with 202 into one queue

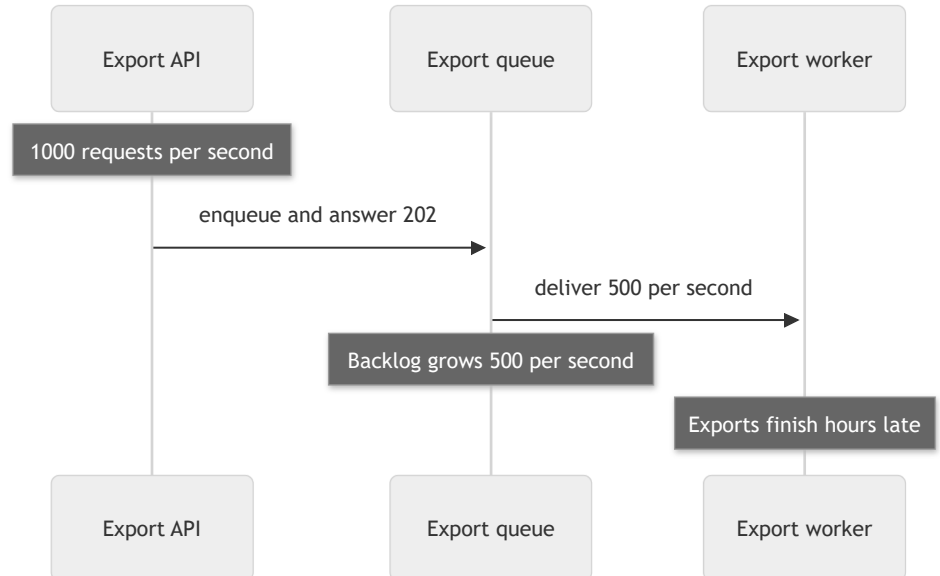
Component	What it is	What it does
Mobile app	BankX customer app on the customer’s phone	Posts statement export requests, including at month-end
Statement export API	POST /statement-exports in the Java monolith	Answers 202 to every request and puts it in the queue
Export queue	One queue between the API and the worker	Holds accepted requests; grows when 1000 msgs/s arrive and 500 msgs/s leave
Export worker	One consumer that reads the queue	Processes 500 msgs/s, however many requests wait

Current architecture



Accepted exports pile up faster than they clear

At month-end the API accepts export requests with 202 faster than the worker can process them. The queue grows by 500 messages every second, so an export accepted now completes hours later or never.



+500/s
growth of the backlog each second

1000 vs 500
messages per second that arrive against messages per second that are processed

≤ 60 s
age of the oldest queued request

Ways to balance arrivals against processing

Solution	What it is	What it does
More workers	Add consumers so processing reaches the arrival rate	Raises throughput until arrival exceeds capacity again
429 + Retry-After backpressure	Bound the queue and answer 429 with Retry-After when it is full	Slows producers instead of letting the backlog grow
Unbounded queue	Keep accepting everything with 202 and let the queue grow	Never refuses a request

Each answer moves the cost to capacity, callers or the backlog

Solution	Gains	Costs
More workers	Throughput rises No API change Quick to apply	Downstream limits still apply Ordering may weaken Next spike overruns again
429 + Retry-After backpressure	Backlog stays bounded Callers learn when to return Accepted work finishes on time	Some requests are rejected Clients must honor Retry-After The limit needs tuning
Unbounded queue	No rejection Simplest API Absorbs short bursts	Backlog grows 500 per second Accepted work may never finish Memory and storage run out

Deciding signal: Arrival rate against processing rate and how long a request may wait: compute the backlog growth (1000 minus 500 is 500 per second), the queue depth at which the wait passes the limit and the downstream capacity. If extra workers lift capacity above arrival and the database keeps up, add them; if arrival stays above capacity during spikes, bound the queue and tell callers when to retry

Smallest safe move

Bound the queue and answer 429 when full

- **Scope:** Contract, behavior and clients: export intake route only
- **Change:** Limit queue depth to 30,000 requests (500 per second times 60 s); when full, POST /statement-exports answers 429 with Retry-After and problem details; clients back off with jitter
- **Safety net:** Rejection precedes acceptance, so nothing is dropped; per-customer fair share protects other callers; Retry-After is a promise the server keeps; unbounded queue is not used
- **Escape route:** A flag raises the limit; removing it means unbounded acceptance
- **Deferred:** More workers when downstream has headroom and rejections exceed the agreed rate; batch processing or partitioning when ordering or hot keys limit throughput

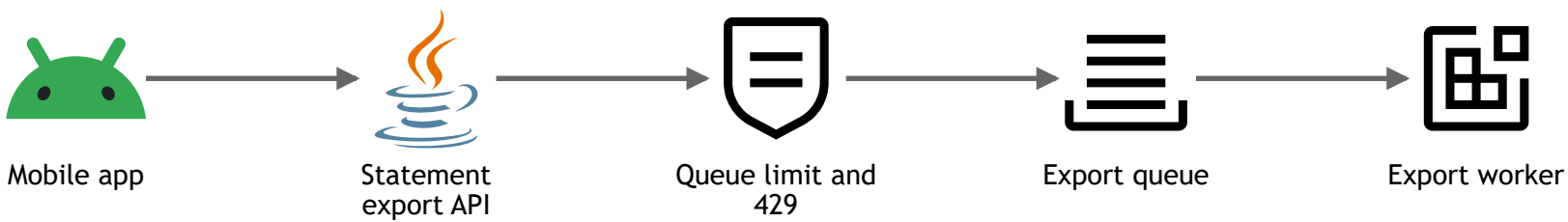
Verification: the month-end spike replayed against the limit

Check	Pass condition
1000 requests per second for 10 minutes	Queue depth never exceeds the limit; oldest request age stays ≤ 60 s
Rejected requests	429 with Retry-After; none was accepted and then lost
Clients honor Retry-After with jitter	Arrival settles at or below the processing rate
Worker crash with a full queue	No accepted request lost; processing resumes
One customer sends 90% of the load	Other customers are still accepted
Fitness function	A load test replays the spike and fails if depth exceeds the limit, an accepted request is lost or the oldest age passes 60 s; an alert tracks queue depth, age and the rejection rate

Rollback: Routing rollback: the flag raises or removes the limit and the unbounded growth returns. State rollback: none for rejected requests, which were never accepted; accepted ones stay in the queue and are processed; clients that gave up simply send again.

Stop: When the spike test keeps the backlog bounded and every accepted request completes in time, no additional queue platform, batch layer or partitioning is added

Solution architecture



Same client, API, queue and worker; the new queue limit applies 429 + Retry-After backpressure, so a full queue refuses a request instead of accepting one more.

A full queue answers 429 and says when to come back

```
POST /statement-exports HTTP/1.1
Idempotency-Key: K1

HTTP/1.1 429 Too Many Requests
Retry-After: 10
Content-Type: application/problem+json

{ "type": "/problems/export-queue-full",
  "title": "The export queue is full",
  "status": 429 }
```

The refusal comes before acceptance, so nothing is queued and no operation exists. `Retry-After` counts seconds.

Part	What it says	What the client does
`429`	The request was refused, not accepted	Expects no export and no operation
`Retry-After`	When the server expects room again	Waits that long, adds jitter, sends again
Problem details body	A machine-readable reason	Reads `type` and `title`; logs or shows them
Same `Idempotency-Key`	The refused attempt created nothing	At most one export results

The queue limit is the processing rate times the wait target

Derived arithmetic from the two measured rates and the wait target, not a measurement.

Quantity	Calculation	Result
Arrive	given	1000 msgs/s
Processed	given	500 msgs/s
Backlog growth	1000 minus 500	500 per second
Backlog after 10 minutes	500 per second times 600 s	300,000 requests
Queue limit	500 per second times 60 s	30,000 requests
Wait at the limit	30,000 requests at 500 per second	60 s

Without a limit the backlog keeps growing; with it, the oldest accepted request waits at most the target.

Glossary

Used in this step

Term	Definition
More workers	Add consumers so that processing capacity rises
429 + Retry-After backpressure	Refuse new work with 429 and a Retry-After when the queue is full
Unbounded queue	A queue with no depth limit that accepts every request
Backpressure	Limit or slow producers when consumers cannot keep up
Backlog	Accepted work that is waiting to be processed
429 Too Many Requests	Status code for a caller that sent too many requests
Retry-After	Header that tells the client how long to wait before the next try
Message Queue	Buffers messages for asynchronous processing by consumers.
Queue depth limit	Maximum number of waiting requests before new ones are refused
Fair share	Limit per caller so that one caller cannot use all capacity
Throughput	Work completed per second

Related concepts

Term	Definition
Rate Limiter	Controls the number of requests allowed within a time window.
Batch processing	Group items and process them together to raise efficiency
Partitioning	Split work or data by key so that parts are processed in parallel
Dead Letter Queue (DLQ)	Stores messages that could not be processed successfully.

Phase 4

Boundaries and distributed consistency

Steps in this phase

Step	Forcing constraint
API20 Map resources onto modules without a one-to-one assumption	One resource spans five modules and three teams change the same controller
API21 Serve account activity without live fan-out	Account activity calls 5 services live and p95 exceeds 500 ms from remote calls
API22 Choose the interface style for internal commands	A submit to clearing takes three REST round trips against a hop budget, for one internal consumer team
API23 Reconcile what HTTP cannot make atomic	Clearing results, transfers and ledger postings disagree after timeouts and retries
API24 Stop adding API machinery no consumer needs	A one-consumer partner API is proposed with four mechanisms while the consumer releases together with the API

API20 · Boundaries and distributed consistency

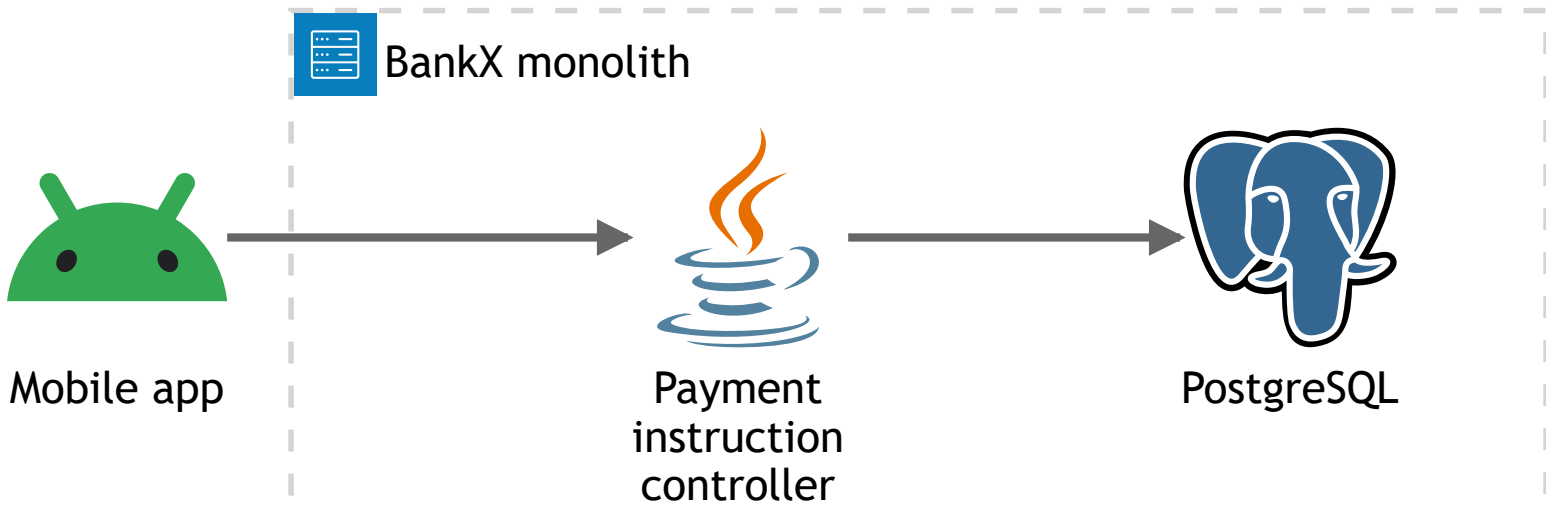
Map resources onto modules without a one-to-one assumption

One resource spans 5 modules and 3 teams change the same controller: 41% of its pull requests need conflict resolution, against a target of 0 URL changes for clients.

One controller serves every payment instruction URL

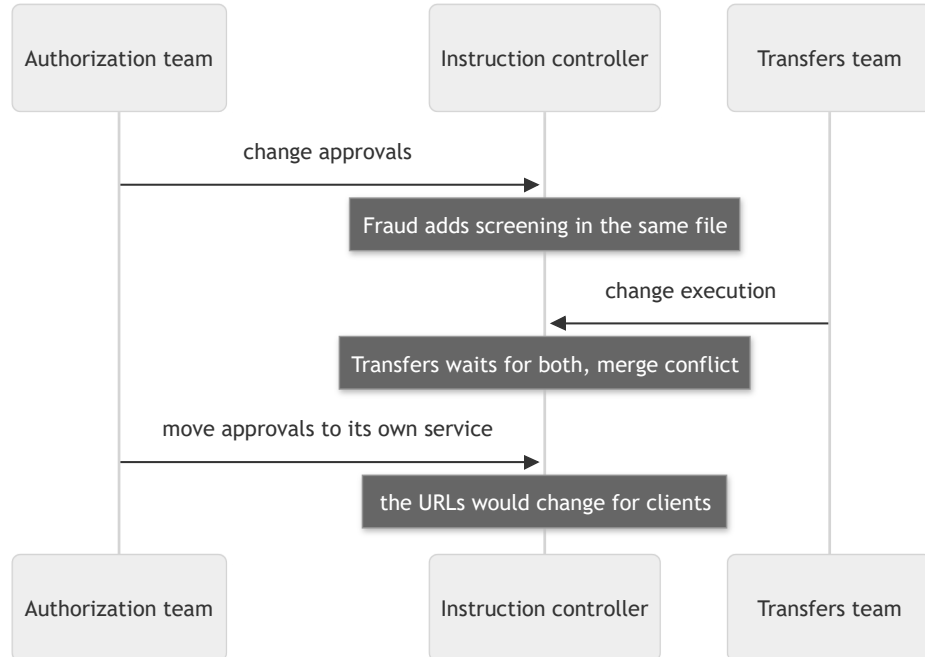
Component	What it is	What it does
Mobile app	BankX customer app on the customer’s phone	Calls /payment-instructions/PI1 and /payment-instructions/PI1/approvals
Payment instruction controller	PaymentInstructionController, one controller for every /payment-instructions URL	Serves the instruction, approvals, limits, fraud screening and links to transfers
Domain modules	Transfers, Accounts, Limits, Fraud, Compliance and Authorization in one deployable	The controller calls each one directly, so the URL prefix decides who owns the logic
Teams	3 teams: Authorization, Fraud and Transfers	Change approvals, screening and execution, all in the same controller
PostgreSQL	Database shared by the modules of the Java monolith	Holds the tables of every module

Current architecture



One URL, one controller, three colliding teams

Authorization, Fraud and Transfers all change PaymentInstructionController because the URL prefix put them there. One team's change blocks the others, and moving approvals out would change URLs.



41%
of controller pull requests need conflict resolution

5 modules
behind /payment-instructions/PI1

0
URL changes while teams change their modules

Ways to separate a resource from the modules behind it

Solution	What it is	What it does
One controller per URL prefix	Each URL prefix maps to one controller and one module, as today	The resource boundary equals the module boundary
Resource facade over module interfaces	A thin facade owns the resource and its URLs and calls each module through an interface	URLs and identity stay while module code moves behind the interfaces
Gateway routes sub-resources to owners	A gateway sends /payment-instructions/*/approvals to the Authorization service and other paths elsewhere	The owner of each sub-resource serves it behind the same URLs

Independent code needs interfaces; independent deployment needs a network

Solution	Gains	Costs
One controller per URL prefix	Easy to find the code No indirection Matches the URL tree	One module ends up owning others' logic Teams collide in one controller Splitting a module changes URLs
Resource facade over module interfaces	URLs stay stable Modules own their rules One place for the resource contract	The facade must stay thin Interfaces to design and test Cross-module calls need a transaction plan
Gateway routes sub-resources to owners	Independent deploys Same public URLs Clear owner per path	A network call and its failure modes Routing rules spread ownership Consistency across resources is now distributed

Deciding signal: Whether independent deployment is needed or only independent code ownership: measure cross-team pull request conflicts, release coupling and the share of instruction changes that touch more than one module. If the pain is code collision inside one deployable, interfaces behind a facade are enough; if teams must deploy separately, route by owner behind the same URLs

Smallest safe move

Facade owns the URLs, modules own the rules

- **Scope:** Behavior and code: the approvals route first; URLs, methods and status codes stay
- **Change:** A resource facade owns URLs and representation; Authorization handles approvals through an in-process interface; other modules follow route by route; no module reads another's tables
- **Safety net:** One deployable, so transactions hold; the facade composes and holds no rules; no one-to-one URL-to-module rule; recorded-traffic tests before every move
- **Escape route:** A per-route flag calls the old controller method; interface and module stay
- **Deferred:** Gateway routing of a sub-resource when a team must deploy separately and accepts network and consistency cost; data per module when ownership is clear

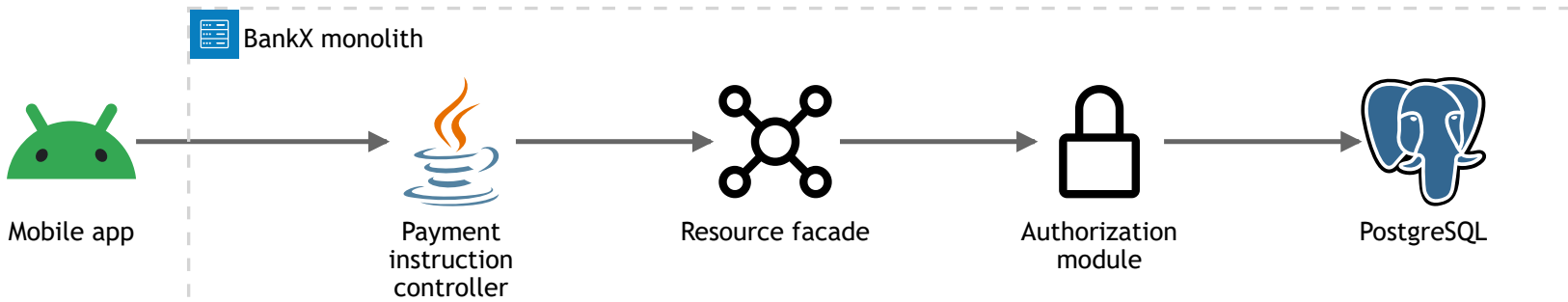
Verification: recorded traffic replayed on the old and the new code

Check	Pass condition
Recorded requests replayed on both versions	Identical status, headers and bodies for every route
Authorization team changes approvals only	No file outside its module changes; other modules' builds are unaffected
Pull requests touching the controller per month	Fall below the agreed number, for example 10
Module dependency check	No module reads another module's tables or internals
Instruction rules under concurrent load	Unchanged results: no duplicate approval and no lost update
Fitness function	A contract test replays recorded traffic in CI; an architecture test fails on forbidden module dependencies; a dashboard tracks files touched per team per pull request

Rollback: Routing rollback: the per-route flag calls the old controller method again and clients see no difference. State rollback: none; no data moved.

Stop: When teams change their modules without touching each other's code and URLs stay unchanged, no gateway routing or separate service is added

Solution architecture



Same client, controller and database; approvals now go through a resource facade that owns the URLs and representation and calls the Authorization module through an in-process interface, all in one deployable.

A resource path says what it relates to, not who owns it

Resource	Modules behind it	Why no one-to-one
`/payment-instructions/PI1`	Transfers, Accounts, Limits, Fraud, Compliance	One resource exposes a workflow coordinated across several domains
`/payment-instructions/PI1/approvals`	Authorization, Identity, Transfers, Audit	An approval relates to the instruction but has its own identity and lifecycle
`/transfers/T1/ledger-postings`	Transfers, Ledger	A relationship collection spanning operational and accounting concepts
`/operations/OP1`	Reporting, Queue, Worker, Object Storage	The resource shows processing state, not necessarily a business entity

The path is an identifier for the client. A deep path is not wrong by itself; trouble starts when its ancestry is read as ownership.

Moving a module out later does not require new URLs

Question	Answer
Can a module become its own service?	Yes, when a team must deploy separately and accepts the network and consistency cost; one API resource is not one microservice
Will the URLs change?	No. The facade or a gateway keeps the same URLs, methods and status codes
Where does the transaction live?	The resource boundary does not say. One deployable keeps today’s transactions; after a split, consistency is distributed

Transactional invariants, team ownership, coupling, change frequency and operational requirements decide the split, not the URL tree.

Glossary

Used in this step

Term	Definition
One controller per URL prefix	Code structure that mirrors the URL tree, one module per prefix
Resource facade over module interfaces	A thin layer that owns the resource's URLs and delegates to modules
Gateway routes sub-resources to owners	Routing rules that send each path to the service that owns it
Resource facade	Thin layer that owns a resource's URLs and representation and delegates to modules
Module interface	The contract through which one module is called by another
Resource graph	The linked resources a client sees, independent of code structure
Modular Monolith	One deployable divided into modules with enforced boundaries
Bounded Context	Boundary inside which a domain model and its language stay consistent
Aggregate	Cluster of records changed together under one consistency rule
Architecture test	Automated check of the dependency rules between modules
Team autonomy	Team can change and release its owned capabilities independently

Related concepts

Term	Definition
API Gateway	Central entry point for routing, authentication, rate limiting, and API policies.
Anti-Corruption Layer (ACL)	Protects a domain model from incompatible external system models.
Backend for Frontend (BFF)	API layer tailored to a specific frontend or client.
Database Per Service	Each service owns its private persistence and schema

API21 · Boundaries and distributed consistency

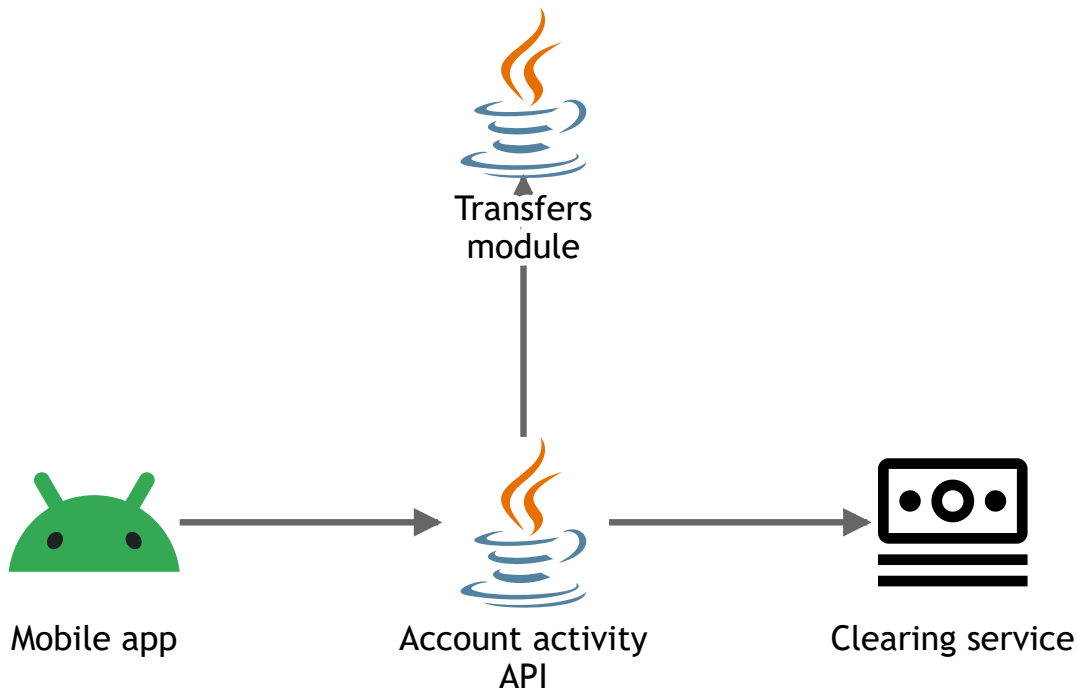
Serve account activity without live fan-out

Account activity calls 5 services live and its p95 exceeds 500 ms, against a target of ≤ 200 ms with the age of the data stated.

Account activity is composed live from 5 services

Component	What it is	What it does
Mobile app	BankX customer app on the customer’s phone	Opens account activity and reads GET /accounts/A1/transfers
Account activity API	The GET /accounts/A1/transfers route of the BankX API	Calls the backends one after another and merges the answers
Backend services	Transfers, Clearing status, Ledger postings, Fraud holds and Standing orders	Each is called live for every account activity request
Remote calls	Clearing, Fraud and Ledger already run as separate services behind the API	Latency adds up across calls and one failure fails the page

Current architecture



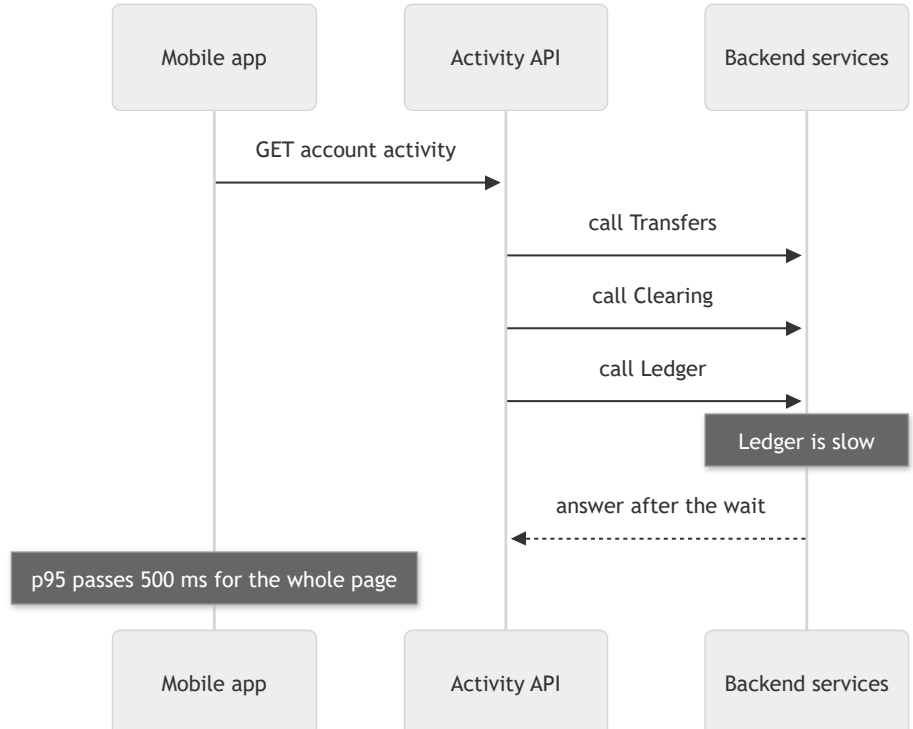
One slow service delays the page and one failing service fails it

A customer opens account activity. The API calls five services one after another; one slow service delays the whole page, and one failing service fails it.

> 500 ms
p95 of account activity, from remote calls

5 services
called live for every account activity request

≤ 200 ms
p95 of account activity, with the age of the data stated



Ways to serve a list that spans several services

Solution	What it is	What it does
Request-time aggregation	The API calls each backend per request and merges the answers	Shows the latest data from every source
CQRS projection	A separate read model is built from events and serves the activity queries	Answers from one store at read time
Materialized view	A table in the same database refreshed from the sources on a schedule or on change	Serves precomputed activity rows

Freshness is traded for latency and for isolation from failures

Solution	Gains	Costs
Request-time aggregation	Fresh data No new store Simple to reason about	Latency adds up across calls One failure fails the page Load multiplied by five
CQRS projection	One fast read Independent of backend failures Shaped for the page	Stale until events arrive Lag and rebuild complexity Event delivery must be reliable
Materialized view	Efficient reads No event platform Familiar SQL	Refresh cost and lag Works only where data shares a database A failed refresh shows old rows

Deciding signal: The staleness the business accepts against the latency of remote calls: measure p95 per backend call, the fan-out count and the age customers tolerate on activity rows. If the sum of backend latencies exceeds the budget and a few seconds of lag is acceptable, a projection pays; if data must be fresh to the second, keep aggregation and parallelize or cache the calls

Smallest safe move

A projection that states its age

- **Scope:** Contract, behavior and data: only GET /accounts/A1/transfers, plus an asOf field
- **Change:** Build a per-account projection from transfer, clearing and ledger events; the list answers with asOf and canonical links; the detail view keeps request-time aggregation; rebuild from events
- **Safety net:** Settled only after the settlement event; asOf states the age; canonical resources are the source of truth; writes never use the projection
- **Escape route:** A flag routes the list back to request-time aggregation; the projection keeps updating
- **Deferred:** Materialized view if data shares one database; request-time aggregation with parallel calls while the budget is met; a cache before the projection

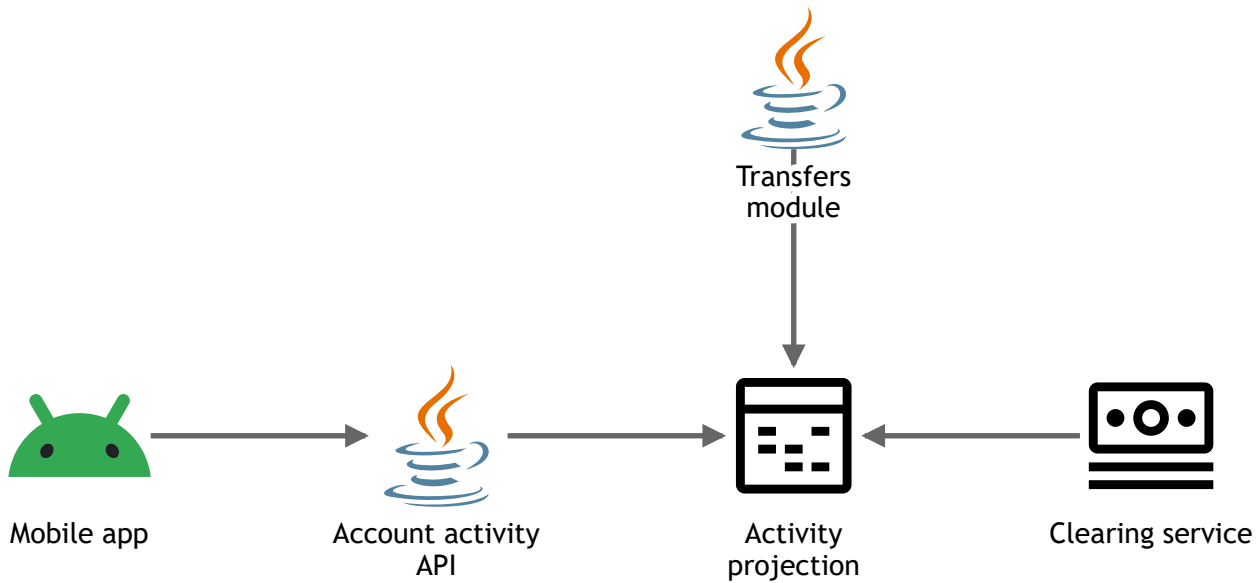
Verification: one backend slowed or cut, projection compared with the sources

Check	Pass condition
p95 of account activity at normal load	≤ 200 ms, independent of backend latency
One backend slow or unavailable	Activity still answers from the projection
Event processing lag	p99 ≤ 5 s, reflected in asOf
Projection compared with the sources	No transfer shown settled without a settlement event
Rebuild from events	Produces the same rows
Fitness function	A nightly comparison of projection and sources fails the build on a mismatch; an alert fires when lag exceeds the freshness promise

Rollback: Routing rollback: the flag returns the list to live aggregation and the old latency returns. State rollback: the projection is derived data and can be rebuilt from events; nothing in the sources changed.

Stop: When activity meets the latency target with an honest asOf, no search engine, data warehouse or second projection is added

Solution architecture



Same client, API and services; the backends feed events into an activity projection, and the list reads only that projection instead of calling five services live.

The options differ in freshness, failure behavior and rebuild

Option	Freshness	When a backend fails	Rebuild
Request-time aggregation	Latest data from every source	One failure fails the page	Nothing stored, nothing to rebuild
CQRS projection	Stale until events arrive; shown in `asOf`	Unaffected: the page answers from the read model	Replay events to rebuild the rows
Materialized view	As old as the last refresh	A failed refresh shows old rows	Refresh again; the sources must share one database

A projection is a promise about freshness, so the response states how old its data is.

A list that states its age and links to the canonical transfers

```
GET /accounts/A1/transfers?role=outgoing&from=2026-10-01 HTTP/1.1

HTTP/1.1 200 OK
{ "asOf": "2026-11-03T09:14:07Z",
  "items": [
    { "href": "/transfers/T1", "status": "settled" },
    { "href": "/transfers/T2", "status": "accepted" }
  ] }
```

The list is read from the projection; the transfers it names stay the source of truth.

Field	What the client may conclude
`asOf`	The rows reflect the state at this moment, not later
`href`	The canonical transfer holds the authoritative state
`status`	`settled` appears only after the settlement event

Glossary

Used in this step

Term	Definition
Request-time aggregation	Merging the answers of several backends while serving one request
CQRS projection	A read model built from events and shaped for one query
Materialized view	A stored query result that the database refreshes
Fan-out	Number of calls needed to build one response
Read model	Read-optimized representation built from updates or events
Eventual consistency	Copies converge after a delay, not immediately
asOf timestamp	Field that tells how fresh a returned view is
p95	Latency that 95% of requests stay under
Canonical resource	The authoritative resource that a derived view links to
Event Replay	Reconsume historical records to rebuild a projection

Related concepts

Term	Definition
CQRS	Separates models for reading data and modifying state.
Aggregated REST endpoint	One REST endpoint that composes several resources into one response
Backend for Frontend (BFF)	API layer tailored to a specific frontend or client.
Domain Event	Statement that an important business fact occurred

API22 · Boundaries and distributed consistency

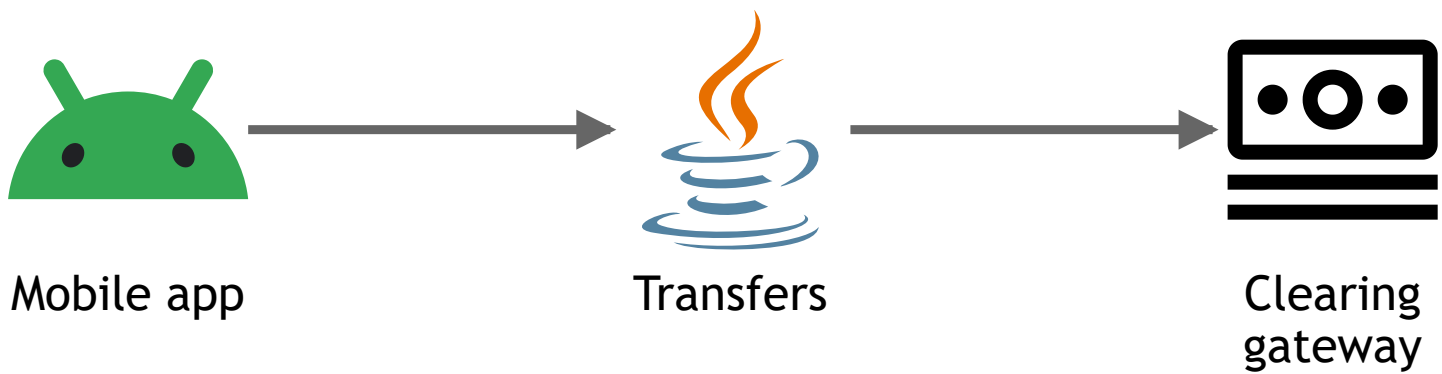
Choose the interface style for internal commands

Every submit to the clearing gateway takes three REST round trips for one internal consumer team: 3 calls against a hop budget of ≤ 20 ms.

Transfers submits to clearing through REST resources

Component	What it is	What it does
Mobile app	BankX customer app on the customer’s phone	Submits a transfer through the public API and never calls clearing
Transfers	The Transfers module of the Java monolith, owner of the transfer lifecycle	Submits an approved instruction to clearing through three REST calls
Clearing gateway	Separate service in front of external clearing, called only by Transfers	Creates a clearing item, accepts its payload and reports its status
Mapping layer	Code in Transfers that turns one transfer into clearing item resources	Maps about 20 fields into three calls that only one team consumes

Current architecture



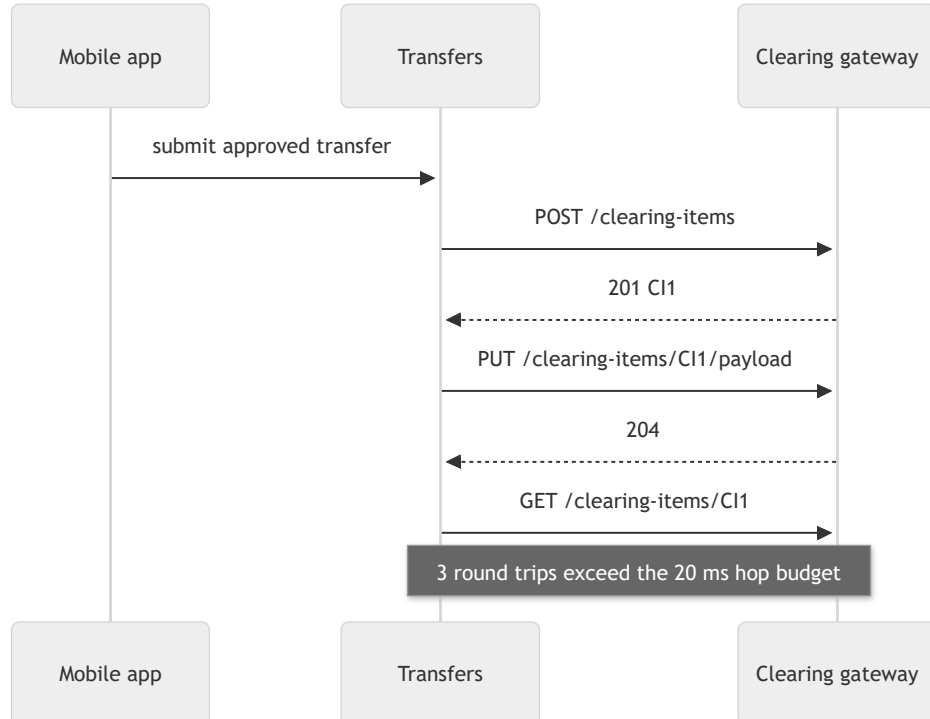
Three round trips serve one internal consumer

Transfers submits an approved instruction to the clearing gateway: create an item, attach the payload, ask for acceptance. The three calls cost a round trip each and a mapping layer that only one team consumes. The team asks whether REST everywhere is a rule or a habit.

3 calls
round trips to submit one transfer to clearing

1 consumer
team that calls the gateway; no browser or partner involved

≤ 20 ms
hop budget for one submit



Interface styles for an internal command

Solution	What it is	What it does
REST resources	Clearing items are resources created and read with HTTP methods	A uniform interface in the same style as the public API
gRPC	Typed RPC over HTTP/2 with Protocol Buffers and one call per command	Calls a named operation through a generated client
Message queue command	Transfers publishes a submit command to a queue that the clearing gateway consumes	Acknowledges once the command is stored and processes it later

Each style moves cost between caller, callee and tooling

Solution	Gains	Costs
REST resources	Same tools as the public API Resources are inspectable Loose coupling to consumers	Several round trips per command Mapping layer for one consumer Hypermedia adds nothing for one team
gRPC	One call per command Typed generated contracts Efficient on the wire	Tooling and browser limits Caller and callee must both be up Binary contract needs schema tooling
Message queue command	Gateway downtime does not stop acceptance Buffers bursts Retries are built in	No immediate result Duplicates need idempotency Another broker to run

Deciding signal: Whether the caller needs an immediate result and whether caller and callee can be unavailable at different times, plus who the consumers are: measure round trips per command, hop latency against the budget, the number of consumer teams and whether browsers call it. One internal consumer that needs the answer now favors typed RPC; if the gateway is down for maintenance (for example 15 minutes a month) while submissions must continue, a queue is needed

Smallest safe move

One typed Submit call, keyed by transfer id

- **Scope:** Contract and code: Transfers to Clearing submit and status calls only
- **Change:** Define a clearing contract with Submit and GetStatus; Transfers calls Submit once per transfer, the transfer id as idempotency key; REST items stay behind a flag
- **Safety net:** A retried Submit never submits twice; calls have a deadline and retry with backoff; services authenticate each other; no gRPC toward browsers or partners
- **Escape route:** The flag restores REST submit; both paths write one record
- **Deferred:** Message queue command when downtime exceeds the agreed minutes or accepted instructions wait; REST resources when a second consumer team needs a self-describing interface

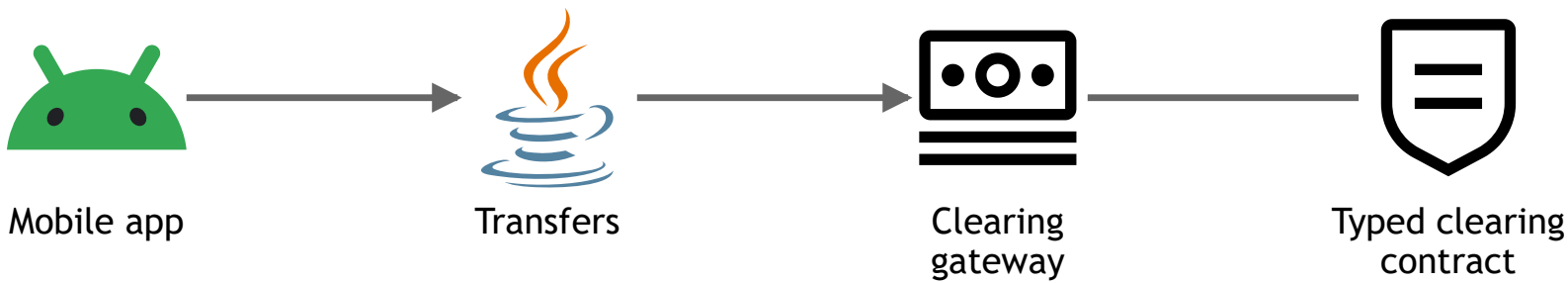
Verification: one call per submit, one item per transfer

Check	Pass condition
Submit of one transfer	1 call; hop p95 ≤ 20 ms
The same Submit sent three times	One clearing item
Gateway unavailable for 1 minute	Deadline exceeded within the budget; no transfer lost; retried later with the same key
Breaking change in the clearing contract	The contract test fails the build
Public API contract tests	Unchanged
Fitness function	A schema breaking-change check and a consumer contract test run in CI; a latency histogram on the Submit hop alerts above the budget

Rollback: Routing rollback: the flag sends submits through the REST items again. State rollback: clearing items created through either path are the same records, and a duplicate is stopped by the transfer id key.

Stop: When one call per submit meets the hop budget and duplicates are harmless, no message broker or service mesh is added

Solution architecture



Same client, Transfers and clearing gateway; the three REST calls become one typed Submit call that Transfers and the gateway share through a versioned contract.

Two questions choose the interface style

Immediate result needed?	Caller and callee unavailable at different times?	Style that fits
Yes	No; one internal consumer calls it	gRPC: one typed call per command
Yes	No; browsers, partners or several teams call it	REST resources: generic tools and discovery
Yes	Yes, but the caller may wait and retry	A synchronous call with a deadline and a retry with the same key
No, acknowledgement is enough	Yes, for example 15 minutes a month, and submits must continue	Message queue command: accept now, process later

REST stays for the public API. Inside the bank the style follows the consumer and the hop budget, so one internal consumer that needs the answer now favors typed RPC.

One typed Submit replaces three REST calls

```
// Illustrative contract, not a final schema
service Clearing {
  rpc Submit (SubmitRequest) returns (SubmitReply);
  rpc GetStatus (StatusRequest) returns (StatusReply);
}
message SubmitRequest {
  string transfer_id = 1;  // idempotency key
  Amount amount = 2;       // and the other fields
}
message SubmitReply { string item_id = 1; string status = 2;
message StatusRequest { string transfer_id = 1; }
message StatusReply { string status = 1; }
```

- `transfer_id` : one key per transfer, so a repeated `Submit` leaves one clearing item.
- **Deadline:** each call carries a time limit; after it expires Transfers retries with the same key.
- **Schema:** the contract is checked in CI, so a breaking change fails the build.

Glossary

Used in this step

Term	Definition
REST resources	Internal capabilities modelled as resources and reached with HTTP methods
gRPC	RPC framework commonly using HTTP/2 and Protocol Buffers
Message queue command	A command stored in a queue and processed by its consumer later
RPC	Operation invocation across a process or network boundary
Protocol Buffers	Binary schema language for typed messages
Deadline	Time limit passed with a call so that the callee can stop work
Temporal coupling	Caller and callee must both be available at the same time
Hop budget	Latency allowed for one internal call
Typed contract	Interface whose messages are defined by a schema and checked by tools
Command	A request to do something, as opposed to a query that only reads
Service-to-service authentication	Proof of identity between internal services

Related concepts

Term	Definition
Message Broker	Routes and delivers messages between producers and consumers.
Consumer-driven contract test	Consumer expectations verified against a provider
Idempotency Key	Stable identifier recognizing multiple attempts at the same logical command
Transactional Outbox	Atomically records business changes and outgoing events in the same database transaction.

API23 · Boundaries and distributed consistency

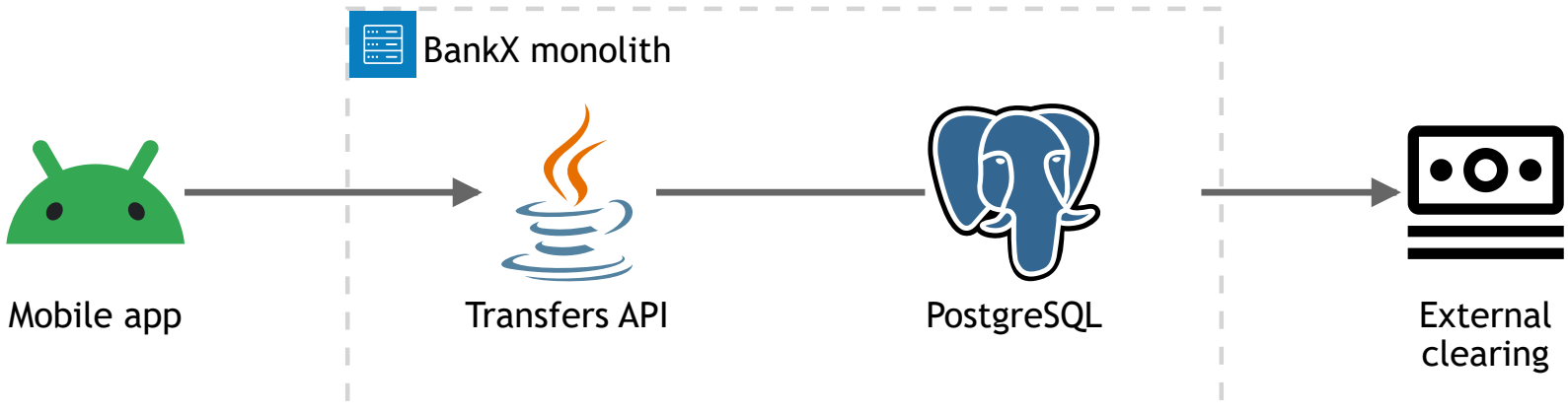
Reconcile what HTTP cannot make atomic

Timeouts and retries leave clearing results, transfers and ledger postings in disagreement: 17 unmatched movements per day against a target of 0 unexplained differences.

A transfer is recorded, cleared and posted in separate steps

Component	What it is	What it does
Mobile app	BankX customer app on the customer's phone	Submits a transfer and reads its status
Transfers API	The transfer routes of the Java monolith	Records the transfer, sends it to clearing and retries after a timeout
PostgreSQL	Database that holds transfers and ledger postings	Records the transfer and posts to the ledger in a separate step
External clearing	The clearing network that settles transfers	Settles a transfer once; a timeout can hide its acceptance
Operations	BankX operations department	Finds unmatched movements at night and asks which system is right

Current architecture



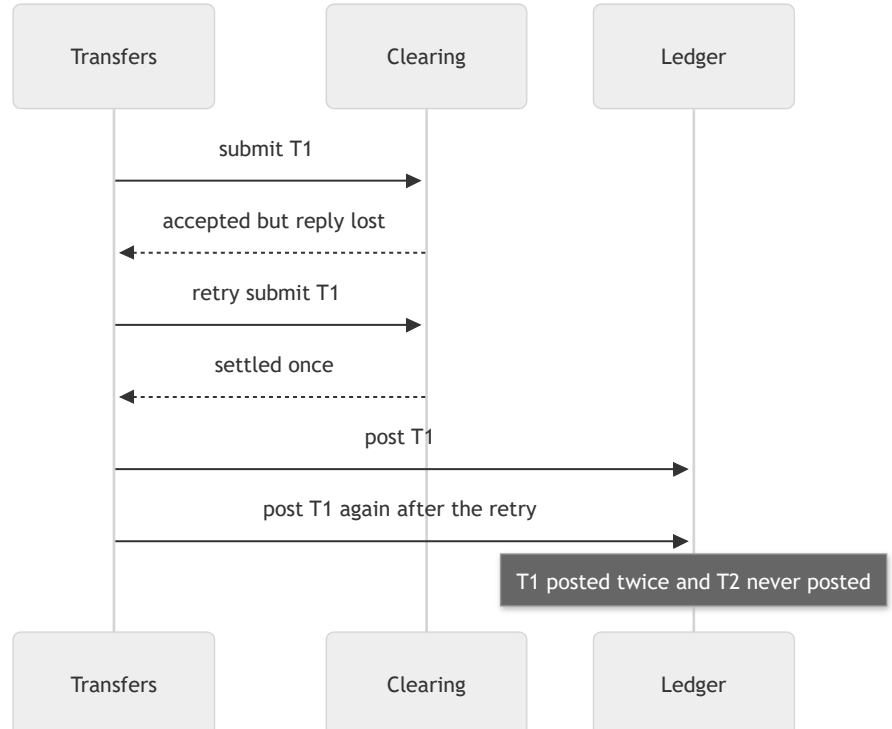
A hidden acceptance makes three systems disagree

A timeout hides a clearing acceptance. Transfers retries, clearing settles once, and the ledger posts twice for one transfer and not at all for another.

17 per day
movements with no matching counterpart across clearing results, transfers and postings

3 sources
clearing results, transfers and ledger postings are reconciled daily

0
unexplained differences at the end of each day



Ways to bring clearing, transfers and postings into agreement

Solution	What it is	What it does
Daily reconciliation	Compare clearing results, transfers and postings by key, amount and currency and report the differences	Finds divergence after the fact and guides the repair
Saga with compensation	A coordinator runs the steps and issues compensating actions when one fails	Repairs a failed workflow automatically and visibly
Synchronous confirmation before commit	Transfers calls clearing and the ledger inside one request and commits only if all succeed	Tries to make the three agree at commit time

Agreement is detected, repaired or promised, and each costs differently

Solution	Gains	Costs
Daily reconciliation	Catches every kind of divergence Hot path unchanged Gives audit evidence	Detects, does not prevent Repair is manual or scripted Delay of up to a day
Saga with compensation	Failure handled in the flow Visible workflow state A compensation per step	Compensations are new business logic Not every step can be undone A coordinator to operate
Synchronous confirmation before commit	Looks atomic to the client No later repair Simple to explain	No transaction spans HTTP and the clearing network A timeout leaves the outcome unknown Holds locks and workers

Deciding signal: Whether the parties can share a transaction and what an undetected difference costs: count unmatched movements per day, their age and value, and whether a safe compensating action exists for each step. Inside one database a local transaction is enough; across an external network start with detection by reconciliation and add a saga only when discrepancies are frequent and each has a safe automatic repair

Smallest safe move

Detect first: a nightly reconciliation run

- **Scope:** Behavior and data: a nightly job, a read-only discrepancy resource
- **Change:** A nightly job matches clearing results, transfers and postings by clearing reference, amount and currency, writes differences to /reconciliation-runs/RR1/discrepancies; settlement and postings commit in one local transaction
- **Safety net:** The job only reads; a repair is a logged command with a reason, not a silent edit; synchronous confirmation is not used, no transaction spans clearing
- **Escape route:** A flag switches the job off; no business data changes
- **Deferred:** Saga with compensation when discrepancies pass an agreed rate and each has a safe automatic repair; hourly runs if value at risk requires

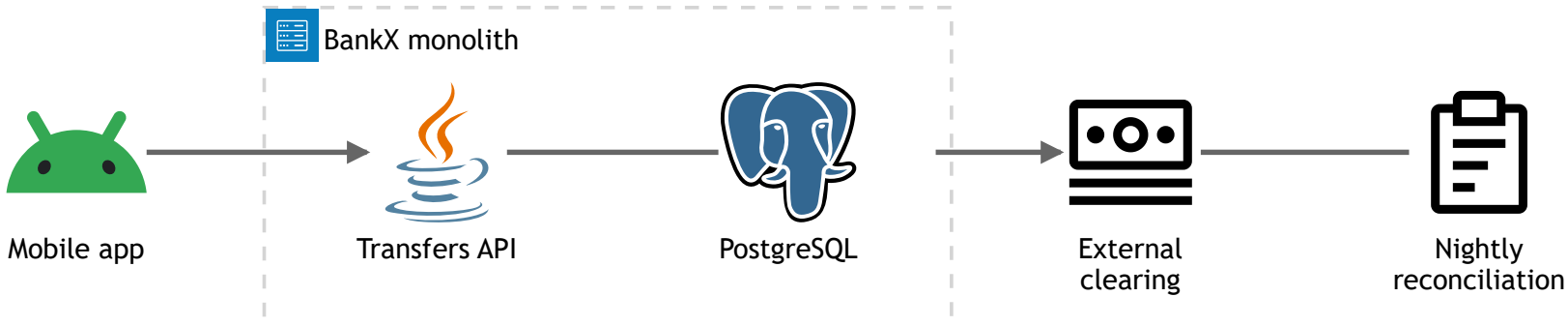
Verification: injected timeouts, duplicates and a lost posting

Check	Pass condition
Clearing timeout hides an acceptance	Found by the next run and classified as accepted-not-recorded
Duplicate clearing callback	One posting set; the duplicate is ignored by the clearing reference
Transfer settled without postings (injected)	Reported with amount and reference
Run on a clean day	0 discrepancies and a signed-off run record
Repair of an injected discrepancy	A logged command with a reason; the next run is clean
Fitness function	The run is a scheduled job that alerts when it fails to complete or leaves an unclassified difference older than one day; a test seeds known discrepancies and asserts that all are found

Rollback: Routing rollback: switch the job off, since it only reads. State rollback: repairs recorded by follow-up commands are new ledger entries and never deletions; a wrong repair is reversed by a new entry, so the history stays complete.

Stop: When every discrepancy is found within a day and cleared with a recorded repair, no saga coordinator or distributed transaction manager is added

Solution architecture



Same nodes plus a nightly reconciliation job that compares clearing results with transfers and postings and reports every difference; the hot path is unchanged.

Each kind of difference points to a cause and a repair

Discrepancy	Likely cause	Repair, as a logged command with a reason
Accepted, not recorded	A timeout hid the clearing acceptance	Record the settlement from the clearing evidence
Posted twice	A retry or duplicate clearing callback posted again	Reverse the extra posting set with a new entry
Settled without postings	A step failed between settlement and posting	Post the missing set under the clearing reference
Amount differs	Unknown until investigated; the movement may be disputed	Record the findings, then correct with a new entry

A repair is never a silent edit and never a deletion. A wrong repair is reversed by a new entry, so the history stays complete.

A nightly run and one discrepancy to read

```
GET /reconciliation-runs/RR1 HTTP/1.1

HTTP/1.1 200 OK
{ "date": "2026-11-01", "status": "completed", "discrepancie

GET /reconciliation-runs/RR1/discrepancies/D1 HTTP/1.1

HTTP/1.1 200 OK
{ "kind": "accepted-not-recorded", "clearingReference": "CLR
  "amount": "500.00", "currency": "PLN", "status": "open" }
```

- **Run:** one per business date, produced by the nightly job; reading it changes nothing.
- **Discrepancy:** a kind, the clearing reference, amount and currency, so operations can find the evidence.
- **Repair:** a separate logged command with a reason, never an edit of this resource.

Glossary

Used in this step

Term	Definition
Daily reconciliation	A scheduled comparison of clearing results, transfers and postings
Saga with compensation	A coordinated workflow that undoes completed steps when a later one fails
Synchronous confirmation before commit	Waiting for several systems inside one request before committing
Data Reconciliation	Compare systems using keys, checksums and domain invariants to find divergence
Discrepancy	A transfer, result or posting that has no matching counterpart
Local transaction	Atomic change inside one database
Distributed transaction	Atomic change across several independent systems
Compensation	Action that undoes the effect of an earlier step
Clearing reference	Identifier that links a transfer to its clearing result and postings
Ledger posting	Record of a financial movement with account, amount, currency, booking date and reference

Related concepts

Term	Definition
Saga Pattern	Coordinates distributed business transactions using local transactions and compensations.
Exactly-Once Effect	Application-visible effect occurs once despite repeated attempts under defined guarantees
Idempotency Key	Stable identifier recognizing multiple attempts at the same logical command
Audit history	Reviewable record of instructions, approvals and decisions

API24 · Boundaries and distributed consistency

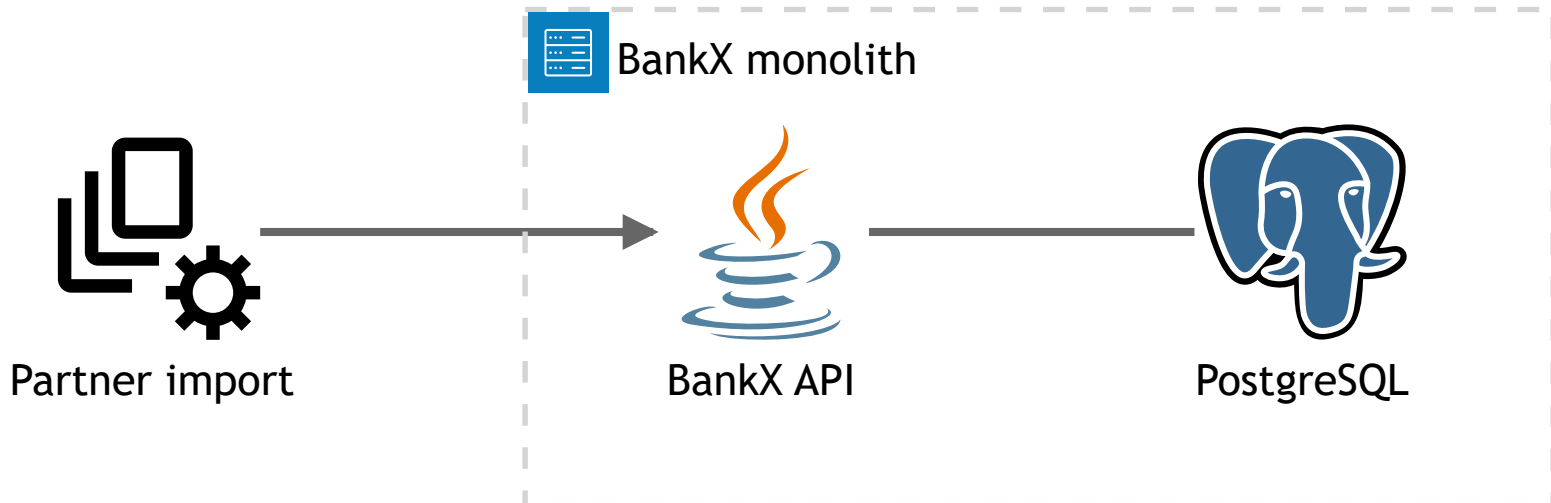
Stop adding API machinery no consumer needs

A partner that imports one report a day and releases with BankX is offered four mechanisms: 4 mechanisms for 1 consumer, against a target of ≤ 1 h to a finished report.

The partner imports a daily CSV report

Component	What it is	What it does
Partner import	Batch job of the partner team, one engineer, released together with BankX	Imports the daily account report from a CSV file once a day
BankX API	The reporting code of the Java monolith	Produces the daily CSV file for the partner
PostgreSQL	Database that holds accounts and ledger postings	Is the source of the report, which must match the ledger
Reporting API proposal	Hypermedia actions, operation resources, webhooks and a versioned media type	Would replace the CSV and cost the partner client code and BankX operations

Current architecture



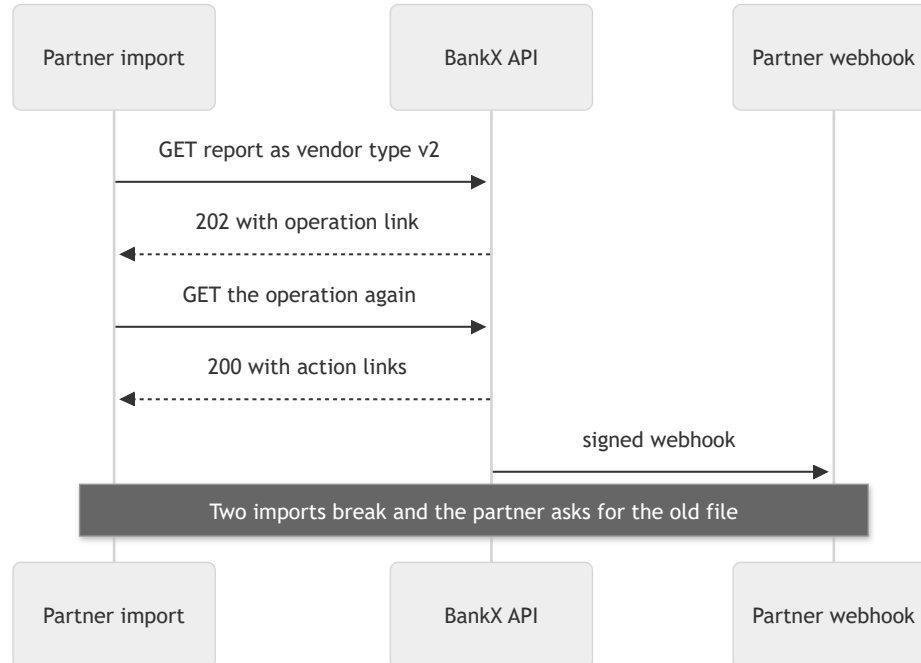
Four mechanisms are proposed for one consumer

A partner team imports a daily report with one engineer and releases together with BankX. The proposal adds four mechanisms; each costs client code and operations for no measurable gain.

4 mechanisms
hypermedia, operation resource, webhooks, versioned media type

1 consumer
partner team that imports daily and releases with the API

≤ 1 h
from end of business day to report available



Ways to serve a daily report to one partner

Solution	What it is	What it does
Plain resource endpoints + OpenAPI	A GET for the daily report resource, documented in an OpenAPI file, with an ETag and a fixed URL pattern	The partner fetches the finished daily report
Full hypermedia + async + webhooks	Siren actions, 202 operation resources, signed webhooks and a versioned media type	Gives the partner every mechanism that the other clients use
RPC-style operation list	A small set of named operations, such as reports.getDaily, with a typed schema	One call per need through a generated client

Each mechanism costs the partner client code and BankX operations

Solution	Gains	Costs
Plain resource endpoints + OpenAPI	Smallest client Documented contract Same tools as the rest of the API	Client knows the URL pattern No push, the partner polls daily A workflow change needs a joint release
Full hypermedia + async + webhooks	Client independence Asynchronous generation Push on completion	Four mechanisms for one consumer Client and operations complexity Larger representations and indirection
RPC-style operation list	Few calls Typed contract Clear operation names	Less visible to generic tools Own caching and error rules Another style to document

Deciding signal: Whether any benefit of the extra mechanisms is measurably needed: count independent consumers, release coordination, report generation time against the request budget and the change rate of the workflow. One consumer that releases with the API, a report ready in minutes and a stable workflow mean that nothing beyond a documented resource is needed; each trigger that fires brings one mechanism back

Smallest safe move

One documented GET resource, nothing more

- **Scope:** Contract and clients: one report resource, one OpenAPI file
- **Change:** GET /partners/P1/daily-reports/2026-11-02 returns the report with an ETag and Cache-Control private; OpenAPI is the contract; a contract test from the partner's expectations runs in CI
- **Safety net:** Authorization checks the partner's own credential on every request; the report is published only after generation; every left-out mechanism has a trigger
- **Escape route:** Add a mechanism later; the resource stays
- **Deferred:** Operation resource when generation exceeds the budget; webhook when a push is needed; hypermedia when a second client appears or the workflow changes often; versioned media type on an incompatible change

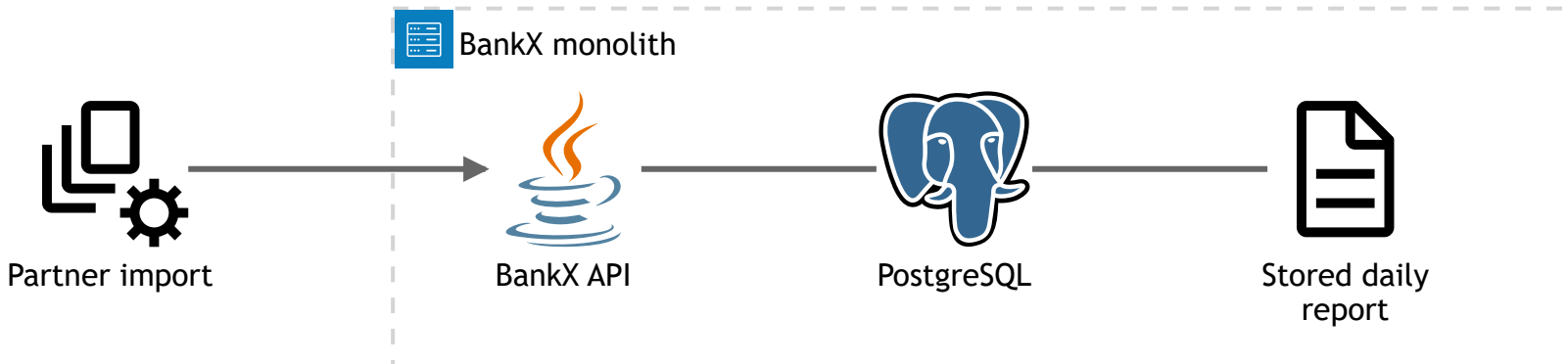
Verification: two weeks of the partner’s real daily import

Check	Pass condition
Daily import for 14 days	Every report is available before the partner’s import time; 0 failed imports
Report total compared with the ledger	Equal for every account and day
Another partner requests the report	403 and no content
Contract test from the partner’s expectations	Green on every release
Triggers for each left-out mechanism	Monitored: generation time, consumers and workflow change rate
Fitness function	A daily check compares the report with the ledger; triggers are metrics with thresholds, such as generation above 60 s or a second consumer, that open a review

Rollback: Routing rollback: remove the route and the partner falls back to the daily CSV it uses today. State rollback: none; the endpoint only serves stored reports.

Stop: When the plain endpoint meets the budget and the partner’s imports succeed, no operation resource, webhook, hypermedia or versioned media type is added; this is where the progression’s machinery ends

Solution architecture



Same partner, API and database; the CSV becomes one documented GET resource that serves the stored daily report with an ETag, and no other mechanism is added.

Each objection is answered by a trigger, not by adding the mechanism

Objection	Answer
Without hypermedia the partner hardcodes a URL	One consumer that releases with the API accepts a joint release; hypermedia returns when a second independent client appears or the workflow changes often
A plain report cannot cope with slow generation or a push	The operation resource returns when generation exceeds the request budget, the webhook when the partner needs a push
A custom media type protects against future change	It returns when the representation must change incompatibly; until then the OpenAPI file and a contract test guard the contract

REST is a set of constraints on how independently evolving components interact. Its benefits come with costs in representation size, indirection and client complexity.

Each mechanism returns when a measured trigger fires

Mechanism	Trigger that brings it back	Until then
Operation resource	Generation exceeds the request budget, for example generation above 60 s	The report is generated before it is published
Webhook	The partner needs a push when a report is ready	The partner polls the daily URL once a day
Hypermedia	A second independent client appears, or the workflow changes often	The partner knows the URL pattern; a change ships in a joint release
Versioned media type	The representation must change incompatibly	One representation, described in the OpenAPI file

A trigger is a metric with a threshold that opens a review: generation time, number of consumers and workflow change rate. When one fires, one mechanism returns and the resource stays.

Glossary

Used in this step

Term	Definition
Plain resource endpoints + OpenAPI	Documented GET resources with standard HTTP semantics and no extra machinery
Full hypermedia + async + webhooks	Advertised actions, operation resources and callbacks used together
RPC-style operation list	A small set of named operations with a typed schema
OpenAPI document	Machine-readable description of an HTTP API's routes and messages
Uniform interface	Standard method meanings that generic clients and intermediaries understand
Hypermedia	Responses that advertise the links and actions available next
Operation resource	Resource that reports the progress and outcome of a long-running request
Webhook	HTTP callback that one system sends to another when an event happens
Custom media type	A vendor-specific media type that names a representation and its version
Release coordination	Need for two parties to ship changes together
Trigger	Measured condition that brings a left-out mechanism back

Related concepts

Term	Definition
REST	Resource-oriented interface style using uniform HTTP semantics
RPC	Operation invocation across a process or network boundary
Richardson Maturity Model	Levels of HTTP API design; not a formal REST compliance test
Fitness function	Measurable or executable architectural quality rule